





Introduction



Ce diaporama est à votre disposition pour vous permettre deux choses :

1. Appliquer la charte graphique à vos diapositives existantes
2. **Utiliser des exemples de slides correspondant à chaque étape clé de votre animation de formation**

!!! Respectez les sections de diapositives

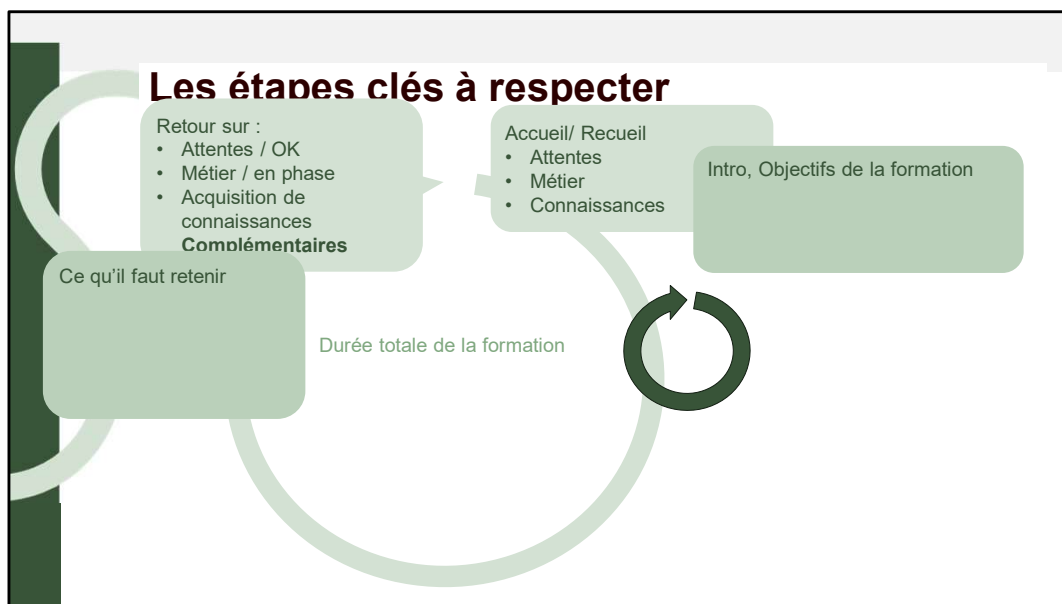


Procédure d'application du thème au jeu de slides existant

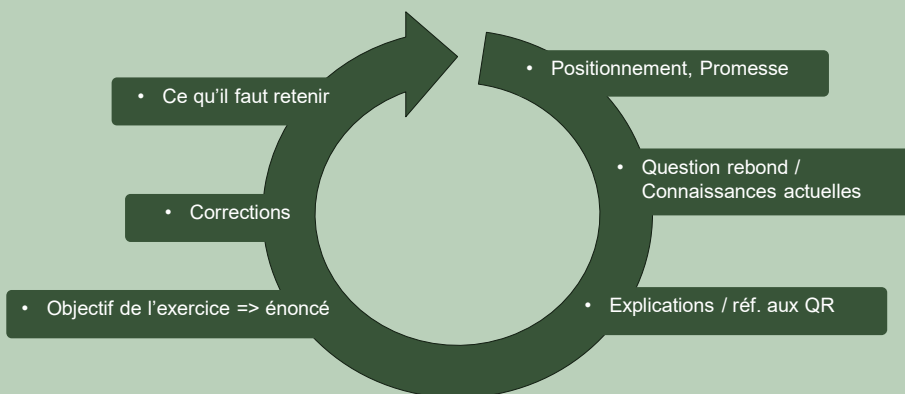
1. Enregistrer le Thème de ce diaporama dans vos thèmes personnels
2. Ouvrir votre diaporama et appliquer le thème ib Cegos
3. Si besoin, modifiez les dispositions de diapos (en sélection multiple si besoin)

Les étapes clés à respecter

- Module 0 (introduction générale) :
 - Diapo de démarrage pour l'accueil des stagiaires dans la salle
 - Diapo de consigne d'affichage vidéo micro etc (pour animation à distance)
 - Diapo d'émargement
 - Tour de table, présentations du contenu de stage logistique etc....
Relever les attentes de chaque stagiaire (Tableau blanc Teams)
- L'accueil de chaque journée supplémentaire :
 - Echanges, Question Réponses
- Pour chaque Module
 - Diapo de lancement d'un sujet avec interrogation des connaissances actuelles des stagiaires
« L'objectif de ces interrogations est de disposer de questions rebonds qui vous permettront de présenter les choses en lien avec les connaissances actuelles de vos stagiaires »
 - Diapo d'énoncé d'atelier
 - Diapo de synthèse & échanges (Essentiel à retenir) du chapitre ou de la demi-journée ou de la journée
- Fin de formation :
 - Diapo d'évaluation de fin de formation
 - Bilan des acquis en rapport aux attentes, conseils de suivi de formation supplémentaire ou réorientation



Les étapes clés à respecter



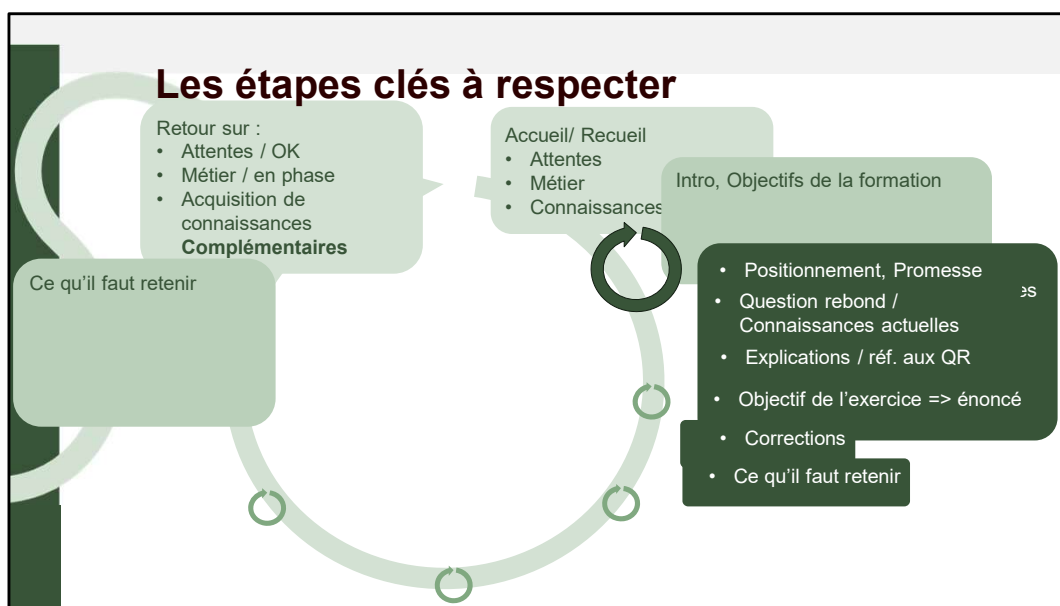
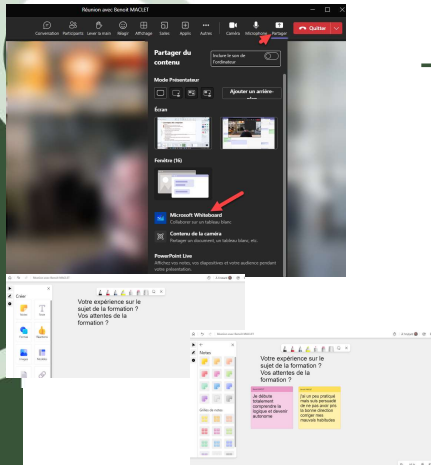


Tableau blanc collaboratif dans Teams

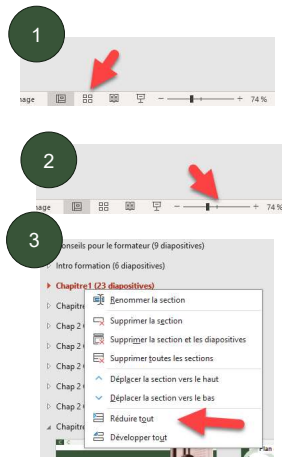


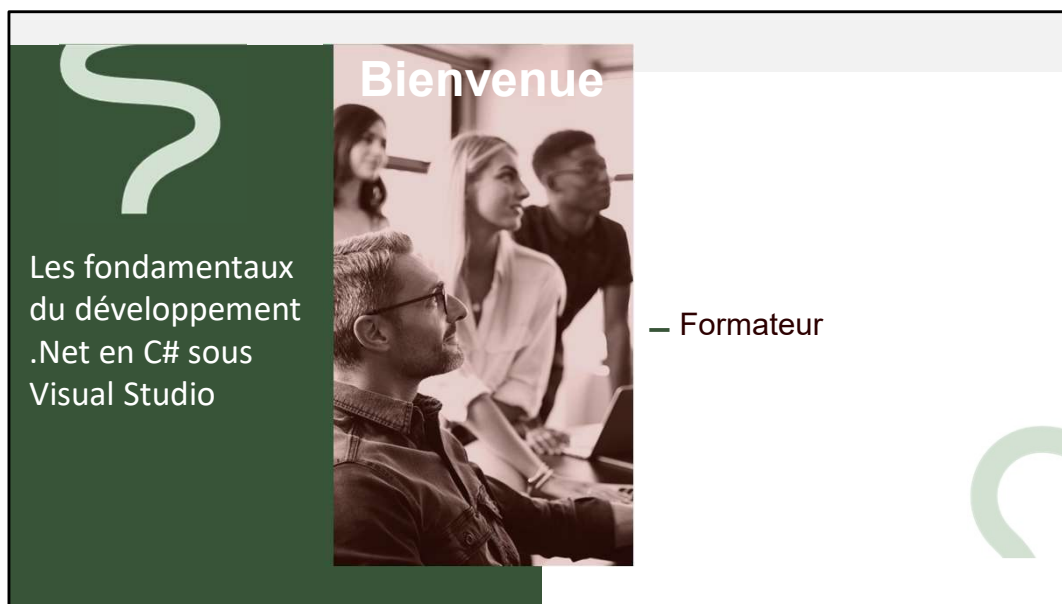
- Préparer un tableau blanc collaboratif d'accueil des stagiaires
 - Dans la session, Partagez Microsoft Whiteboard
 - Si c'est votre 1^{er} tableau blanc :
 - Cliquez sur « Nouveau »
 - Si non, utilisez un ancien tableau blanc proposé dans la liste des historiques
 - Pour guider les stagiaires, insérez 2 blocs textes pour indiquer les questions suivantes :
 - Votre expérience sur le sujet de la formation ?
 - Vos attentes de la formation ?
 - Invitez les stagiaires à utiliser l'outil « Notes » pour choisir leur couleur de post-it et l'ajouter sur le tableau avant de répondre aux 2 questions

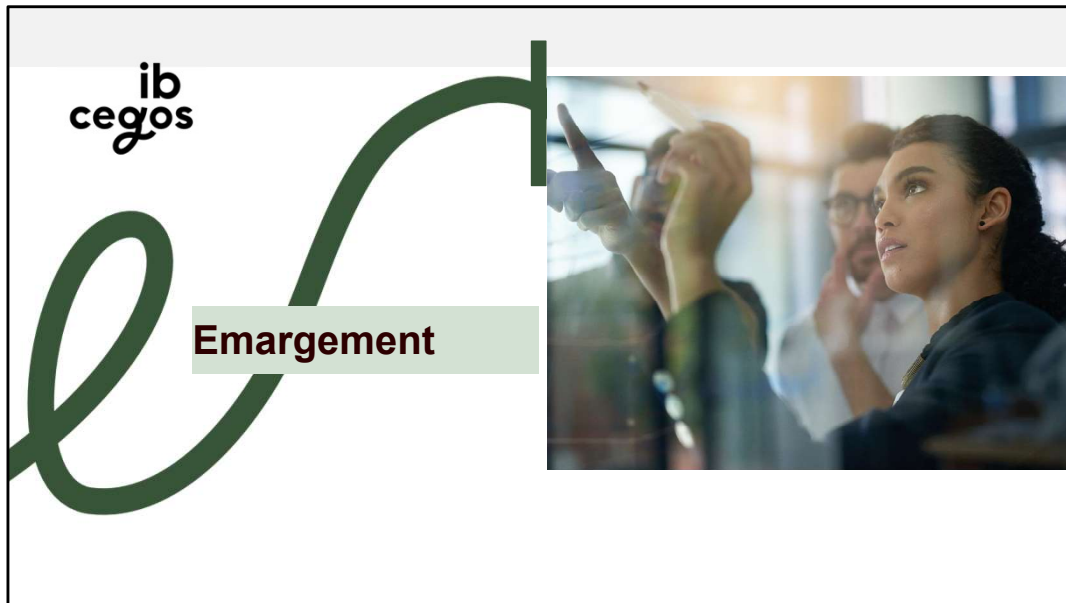
Naviguer dans le jeu de slides

— Une organisation par sections

- Les diapositives sont organisées dans des sections
- Pour faciliter la navigation dans les différentes diapositives,
 1. Activez l'affichage par trieuse (en bas de l'écran)
 2. Zoomez dézoomez pour avoir une vue plus ou moins précise des vignettes de diapositives.
 3. Selon vos besoins, vous pouvez réaliser un clic droit sur une section pour réduire une ou toutes les sections
 4. De façon générale le nom des sections respecte le chapitrage de la formation







Consignes pour le formateur :

1. Attendre que tout le monde soit présent
si nécessaire, retarder un peu cette étape d'émargement
2. Dans l'appli Bienvenue Formation <https://formateur.bienvenue.pro/>
: **Pointer les stagiaires présents**
3. **Lancer un émargement collectif**
4. **Afficher le QR Code aux stagiaires**

Indiquez-leur qu'ils devraient retrouver leur nom dans l'appli mobile avant d'émarger
sur l'écran de leur Smartphone

Avant de démarrer

○ Règles de fonctionnement



Si possible, pour faciliter les échanges :
Activez votre webcam



Pour le confort de tous
Désactivez votre micro



Pour vous accompagner durant les ateliers
Partagez votre écran



Pour échanger, partager...
Utilisez l'outil de conversation (Tchat)



— Durée de la formation : 2 Jours

— Organisation

- ▶ Horaires - 9h à 17h-17h30
- ▶ Pauses : 15 min en matinée et après midi (vers 10h30 et 15h30)
- ▶ Déjeuner : 12h30 à 14h00



Avant de commencer

- Priorité aux échanges
(durant les ateliers également)
- Tour de table :
 - Prénom, nom, Société, Titre/fonction
Dans le tableau blanc Teams :
 - Votre expérience sur le sujet de la formation ?
 - Vos attentes de la formation

Plan de la formation

- 1 - Introduction à C# et .Net
- 2 - Structure de programmation C#
- 3 - Déclaration et appel de méthodes
- 4 - Gestion d'exceptions
- 5 - Lire et écrire dans des fichiers
- 6 - Création de nouveaux types de données
- 7 - Encapsulation de données et de méthodes
- 8 - Héritage de classes et implémentation d'interfaces
- 9 - Gestion de la durée de vie des objets et contrôle des ressources
- 10 - Encapsulation avancée
- 11 - Découplage de méthodes et gestion d'évènements
- 12 - Utilisation des collections et construction de types génériques
- 13 - Programmation asynchrone et personnalisation du code
- 14 - Utilisation de LINQ pour interroger des données
- 15 - Développement dirigé par les Tests

Objectifs:

1. Maîtriser les concepts fondamentaux de la programmation en C# et du framework .NET.
2. Savoir utiliser efficacement l'IDE Visual Studio 2022 pour créer, documenter, exécuter et déboguer des applications.
3. Comprendre les principes de la programmation orientée objet (POO) dans le contexte de C# et comment les appliquer pour construire des applications robustes et maintenables.
4. Acquérir des compétences en gestion d'exceptions, manipulation de fichiers, et gestion de la mémoire avec le Garbage Collector.
5. Être en mesure d'utiliser les fonctionnalités avancées de C# telles que LINQ, la programmation asynchrone, et le développement dirigé par les tests.

Positionnement: Cette formation est destinée aux développeurs et chefs de projet techniques ayant une expérience préalable en programmation, notamment avec des langages tels que C, C++, VB ou Java. Elle a pour objectif de fournir une introduction solide aux fondamentaux de la programmation en C# et du framework .NET, tout en utilisant Visual Studio 2022 comme environnement de développement.

Progression / Fil rouge de la formation: La formation débute par une introduction à .NET et C#, suivie d'une immersion progressive dans la structure de programmation de C#. Elle guide ensuite les apprenants à travers des concepts plus avancés tels que la gestion d'exceptions, la manipulation de fichiers, et la création de types de données

personnalisés. Vers la fin de la formation, les participants exploreront des fonctionnalités plus avancées et contemporaines de C#, notamment LINQ, la programmation asynchrone, et les techniques de développement dirigé par les tests.

À la fin de cette formation, les stagiaires sauront:

1. Construire des applications robustes et efficaces en C# sous Visual Studio 2022.
2. Utiliser les principaux outils de Visual Studio pour déboguer et optimiser leurs codes.
3. Appliquer les principes de la POO pour créer des classes, des interfaces et des structures en C#.
4. Manipuler des fichiers, gérer des exceptions, et contrôler la mémoire de manière efficace.
5. Exploiter les fonctionnalités avancées de C# pour écrire du code plus propre, plus performant et plus maintenable.
6. Intégrer des techniques de développement dirigé par les tests pour garantir la robustesse et la fiabilité de leurs applications.



Environnement de travail

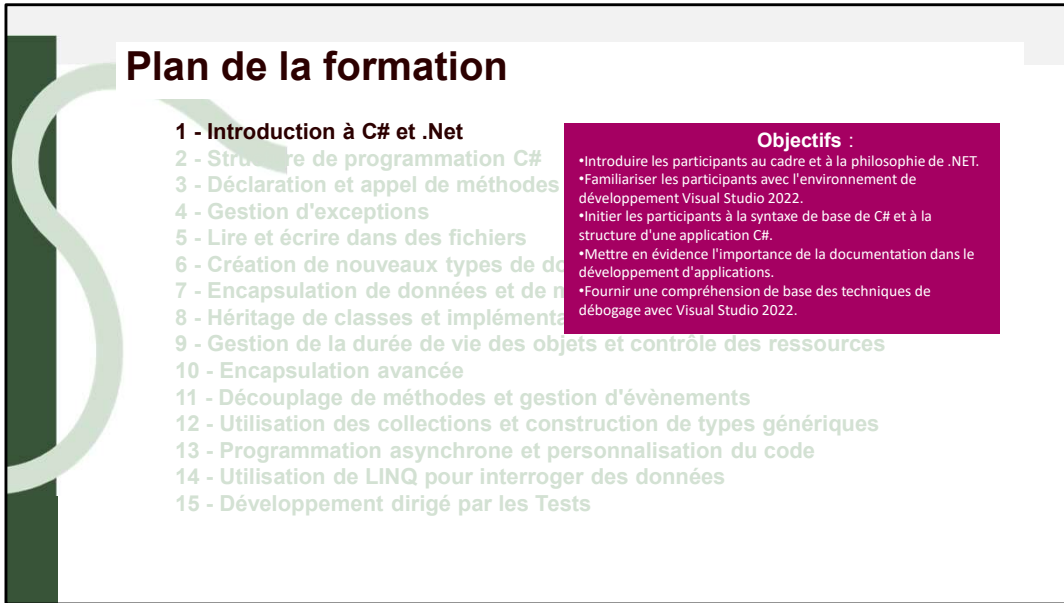
Pour réaliser les ateliers nous allons utiliser

- Visual Studio 2022 Community Edition
- .NET 6

Il serait également possible de travailler avec
Visual Studio Code et l'utilitaire *dotnet* fourni avec
.NET 6

Chapitre 1 - Introduction à C# et .Net





Plan de la formation

- 1 - Introduction à C# et .Net
- 2 - Structure de programmation C#
- 3 - Déclaration et appel de méthodes
- 4 - Gestion d'exceptions
- 5 - Lire et écrire dans des fichiers
- 6 - Création de nouveaux types de données
- 7 - Encapsulation de données et de méthodes
- 8 - Héritage de classes et implémentation
- 9 - Gestion de la durée de vie des objets et contrôle des ressources
- 10 - Encapsulation avancée
- 11 - Découplage de méthodes et gestion d'événements
- 12 - Utilisation des collections et construction de types génériques
- 13 - Programmation asynchrone et personnalisation du code
- 14 - Utilisation de LINQ pour interroger des données
- 15 - Développement dirigé par les Tests

Objectifs :

- Introduire les participants au cadre et à la philosophie de .NET.
- Familiariser les participants avec l'environnement de développement Visual Studio 2022.
- Initier les participants à la syntaxe de base de C# et à la structure d'une application C#.
- Mettre en évidence l'importance de la documentation dans le développement d'applications.
- Fournir une compréhension de base des techniques de débogage avec Visual Studio 2022.

Objectifs:

1. Introduire les participants au cadre et à la philosophie de .NET.
2. Familiariser les participants avec l'environnement de développement Visual Studio 2022.
3. Initier les participants à la syntaxe de base de C# et à la structure d'une application C#.
4. Mettre en évidence l'importance de la documentation dans le développement d'applications.
5. Fournir une compréhension de base des techniques de débogage avec Visual Studio 2022.

Positionnement: Ce chapitre sert de point d'entrée pour la formation, posant les bases nécessaires pour comprendre et naviguer dans l'écosystème de .NET et C#. Il est conçu pour des participants ayant déjà une certaine expérience en programmation, mais qui pourraient être nouveaux dans le monde de .NET et C#. En fin de chapitre, les participants devraient se sentir à l'aise avec l'IDE Visual Studio 2022 et avoir une compréhension de base de la manière dont les applications C# sont construites et structurées.

Sujets clés qui seront couverts:

1. Présentation de .NET :

1. Historique et évolution de .NET.
2. Composants et architecture de .NET.
3. La plateforme .NET Core vs .NET Framework.

2. Création de projets avec Visual Studio 2022 :

1. Navigation dans l'interface de Visual Studio.
2. Création d'un nouveau projet et exploration des modèles de projet disponibles.
3. Compréhension de la structure d'un projet .NET.

3.Écriture d'une application C# :

1. Introduction à la syntaxe de base de C#.
2. Structure d'une application C# : méthodes, classes, espaces de noms.
3. Premier "Hello World" en C#.

4.Documentation d'une application :

1. Importance de la documentation pour la maintenance et la collaboration.
2. Comment utiliser les commentaires en C#.
3. Introduction à XML Documentation Comments pour la génération de documentation automatisée.

5.Exécution et débogage d'applications avec Visual Studio 2022 :

1. Utilisation du débogueur intégré.
2. Comprendre les points d'arrêt, la surveillance des variables et le pas à pas.
3. Gestion des erreurs courantes et compréhension des messages d'erreur.

Commençons par échanger



Expériences précédentes avec .NET et C# :

"Parmi vous, qui a déjà eu l'occasion de travailler ou d'expérimenter avec la plateforme .NET ou le langage C# ? Si oui, dans quel contexte ou projet l'avez-vous utilisé ?"

Comparaison avec d'autres langages :

"Certains d'entre vous ont de l'expérience en programmation avec d'autres langages comme Java, C++ ou Python. Comment imaginez-vous que C# se compare à ces langages en termes de syntaxe, de fonctionnalités et d'écosystème ?"

Attentes envers Visual Studio 2022 :

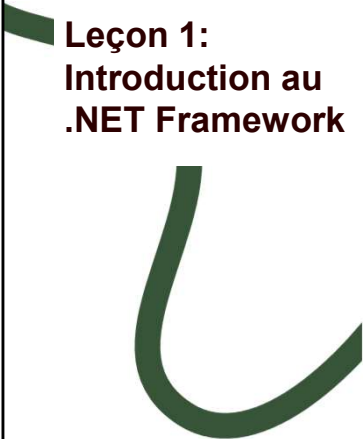
"Quelle est votre expérience avec les environnements de développement intégrés (IDE) en général ? Que recherchez-vous dans un bon IDE et quelles sont vos attentes envers Visual Studio 2022 en particulier ?"

Importance de la documentation :

"Quelle est votre approche personnelle concernant la documentation de votre code ? Pourquoi pensez-vous qu'il est important de bien documenter une application, et quelles sont vos méthodes préférées pour le faire ?"

Expérience avec le débogage :

"Le débogage est une compétence essentielle pour tout développeur. Pouvez-vous partager une situation où vous avez été confronté à un bug difficile à résoudre et comment vous avez finalement réussi à le surmonter ? Quels outils ou techniques avez-vous trouvés les plus utiles dans ce processus ?"



Leçon 1: Introduction au .NET Framework

- Qu'est-ce que .NET?
- Historique
- .NET
- C#
- Notion d'Assembly
- Comment le CLR charge, compile et exécute les assemblies
- Outils .NET

Qu'est-ce que .NET?

- Plateforme de développement gratuite et open-source
- Support d'un grand nombre de types d'application
 - Web, Web API, Cloud, Mobile, Desktop...
- Multi-plateforme
 - Windows, macOS, Linux, Android, iOS...
- Choix du langage de programmation
 - C#, F# ou Visual Basic
- Basé sur un environnement d'exécution « managé » : CLR (Common Language Runtime)

Qu'est-ce que .NET?

.NET est une plateforme de développement logiciel de Microsoft qui se caractérise par sa flexibilité, sa polyvalence et sa capacité à soutenir un large éventail de langages de programmation.

Plateforme de développement gratuite et open-source

.NET est une plateforme open-source, ce qui signifie que son code source est accessible à tous et que chacun peut y contribuer. Le fait qu'elle soit gratuite facilite son adoption et sa démocratisation. Les développeurs peuvent l'utiliser pour créer des applications sans se soucier des coûts de licence.

Support d'un grand nombre de types d'application**•Web, Web API, Cloud, Mobile, Desktop...**

•.NET est extrêmement polyvalent. Que vous souhaitiez développer une application web simple, une API RESTful pour le cloud, une application mobile ou une application de bureau traditionnelle, .NET offre les outils et les frameworks nécessaires pour faciliter ce processus.

Multi-plateforme**•Windows, macOS, Linux, Android, iOS...**

•Avec l'introduction de .NET Core et, plus tard, de .NET 5 et suivants, .NET est devenu une plateforme multi-plateforme. Cela signifie que vous pouvez développer une application sur un système d'exploitation et la déployer sur un autre, par exemple, développer sur macOS et déployer sur Linux.

Choix du langage de programmation

•C#, F# ou Visual Basic

•.NET n'est pas limité à un seul langage de programmation. C# est le langage le plus populaire utilisé avec .NET, mais les développeurs peuvent également choisir F# pour la programmation fonctionnelle ou Visual Basic pour ceux qui préfèrent sa syntaxe et son style.

Basé sur un environnement d'exécution « managé »: CLR (Common Language Runtime)

CLR est le cœur de la plateforme .NET. Il fournit un environnement d'exécution managé où le code est exécuté. CLR gère la mémoire, le garbage collection, la sécurité et fournit de nombreux autres services, ce qui facilite la vie du développeur. L'avantage de cet environnement est qu'il permet d'exécuter du code écrit dans différents langages de programmation sur le même runtime.

Historique

	Windows	Multi-OS
2002	.NET Framework 1.0	
2005	.NET Framework 2.0	
2006	.NET Framework 3.0 (WPF – WCF – WF)	
2007	.NET Framework 3.5 (LINQ)	
2008	.NET Framework 3.5 SP1 (Entity Framework)	
2010	.NET Framework 4.0	
2012	.NET Framework 4.5	
2015	.NET Framework 4.6	
2016		.NET Core 1.0
2017	.NET Framework 4.7	.NET Core 2.0
2019	.NET Framework 4.8	.NET Core 3.0
2020		.NET 5
2021		.NET 6

Historique

Le tableau montre l'évolution de la plateforme .NET au fil des années, en mettant en évidence les versions spécifiques à Windows et celles qui sont multi-OS (systèmes d'exploitation multiples).

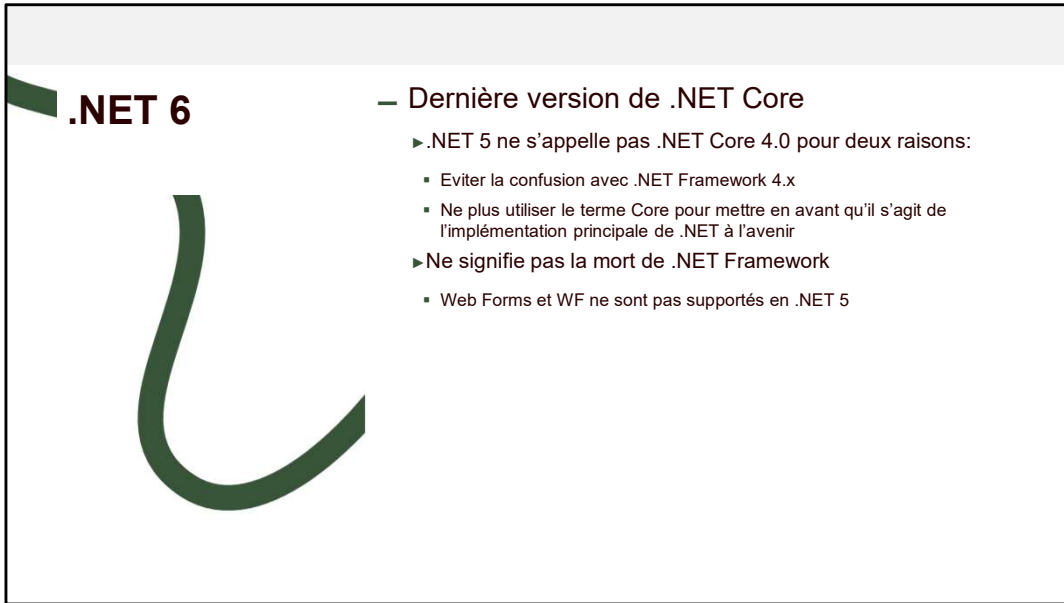
Windows

- 2002 - .NET Framework 1.0:** Il s'agit de la toute première version du .NET Framework, marquant le début d'une nouvelle ère pour le développement Microsoft.
- 2005 - .NET Framework 2.0:** Introduction de nombreuses améliorations et nouvelles fonctionnalités, notamment les génériques.
- 2006 - .NET Framework 3.0:** Intégration de Windows Presentation Foundation (WPF), Windows Communication Foundation (WCF) et Windows Workflow Foundation (WF).
- 2007 - .NET Framework 3.5:** Intégration de LINQ (Language Integrated Query), un ensemble de technologies pour interroger des données de manière native dans C#.
- 2008 - .NET Framework 3.5 SP1:** Inclusion de Entity Framework, un ORM pour travailler avec des bases de données.
- 2010 - .NET Framework 4.0:** Introduit un grand nombre de nouvelles fonctionnalités et améliorations.
- 2012 - .NET Framework 4.5:** Mise à jour avec des améliorations de performance et d'autres optimisations.
- 2015 - .NET Framework 4.6:** D'autres optimisations et nouvelles fonctionnalités ont été ajoutées.

- 2016 - .NET Framework 4.7:** Continuation de la série 4.x avec des mises à jour mineures.
- 2019 - .NET Framework 4.8:** La dernière version majeure du .NET Framework traditionnel.

Multi-OS

- 2016 - .NET Core 1.0:** L'avènement du .NET multiplateforme. .NET Core est une version allégée du .NET Framework, conçue pour être plus modulaire et fonctionner sur plusieurs systèmes d'exploitation.
- 2017 - .NET Core 2.0:** Une mise à jour majeure de .NET Core avec de nombreuses nouvelles fonctionnalités et améliorations.
- 2019 - .NET Core 3.0:** Introduit le support pour le développement d'applications de bureau sur Windows avec WPF et Windows Forms.
- 2020 - .NET 5:** Fusion de .NET Core et .NET Framework pour créer une seule plateforme .NET unifiée. Le "Core" a été omis du nom pour refléter cette unification.
- 2021 - .NET 6:** Dernière version à la date de la diapositive, continuant d'améliorer et d'étendre les capacités de .NET.



.NET 6

— Dernière version de .NET Core

- .NET 5 ne s'appelle pas .NET Core 4.0 pour deux raisons:
 - Éviter la confusion avec .NET Framework 4.x
 - Ne plus utiliser le terme Core pour mettre en avant qu'il s'agit de l'implémentation principale de .NET à l'avenir
- Ne signifie pas la mort de .NET Framework
 - Web Forms et WF ne sont pas supportés en .NET 5

.NET 6

.NET 6 est présenté comme la dernière évolution de la série .NET Core. Cette diapositive met en lumière certaines clarifications importantes concernant le positionnement de .NET 5 et de .NET 6 par rapport aux versions précédentes.

Dernière version de .NET Core

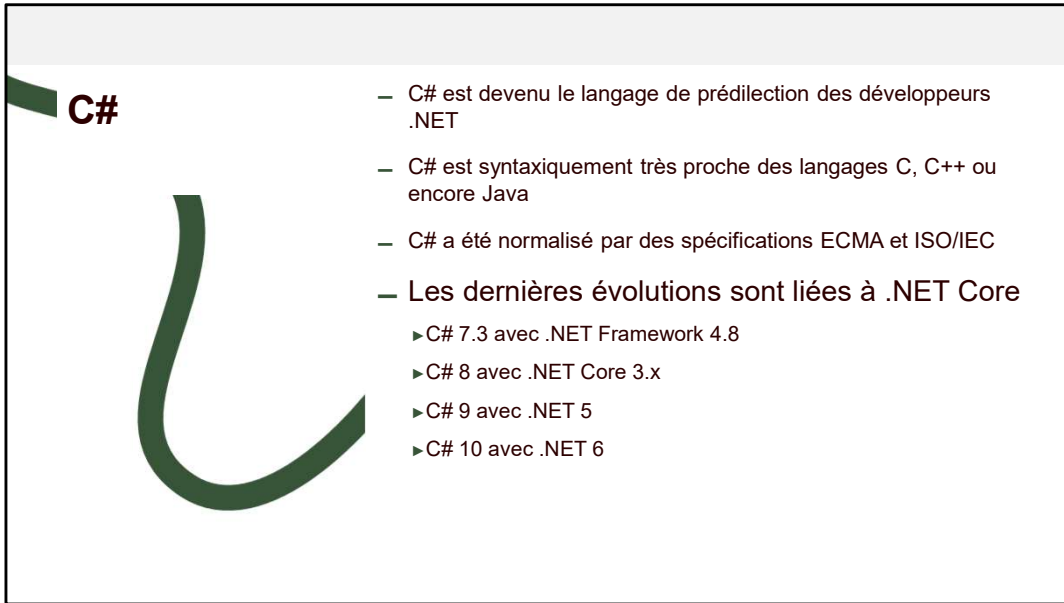
• .NET 5 ne s'appelle pas .NET Core 4.0 pour deux raisons:

- **Éviter la confusion avec .NET Framework 4.x:** Étant donné que les numéros de version 4.x sont fortement associés au .NET Framework traditionnel, appeler la nouvelle version ".NET Core 4.0" aurait pu prêter à confusion pour de nombreux développeurs.
- **Ne plus utiliser le terme Core pour mettre en avant qu'il s'agit de l'implémentation principale de .NET à l'avenir:** Le passage de ".NET Core" à ".NET" signale l'intention de Microsoft de consolider et d'unifier la plateforme. Le terme "Core" a été omis pour indiquer que cette version représente l'avenir de .NET, et non une variante parallèle.

Ne signifie pas la mort de .NET Framework:

• **Web Forms et WF ne sont pas supportés en .NET 5:** Bien que .NET 5 et .NET 6 introduisent de nombreuses nouvelles fonctionnalités et améliorations, certains éléments, tels que Web Forms et Windows Workflow Foundation (WF), ne sont pas pris en charge. Il est important de noter que le non-support de ces technologies dans .NET 5 ne signifie pas la fin du .NET Framework traditionnel. Les développeurs peuvent toujours

utiliser .NET Framework pour les applications qui dépendent de ces technologies.
En résumé, cette diapositive donne un aperçu de la raison pour laquelle .NET 5 n'a pas été appelé ".NET Core 4.0" et souligne le fait que, bien que .NET 5 et .NET 6 représentent l'avenir de la plateforme, le .NET Framework traditionnel continue d'exister et de supporter certaines technologies qui ne sont pas incluses dans les versions plus récentes.



C#

- C# est devenu le langage de prédilection des développeurs .NET
- C# est syntaxiquement très proche des langages C, C++ ou encore Java
- C# a été normalisé par des spécifications ECMA et ISO/IEC
- Les dernières évolutions sont liées à .NET Core
 - ▶ C# 7.3 avec .NET Framework 4.8
 - ▶ C# 8 avec .NET Core 3.x
 - ▶ C# 9 avec .NET 5
 - ▶ C# 10 avec .NET 6

C#

Le langage C# (prononcé "C sharp") est un pilier central de la plateforme .NET. Depuis sa création, il a continué d'évoluer, s'adaptant aux besoins changeants de l'industrie du développement et intégrant de nouvelles fonctionnalités avec chaque version.

Langage de prédilection pour .NET:

• **C# est devenu le langage de prédilection des développeurs .NET:** C# a gagné une immense popularité parmi les développeurs depuis son introduction. Sa simplicité, sa puissance et son intégration étroite avec .NET en font le choix de prédilection pour de nombreux professionnels.

Proximité syntaxique avec d'autres langages:

• **C# est syntaxiquement très proche des langages C, C++ ou encore Java:** Pour ceux qui ont une expérience préalable avec ces langages, se mettre au C# est souvent plus simple. La familiarité de la syntaxe facilite la transition et permet une montée en compétence rapide.

Normalisation:

• **C# a été normalisé par des spécifications ECMA et ISO/IEC:** Cette normalisation garantit que le langage suit des standards internationaux, ce qui renforce sa crédibilité et sa portabilité à travers différents environnements et plateformes.

Évolutions récentes liées à .NET Core:

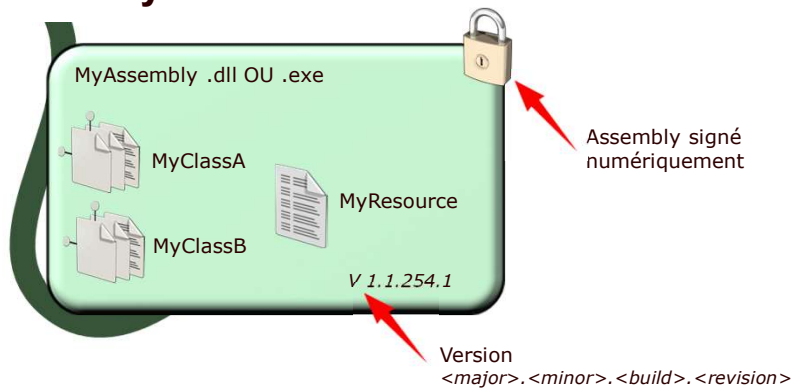
- Les versions récentes de C# ont été étroitement liées à l'évolution de .NET. Par exemple:
 - **C# 7.3 a été associé à .NET Framework 4.8.**

- **C# 8** est venu avec **.NET Core 3.x**, introduisant une série de nouvelles fonctionnalités et améliorations.
- **C# 9** a été lancé avec **.NET 5**, continuant sur la lancée de son prédécesseur.
- **C# 10** est synchronisé avec la sortie de **.NET 6**, montrant une fois de plus l'engagement de Microsoft à faire évoluer le langage et la plateforme conjointement.

En conclusion, C# a évolué et s'est adapté au fil des ans, reflétant les tendances et les besoins changeants du développement logiciel. La proximité syntaxique avec d'autres langages populaires, sa normalisation et ses évolutions conjointes avec .NET montrent son importance et sa pertinence dans l'écosystème de développement moderne.

Notion d'Assembly

- Blocs élémentaires des applications .NET
- Collection de types et de ressources



Comment le CLR charge, compile et exécute les Assemblies

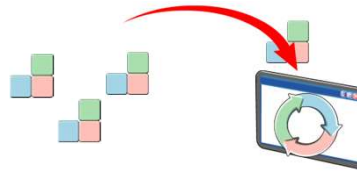
Les Assemblies contiennent du code MSIL non exécutable

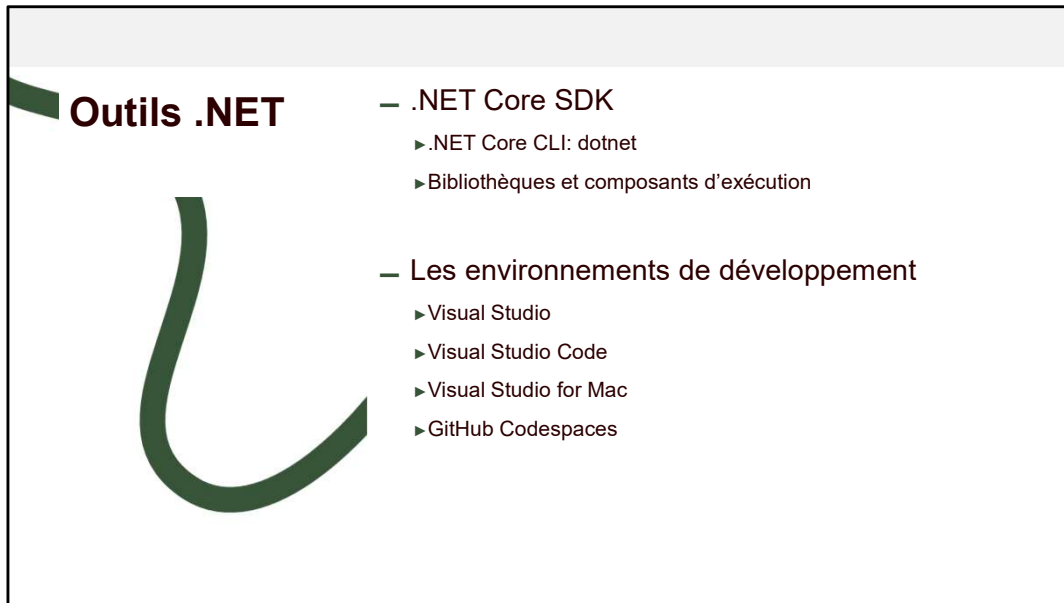
Le CLR charge le code MSIL et le compile en code natif en fonction de la machine physique

1 Charge les Assemblies que l'application référence

2 Vérifie et compile le code MSIL en langage machine

3 Exécute le code





Outils .NET

.NET offre une suite d'outils et de plateformes pour faciliter le développement d'applications. Ces outils comprennent des kits de développement logiciel (SDK), des interfaces en ligne de commande (CLI) et des environnements de développement intégrés (IDE) qui répondent aux besoins des développeurs.

.NET Core SDK:

•**.NET Core CLI: dotnet:** Il s'agit de l'interface en ligne de commande pour .NET Core, qui permet aux développeurs de créer, exécuter, publier et gérer leurs applications .NET Core. Avec la commande dotnet, les développeurs peuvent initialiser de nouveaux projets, ajouter des dépendances, compiler le code et bien plus encore.

•**Bibliothèques et composants d'exécution:** Le SDK .NET Core est livré avec des bibliothèques standard et des composants d'exécution nécessaires pour créer et exécuter des applications .NET Core. Ces bibliothèques couvrent une vaste gamme de fonctionnalités, de la gestion des chaînes de caractères à l'accès aux bases de données.

Environnements de développement:

•**Visual Studio:** C'est l'IDE phare de Microsoft pour le développement .NET. Il offre une gamme complète d'outils pour le développement, le débogage et le déploiement d'applications. Avec une interface utilisateur riche, il est idéal pour les grands projets et les équipes de développement.

•**Visual Studio Code:** VS Code est un éditeur de code source léger et puissant avec prise en charge de .NET Core. Il est open-source et disponible sur plusieurs plateformes,

notamment Windows, macOS et Linux. Sa légèreté et sa modularité, grâce à une vaste gamme d'extensions, en font un choix populaire parmi les développeurs.

- Visual Studio for Mac:** Conçu spécifiquement pour macOS, cet IDE apporte l'expérience de développement .NET aux utilisateurs de Mac. Il est idéal pour le développement d'applications mobiles, Web et cloud sur .NET Core et Xamarin.

- GitHub Codespaces:** Il s'agit d'un environnement de développement entièrement configuré, hébergé dans le cloud, qui permet aux développeurs de commencer à coder immédiatement sans aucune configuration locale. Intégré à GitHub, Codespaces offre une expérience fluide pour le développement collaboratif et la révision de code.

En résumé, la plateforme .NET offre une variété d'outils et d'environnements pour répondre aux besoins des développeurs, qu'ils travaillent localement sur leur machine, dans le cloud ou collaborativement avec des équipes distribuées.

A large, dark green bracket-like shape that starts as a horizontal line on the left, curves downwards and then upwards to the right, framing the text.

Leçon 2: Création de projets avec Visual Studio 2022

- Visual Studio Code
- Visual Studio
- Gestion de projet
- Modèles de projet
- Démonstration :Création d'une application .NET,
compilation et exécution



Visual Studio Code

Visual Studio Code, souvent abrégé en VS Code, est un éditeur de code source développé par Microsoft. Il a rapidement gagné en popularité parmi les développeurs en raison de sa flexibilité, de sa légèreté et de ses puissantes fonctionnalités.

• Éditeur de code source:

- **Gratuit:** L'un des grands avantages de VS Code est qu'il est complètement gratuit. Cela le rend accessible à un large éventail de développeurs, des hobbyistes aux professionnels.
- **Léger:** Contrairement à d'autres environnements de développement intégrés (IDE) plus lourds, VS Code est conçu pour être rapide et réactif. Il démarre rapidement et consomme moins de ressources système.
- **Puissant:** Malgré sa légèreté, VS Code offre un ensemble robuste de fonctionnalités, y compris un débogueur intégré, une intégration Git, une complétion de code intelligente et bien plus encore.
- **Multi-plateforme:** Il est disponible pour Windows, macOS et Linux. Cela signifie que les développeurs peuvent utiliser le même éditeur sur plusieurs systèmes d'exploitation, offrant une expérience cohérente.

• Support de la plupart des langages (natif ou via extensions):

- **Natif:** De base, sans aucune extension supplémentaire, VS Code offre une prise en charge pour des langages tels que:
 - JavaScript: Langage de script largement utilisé pour le

développement Web.

- TypeScript: Surcouche typée de JavaScript offrant des fonctionnalités supplémentaires.
- Node.js: Environnement d'exécution pour exécuter le code JavaScript côté serveur.
- **Par extensions:** L'une des plus grandes forces de VS Code est son système d'extensions. Les développeurs peuvent ajouter des extensions pour augmenter les capacités de l'éditeur. Grâce à ce système:
 - C++: Langage de programmation de bas niveau largement utilisé.
 - C#: Langage de programmation orienté objet développé par Microsoft.
 - Java: Langage de programmation orienté objet utilisé pour le développement d'applications mobiles, Web et d'entreprise.
 - Python: Langage de script polyvalent populaire pour le développement Web, la science des données et plus encore.
 - PHP: Langage de script côté serveur pour le développement Web.

En conclusion, Visual Studio Code est un éditeur de code polyvalent et extensible qui a été adopté par un grand nombre de développeurs à travers le monde. Sa capacité à s'adapter à divers besoins de développement, grâce à son riche écosystème d'extensions, en fait un outil essentiel pour de nombreux professionnels de la programmation.



Visual Studio

Visual Studio est un environnement de développement intégré (IDE) produit par Microsoft. Il offre aux développeurs un ensemble d'outils pour faciliter la création, le débogage et le déploiement d'applications sur différentes plateformes.

•Environnement de développement intégré:

- **Plusieurs éditions:**

- **Community:** C'est la version gratuite de Visual Studio destinée aux étudiants, aux développeurs individuels et aux petits projets. Elle offre un ensemble complet d'outils pour le développement.
- **Professional:** Cette version est destinée aux développeurs professionnels et offre des fonctionnalités supplémentaires par rapport à l'édition Community, notamment en matière d'analyse de code et de collaboration en équipe.
- **Enterprise:** La version Enterprise est la plus complète et est destinée aux grandes entreprises. Elle comprend des outils avancés pour la gestion de projets à grande échelle, des tests, et bien d'autres fonctionnalités spécialisées.

•Support d'un grand nombre de fonctionnalités:

- **Rapid Application Development (RAD):** Visual Studio prend en charge le RAD, qui est une méthode de développement logiciel qui permet une création rapide d'applications.

- **Gestion de données locales ou distantes:** Les développeurs peuvent facilement intégrer et gérer des bases de données, qu'elles soient hébergées localement ou à distance.
- **Débogage et Gestion d'erreur:** L'IDE offre un puissant débogueur qui aide à identifier et à résoudre les erreurs dans le code. Les développeurs peuvent définir des points d'arrêt, inspecter les variables et suivre l'exécution du code étape par étape.
- **Aide et documentation:** Visual Studio fournit une documentation exhaustive qui guide les développeurs à travers différentes tâches et problèmes courants.
- **Extensions par NuGet:** NuGet est un gestionnaire de paquets pour .NET. Il permet aux développeurs d'ajouter facilement des bibliothèques et des outils tiers à leurs projets. Cela enrichit l'écosystème de Visual Studio et rend le développement plus fluide et efficace.

En conclusion, Visual Studio est un IDE robuste et polyvalent qui répond aux besoins de nombreux développeurs, qu'ils soient novices ou experts. Sa capacité à s'adapter et à offrir une large gamme d'outils en fait un choix de prédilection pour de nombreux professionnels dans le domaine du développement logiciel.



Gestion de projet

- Visual Studio gère toute l'arborescence physique d'un projet
 - Fichier Solution *.sln, Fichier Projet *.csproj
- Visual Studio Code nécessite plus de travail:
 - Création du dossier de projet
 - Création du projet via `dotnet new template`
 - `dotnet new --list`
 - Ouverture dans Visual Studio Code via `code .`
 - Workspace: ensemble de projets/dossiers associés dans VS Code, référencé par un fichier `.code-workspace` (json)

Gestion de projet

La gestion de projet est un élément essentiel pour tout développeur, car elle garantit une organisation et une structuration optimales des ressources, du code et des dépendances. Visual Studio et Visual Studio Code, bien qu'étant des produits de Microsoft, abordent la gestion de projet de manière légèrement différente.

•Visual Studio:

- **Gère toute l'arborescence physique d'un projet:** Cela signifie que lors de la création d'un nouveau projet dans Visual Studio, l'IDE crée automatiquement tous les dossiers et fichiers nécessaires pour vous.
- ***Fichier Solution (.sln)**:** Il s'agit du fichier principal qui contient des informations sur l'ensemble de la solution, qui peut englober plusieurs projets.
- ***Fichier Projet (.csproj)**:** Ce fichier contient des détails spécifiques sur un projet particulier, y compris les dépendances, les configurations et d'autres paramètres.

•Visual Studio Code:

- **Nécessite plus de travail :** Contrairement à Visual Studio, VS Code est un éditeur de code source plutôt qu'un IDE complet. Il offre une grande flexibilité, mais nécessite souvent une configuration supplémentaire pour certains types de projets.
- **Création du dossier de projet:** Avant de commencer, les développeurs

doivent créer un dossier pour leur projet.

- **Création du projet via dotnet new template:** Utilisez la CLI .NET Core pour initialiser un nouveau projet. Par exemple, `dotnet new --list` montre tous les templates disponibles, et vous pouvez choisir celui qui convient à vos besoins.

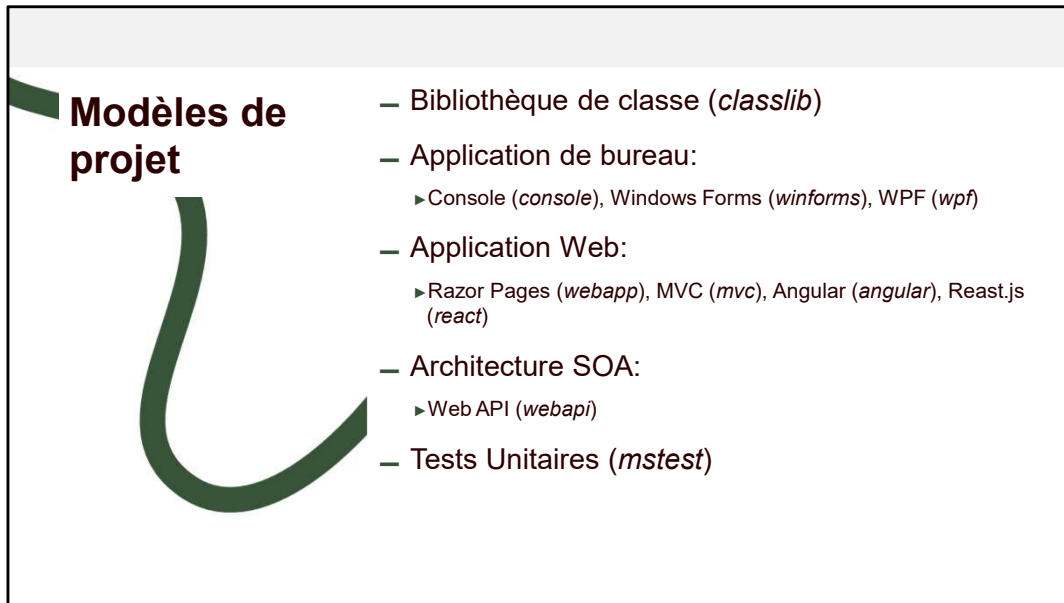
- ****Ouverture dans Visual Studio Code via**

code .:** Après avoir configuré le projet via la CLI .NET Core, vous pouvez facilement l'ouvrir dans VS Code en naviguant vers le dossier du projet dans le terminal et en tapant `code .` pour lancer l'éditeur.

• **Workspace:** Dans Visual Studio Code, un "workspace" est un concept qui permet de regrouper plusieurs projets ou dossiers. Un fichier `.code-workspace` (au format JSON) est utilisé pour stocker des informations sur ce regroupement. Cela est particulièrement utile si vous travaillez sur plusieurs projets interdépendants ou si vous souhaitez avoir des paramètres spécifiques pour différents dossiers au sein d'un même projet.

Conclusion:

Tandis que Visual Studio offre une expérience plus intégrée et automatisée pour la gestion de projets, Visual Studio Code offre une plus grande flexibilité, en permettant aux développeurs de configurer leurs projets selon leurs besoins spécifiques. Le choix entre les deux dépendra largement des préférences individuelles, des exigences du projet, et de la familiarité du développeur avec les outils en question.



Modèles de projet

Lors de la création d'un nouveau projet à l'aide de Visual Studio ou de la CLI .NET Core, vous pouvez choisir parmi différents modèles de projet qui définissent la structure et les paramètres de base de votre projet. Ces modèles sont conçus pour vous aider à démarrer rapidement, en fournissant le code de démarrage et les configurations nécessaires pour différents types d'applications.

• **Bibliothèque de classe (*classlib*)**: Une bibliothèque de classe est un type de projet qui permet de créer des bibliothèques réutilisables de classes et de méthodes. Ces bibliothèques peuvent ensuite être référencées et utilisées dans d'autres projets.

• **Application de bureau:**

- **Console (*console*)**: Crée une application console simple qui s'exécute dans un terminal ou une fenêtre de commande.
- **Windows Forms (*winforms*)**: Permet de créer des applications GUI pour Windows en utilisant le framework Windows Forms.
- **WPF (*wpf*)**: Ou Windows Presentation Foundation, est un framework pour créer des applications desktop riches pour Windows avec des graphismes avancés et des capacités d'interface utilisateur.

• **Application Web:**

- **Razor Pages (*webapp*)**: Un modèle plus récent pour créer des applications web avec ASP.NET Core. C'est une manière simplifiée de créer des pages web avec un backend C#.

- **MVC (mvc):** Model-View-Controller est un motif de conception couramment utilisé pour créer des applications web. ASP.NET Core MVC permet une séparation claire entre la logique, la présentation et les données.
- **Angular (angular):** Un framework front-end populaire pour créer des applications web modernes et réactives.
- **React.js (react):** Une bibliothèque JavaScript pour créer des interfaces utilisateur.

• **Architecture SOA:**

- **Web API (webapi):** ASP.NET Core Web API permet de créer des services web RESTful. Ces services peuvent être consommés par différents clients comme des applications web, mobiles ou desktop.

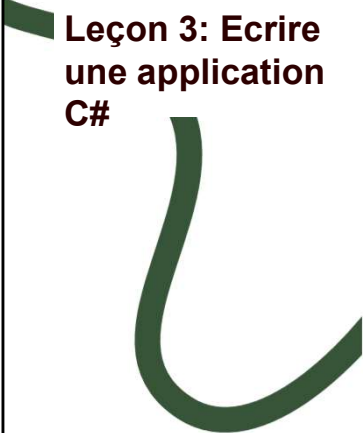
• **Tests Unitaires (mstest):** MSTest est le framework de test intégré à Visual Studio pour écrire des tests unitaires. Il permet de vérifier que des portions individuelles de code (comme des méthodes ou des classes) fonctionnent comme prévu.

Conclusion:

Le choix du modèle de projet dépend de vos besoins spécifiques et du type d'application que vous souhaitez développer. Chaque modèle offre une base solide pour commencer, avec des configurations et des outils adaptés à chaque cas d'utilisation. Il est toujours recommandé de bien comprendre les spécificités et les avantages de chaque modèle avant de prendre une décision.

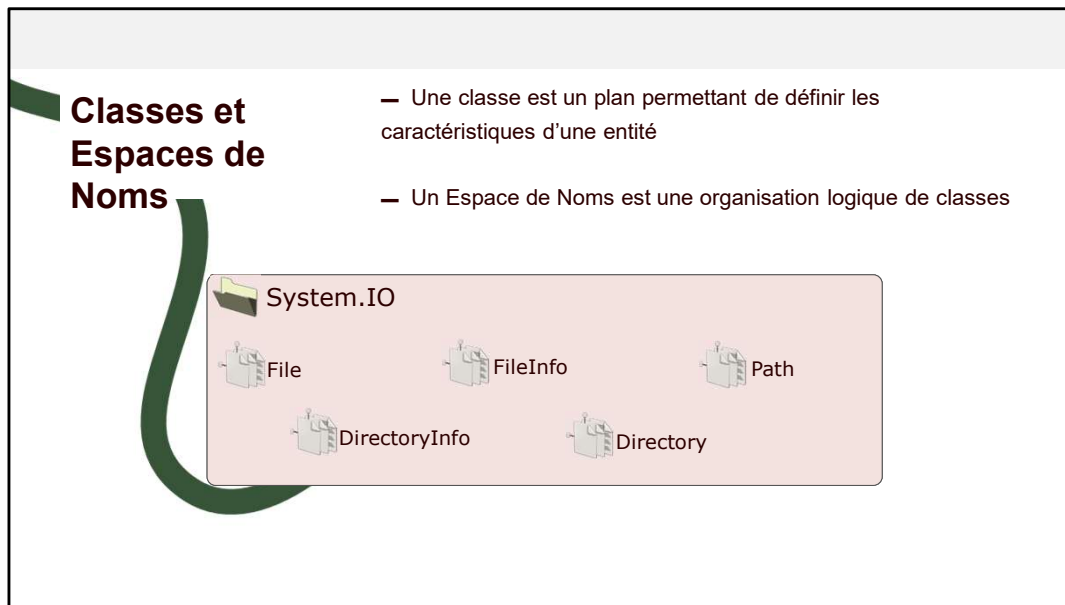
Démonstration— Création d'une application .NET, compilation et
exécution

- mkdir TestApp
 - cd TestApp
- dotnet new console
 - code .
- Options d'enregistrement automatique !!
- Ajouter les éléments nécessaires pour exécuter/déboguer, et montrer l'exécution via le menu Run
- Montrer ensuite le terminal, modifier le code, compiler et exécuter via dotnet build/dotnet run
- Modifier le code pour ne conserver que le using et le Console.WriteLine, dotnet run et TADAAAA!!!!

A large, thick green bracket-like graphic on the left side of the slide, spanning the height of the content area.

Leçon 3: Ecrire une application C#

- Classes et Espaces de Noms
- Structure d'une application Console
- Entrées/Sorties dans une application Console
- Bonnes pratiques de commentaires



Classes et Espaces de Noms

Dans la programmation orientée objet, telle qu'elle est utilisée dans des langages comme C#, les classes et les espaces de noms sont des concepts fondamentaux.

•Classe:

•Une classe est décrite comme un plan ou un modèle à partir duquel des objets sont créés. Elle permet de définir les caractéristiques (souvent appelées attributs ou champs) et les comportements (souvent appelés méthodes) d'une entité. En d'autres termes, une classe est une spécification pour un type d'objet particulier, décrivant ce que l'objet peut faire (ses méthodes) et quelles données il contient (ses champs).

•Espace de Noms:

•Les espaces de noms sont utilisés pour organiser de grandes quantités de code de manière logique. Pensez-y comme à des conteneurs pour les classes. Un espace de noms permet d'éviter les conflits de nommage en regroupant des classes similaires ou liées sous un même nom.

Exemple : System.IO

L'espace de noms System.IO dans .NET est dédié à la manipulation d'entrées et de sorties, notamment les fichiers et les répertoires. Quelques classes de cet espace de noms sont :

•**File** : Fournit des méthodes statiques pour la création, la copie, la suppression, le déplacement et l'ouverture de fichiers.

•**FileInfo** : Offre des méthodes et des propriétés similaires à la classe File, mais est

instanciée autour d'un fichier particulier, permettant ainsi des opérations multiples sur ce fichier sans réouvrir le fichier pour chaque opération.

- Directory** : Propose des méthodes statiques pour créer, déplacer et parcourir les répertoires.

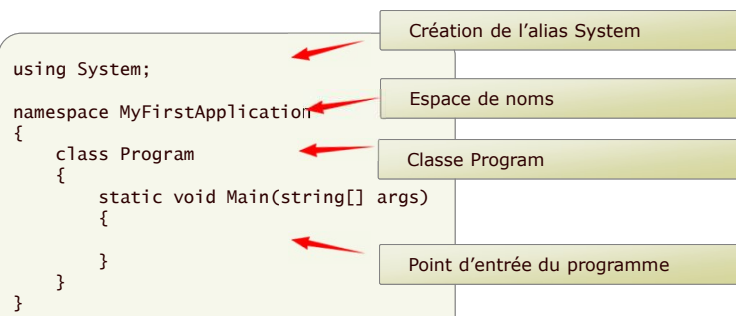
- DirectoryInfo** : Similaire à FileInfo, mais pour les répertoires. Il fournit des méthodes et des propriétés pour créer, déplacer et parcourir les sous-répertoires.

- Path** : Contient des méthodes pour traiter des chaînes de caractères représentant des chemins de fichiers et de répertoires.

Conclusion:

La compréhension des classes et des espaces de noms est essentielle pour la programmation en C# et dans d'autres langages orientés objet. Ces concepts permettent d'organiser et de structurer le code, facilitant ainsi son développement, sa maintenance et sa compréhension.

Structure d'une application Console



Entrées/Sorties dans une application Console

System.Console

Clear()

Read()

ReadKey()

ReadLine()

Write()

WriteLine()

```
using System;
...
Console.WriteLine("Hello there!");
```

C:/

Bonnes pratiques de commentaires

- Faire précéder les déclarations par un commentaire
- Dans une méthode, ajouter des commentaires intermédiaires pour aérer le code ou documenter les structures de décision
- Respecter les conventions de nommage:
 - **PascalCasing**: Espaces de noms, Classes, Méthodes
 - **camelCasing**: variables, paramètres
- Utiliser un préfixe de type pour les variables et paramètres

```
/* Ceci est un bloc  
de commentaires */  
string strMessage = "Hello there!"; // Commentaire de ligne
```

Bonnes pratiques de commentaires

1. Commenter les déclarations:

Il est recommandé de faire précéder toute déclaration, que ce soit une variable, une classe ou une méthode, par un commentaire explicatif. Cela facilite la compréhension du code pour vous-même et pour les autres développeurs qui pourraient travailler sur votre code à l'avenir.

2. Commentaires intermédiaires dans les méthodes:

À l'intérieur d'une méthode, il peut être bénéfique d'ajouter des commentaires intermédiaires pour "aérer" le code, surtout s'il est complexe ou long. De plus, documenter les structures de décision (comme les if, switch, etc.) peut clarifier la logique derrière ces décisions.

3. Conventions de nommage:

• **PascalCasing**: Il est courant d'utiliser PascalCasing pour les noms des espaces de noms, des classes et des méthodes. C'est-à-dire que chaque mot commence par une majuscule. Exemple: MyClass, CalculateTotal.

• **camelCasing**: Pour les variables et les paramètres, il est recommandé d'utiliser camelCasing, où le premier mot commence par une minuscule et les mots suivants par une majuscule. Exemple: myVariable, totalAmount.

4. Utilisation de préfixes pour les variables:

Il était autrefois courant d'utiliser un préfixe pour indiquer le type d'une variable (par exemple, str pour les chaînes). Cependant, cette pratique est moins courante dans les

langages modernes, en particulier avec des outils de développement qui fournissent des indices de type. Néanmoins, certains développeurs et équipes préfèrent toujours l'utiliser pour des raisons de clarté.

Exemple de commentaires:

```
/* Ceci est un bloc de commentaires */ string strMessage = "Hello there!"; // Ceci est un  
commentaire de ligne
```

Conclusion:

Les commentaires sont essentiels pour maintenir un code propre et compréhensible. Ils servent de documentation intégrée et permettent aux développeurs, y compris à vous dans le futur, de comprendre rapidement l'intention et la logique du code. Adopter de bonnes pratiques dès le début du développement peut économiser beaucoup de temps et d'efforts à long terme.

Leçon 4:**Documenter une
Application**

- Commentaires XML
- Génération de la documentation



Commentaires XML

```
/// <summary> The Hello class prints a greeting on the screen  
/// </summary>  
public class Hello  
{  
    /// <summary> We use console-based I/O. For more information  
    /// about  
    /// WriteLine, see <seealso cref="System.Console.WriteLine"/>  
    /// </summary>  
    public static void Main( )  
    {  
        Console.WriteLine("Hello World");  
    }  
}
```

Ces commentaires peuvent être générés en XML à la compilation pour être ensuite dissociés du code.

L'extension C# de VS Code ajoute les commentaires si l'option Format on Type est activée (File/Preferences/Settings/Text Editor/Format on Type)



Génération de la documentation

- Sandcastle Help File Builder
 - solution graphique basée sur l'outil Sandcastle initialement fourni et développé par Microsoft
- DocFX
 - solution mise en avant par Microsoft depuis 2017, basée sur un outil ligne de commande (proche de javadoc)
- Différents produits tiers existent également
 - GhostDoc, Atomineer Pro Documentation, Doxygen, Wyam...

entation" :

Génération de la documentation

1. Sandcastle Help File Builder:

Il s'agit d'une solution graphique pour générer des fichiers d'aide. Basée sur l'outil Sandcastle fourni initialement par Microsoft, elle permet aux développeurs de créer une documentation complète à partir de leurs projets .NET. Le résultat est souvent un ensemble de fichiers HTML, mais d'autres formats peuvent également être générés.

2. DocFX:

DocFX est mis en avant par Microsoft depuis 2017. Contrairement à Sandcastle qui est principalement graphique, DocFX est basé sur une interface en ligne de commande, ce qui le rapproche d'outils comme Javadoc utilisé pour Java. Il prend en charge la génération de documentation à partir de commentaires de code mais permet également d'intégrer des articles, des blogs et des tutoriels.

3. Produits tiers:

Il existe de nombreux outils tiers pour la génération de documentation. Certains des plus populaires comprennent :

- **GhostDoc**: Un outil qui génère automatiquement des commentaires à partir de votre code, ce qui peut être un gain de temps considérable. Il est particulièrement apprécié pour sa capacité à générer des commentaires basés sur le nom des méthodes, des variables, etc.

- **Atomineer Pro Documentation**: Une autre solution pour générer et gérer la

documentation dans divers langages de programmation.

- Doxygen**: Bien que plus associé à des langages comme C++, il est également utilisé pour d'autres langages. Il est particulièrement puissant pour les projets de grande envergure.

- Wyam**: Un générateur de sites statiques et de documentation écrit en .NET. Il est extensible et permet de créer tout type de contenu, pas seulement de la documentation.

Conclusion:

La documentation est cruciale pour la maintenance et l'évolutivité d'un projet. Elle permet aux nouveaux membres de l'équipe de comprendre rapidement le projet et facilite également la détection d'erreurs ou d'incohérences dans le code. L'utilisation d'outils de génération de documentation garantit une documentation cohérente, à jour et de qualité.

**Leçon 6:
Exécuter et
débuguer des
applications
avec Visual
Studio 2022**

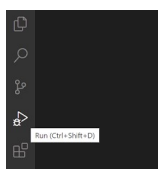
- Débogage
- Points d'arrêt
- Pas à pas
- Panneaux de débogage





Débogage

- Le débogage consiste habituellement à corriger les erreurs de logique du code
- Composante importante de tout éditeur de code
- Visual Studio Code propose une vue d'exécution:
 - ▶ Gestion de points d'arrêt
 - ▶ Gestion du pas à pas
 - ▶ Visualisation et modification des variables pendant le débogage
 - ▶ Pile des appels
- Visual Studio offre une gestion complète du débogage



Débogage

Le débogage est une étape cruciale du cycle de développement logiciel. Il permet aux développeurs de détecter, de localiser et de corriger les erreurs, ou "bugs", dans le code source.

1. Qu'est-ce que le débogage?

Le débogage consiste à trouver et à résoudre les défauts ou anomalies présents dans le code d'un programme qui empêchent son bon fonctionnement. Cela peut aller de simples erreurs typographiques à des problèmes de logique plus complexes.

2. Importance du débogage dans les éditeurs de code:

La capacité à déboguer efficacement est essentielle pour tout développeur. La plupart des environnements de développement intégrés (EDI) modernes, tels que Visual Studio Code ou Visual Studio, fournissent des outils avancés pour faciliter cette tâche.

3. Visual Studio Code et le débogage:

•**Gestion des points d'arrêt:** Les points d'arrêt permettent aux développeurs de "stopper" l'exécution du code à un endroit précis, afin d'examiner le flux d'exécution et les valeurs des variables.

•**Gestion du pas à pas:** Cette fonction permet d'exécuter le code ligne par ligne, donnant aux développeurs une visibilité totale sur le flux d'exécution.

•**Visualisation et modification des variables:** Pendant la session de débogage, les développeurs peuvent consulter et même modifier les valeurs des variables, ce qui est essentiel pour comprendre et résoudre les problèmes.

- Pile des appels:** La pile des appels montre la séquence de méthodes et de fonctions qui ont été appelées jusqu'au point actuel de l'exécution.

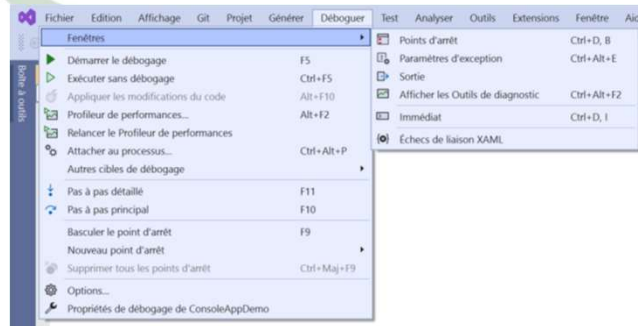
4. Visual Studio et le débogage:

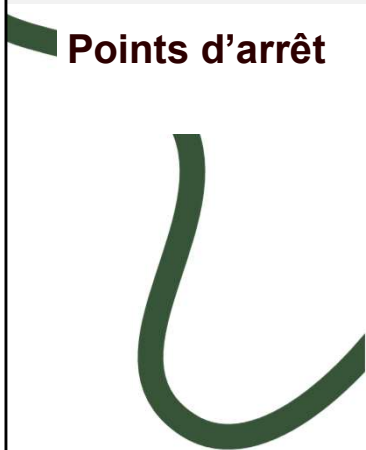
Visual Studio, étant une solution plus complète et plus robuste que Visual Studio Code, offre également une suite complète d'outils de débogage, y compris, mais sans s'y limiter, la gestion des exceptions, l'analyse de la mémoire, et la visualisation en temps réel des performances.

Conclusion:

Le débogage est une compétence essentielle pour tout développeur. Disposer d'outils de débogage efficaces et intégrés directement dans l'EDI facilite grandement le processus de développement, permettant aux développeurs de créer des applications plus stables et performantes.

Débogage et Visual Studio





Points d'arrêt

- Mécanisme de mise en pause de l'exécution
 - ▶ Sur une ligne précise
 - ▶ Sur une ligne précise, en fonction d'une condition
 - ▶ Lors de l'appel d'une méthode (l'interruption se fait à la première ligne de cette méthode)
 - ▶ Sur une ligne précise, sur une colonne précise (utile pour du code minifié)
- Désactivation/Réactivation
 - ▶ Conserver les points d'arrêts sans qu'ils soient actifs
- Persistance après un redémarrage de Visual Studio/Visual Studio Code

Points d'arrêt

Les points d'arrêt sont des outils essentiels pour les développeurs, car ils permettent de contrôler l'exécution d'un programme, en le mettant en pause à des endroits spécifiques. Ceci est particulièrement utile pour identifier et résoudre des problèmes dans le code.

1. Mécanisme de mise en pause de l'exécution:

- **Sur une ligne précise:** C'est le type de point d'arrêt le plus courant. Lorsque le programme atteint cette ligne, il s'arrête, permettant au développeur d'examiner l'état du système à ce moment précis.
- **En fonction d'une condition:** Parfois, un développeur souhaite arrêter l'exécution uniquement lorsque certaines conditions sont remplies, comme une variable atteignant une valeur spécifique.
- **Lors de l'appel d'une méthode:** Cela est utile pour déboguer des méthodes spécifiques, en mettant le programme en pause dès que la méthode est appelée.
- **Sur une colonne précise:** Cette fonction est essentielle lorsque vous travaillez avec du code minifié, où plusieurs instructions peuvent être sur une seule ligne.

2. Désactivation/Réactivation:

Il n'est pas toujours nécessaire de supprimer un point d'arrêt si vous ne souhaitez pas l'utiliser immédiatement. Au lieu de cela, vous pouvez le désactiver, ce qui le garde en place mais permet au code de s'exécuter normalement. Plus tard, vous pouvez le réactiver selon vos besoins.

3. Persistance après un redémarrage:

Un autre avantage des points d'arrêt dans des environnements comme Visual Studio et Visual Studio Code est leur persistance. Si vous fermez et rouvrez votre projet, ou même redémarrez l'EDI, vos points d'arrêt seront toujours là, à moins que vous ne les supprimiez explicitement. Cela permet une continuité dans le processus de débogage.

Conclusion:

Les points d'arrêt sont des outils indispensables pour les développeurs. Ils offrent un contrôle précis sur l'exécution d'un programme, ce qui est crucial pour identifier et résoudre les problèmes. Les environnements de développement modernes, tels que Visual Studio et Visual Studio Code, fournissent des fonctionnalités avancées de points d'arrêt qui rendent le processus de débogage plus fluide et efficace.



Pas à Pas

- Permet d'avancer dans l'exécution ligne par ligne durant une session de débogage
- 3 modes:
 - ▶ Pas à Pas Principal (*Step Over* – F10)
 - Avance ligne par ligne
 - Ne rentre pas dans les méthodes appelées
 - ▶ Pas à Pas Détaillé (*Step Into* – F11)
 - Avance ligne par ligne
 - Parcours tout le code
 - ▶ Pas à Pas Sortant (*Step Out* – Shift+F11)
 - Finis l'exécution de la méthode en cours
 - Interrompt de nouveau l'exécution à la ligne d'appel de la méthode

Pas à Pas (Step-by-Step Debugging)

Le débogage "Pas à Pas" est une technique qui permet aux développeurs d'exécuter leur code ligne par ligne, ce qui facilite la localisation et la correction des erreurs. Dans des outils tels que Visual Studio, cette fonctionnalité est généralement accessible via des raccourcis clavier, offrant aux développeurs une granularité fine pour le débogage.

1. Pas à Pas Principal (Step Over – F10):

•**Description:** Ce mode permet de passer à la ligne suivante du code en cours d'exécution. Si la ligne contient un appel de méthode, la méthode est exécutée dans son intégralité, et l'exécution reprend à la ligne suivante de l'appelant, sans entrer dans le corps de la méthode appelée.

•**Utilisation:** C'est utile quand on veut suivre le flux principal du code sans se plonger dans les détails des méthodes appelées.

2. Pas à Pas Détaillé (Step Into – F11):

•**Description:** Contrairement au "Step Over", ce mode exécute le code ligne par ligne et entre dans les méthodes pour exécuter également leur contenu ligne par ligne. C'est le niveau de débogage le plus détaillé.

•**Utilisation:** Il est préférable d'utiliser cette option lorsque vous soupçonnez qu'une erreur se trouve à l'intérieur d'une méthode spécifique et que vous souhaitez examiner son exécution pas à pas.

3. Pas à Pas Sortant (Step Out – Shift+F11):

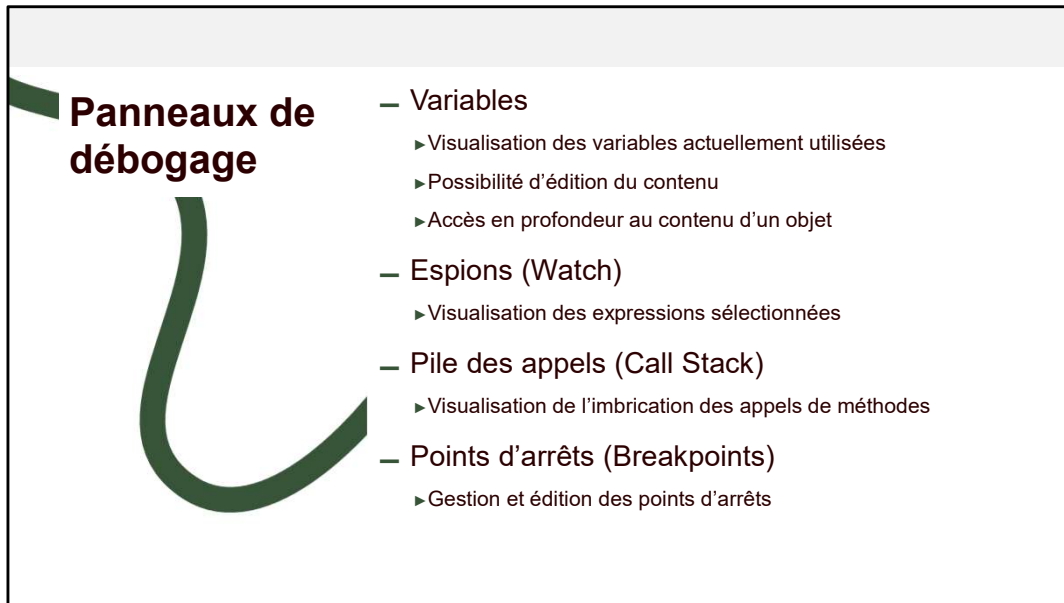
•**Description:** Si vous êtes à l'intérieur d'une méthode et que vous avez déterminé qu'il

n'y a pas de problèmes à l'intérieur, vous pouvez utiliser "Step Out" pour sortir rapidement de cette méthode et revenir au point d'appel dans le code principal.

•**Utilisation:** C'est un gain de temps, car cela évite d'avoir à passer par toutes les lignes restantes d'une méthode lorsque vous savez que tout est correct.

Conclusion:

Le débogage "Pas à Pas" est un outil précieux pour tout développeur. Il offre un niveau de contrôle inégalé sur l'exécution du code, permettant de localiser rapidement les erreurs et d'en comprendre la cause. La maîtrise des différents modes de débogage "Pas à Pas" peut considérablement accélérer le processus de débogage et aider à garantir un code robuste et fiable.



Panneaux de débogage

1. Variables:

• **Description** : Le panneau de variables affiche toutes les variables actuellement en scope pendant une session de débogage. Cela permet aux développeurs de suivre les valeurs actuelles de leurs variables et de détecter tout comportement inattendu.

• **Fonctionnalités** :

- **Édition de contenu** : Certains environnements de développement permettent de modifier la valeur d'une variable pendant le débogage pour tester différents scénarios.
- **Accès en profondeur** : Cette fonction donne la possibilité d'explorer en détail les objets et leurs propriétés, aidant à déceler des erreurs ou des comportements non désirés.

2. Espions (Watch):

• **Description** : Le panneau "Watch" permet aux développeurs d'ajouter des expressions spécifiques qu'ils souhaitent surveiller. Au fur et à mesure que le code s'exécute, l'évaluation de ces expressions est mise à jour en temps réel.

3. Pile des appels (Call Stack):

• **Description** : Ce panneau donne une vue hiérarchique de l'ordre des méthodes appelées, menant à l'endroit actuel de l'exécution. C'est essentiel pour comprendre le chemin d'exécution et localiser où une erreur pourrait s'être introduite.

4. Points d'arrêts (Breakpoints):

•**Description** : Les points d'arrêt sont des marqueurs que les développeurs placent dans le code pour interrompre l'exécution. Une fois qu'un point d'arrêt est atteint pendant le débogage, l'exécution s'arrête, permettant au développeur d'inspecter l'état actuel du programme.

•**Fonctionnalités** :

- **Gestion des points d'arrêts** : Les développeurs peuvent ajouter, supprimer ou désactiver des points d'arrêt selon les besoins.
- **Édition des points d'arrêts** : Dans des outils avancés, il est possible de configurer des points d'arrêt conditionnels qui ne s'activent que lorsque certaines conditions sont remplies.

Conclusion :

Ces panneaux offrent aux développeurs une vue complète et granulaire de l'exécution de leur code, facilitant grandement le processus de débogage. Avoir une compréhension claire de la manière dont chacun de ces outils fonctionne peut rendre le débogage plus efficace et moins frustrant.

Atelier 01



Activité

20 min

- Exercice 1: Création d'un programme simple en C#
- Exercice 2: Utilisation du débogueur



1. **.NET Framework et sa Philosophie** : ".NET est une plateforme de développement polyvalente et puissante qui prend en charge de nombreux langages, C# étant l'un des plus populaires."
2. **Historique et Évolution** : "Le .NET Framework a considérablement évolué au fil des ans, avec .NET 6 représentant la dernière version unifiée, regroupant les meilleures fonctionnalités de ses prédécesseurs."
3. **Comprendre les Assemblies** : "Un Assembly est l'unité de base du déploiement, de la version, des autorisations et de la sécurité dans .NET. Comprendre son fonctionnement est essentiel pour travailler avec .NET."
4. **Rôle du CLR** : "Le Common Language Runtime (CLR) est le moteur au cœur de .NET, responsable de la compilation Just-In-Time (JIT) et de l'exécution des applications .NET."
5. **Pouvoir des Outils .NET** : "Les outils fournis avec .NET, tels que Visual Studio, simplifient et accélèrent le processus de développement, de débogage et de déploiement."
6. **Visual Studio 2022 vs Visual Studio Code** : "Visual Studio 2022 est un IDE complet, tandis que Visual Studio Code est un éditeur de code source léger mais puissant. Chacun a ses propres forces et est adapté à des scénarios d'utilisation spécifiques."
7. **Structuration en C#** : "Les classes et les espaces de noms sont essentiels pour organiser et structurer votre code en C#. Ils aident à encapsuler la logique et à favoriser la réutilisabilité."
8. **Importance de la Documentation** : "Documenter votre code, en particulier avec des commentaires XML, n'est pas seulement pour les autres développeurs. C'est un investissement pour vous-même, pour l'avenir, facilitant la maintenance et la compréhension du code."
9. **Débogage dans Visual Studio 2022** : "Le débogueur de Visual Studio 2022 est un outil puissant qui permet de traquer et de résoudre les problèmes dans le code. Maîtriser les points d'arrêt, le pas à pas et les panneaux de débogage est essentiel pour tout développeur .NET."
10. **Bonnes Pratiques** : "Outre l'écriture et le débogage de code, adopter de bonnes pratiques, comme commenter correctement son code et comprendre la structure d'une application, est essentiel pour la qualité et la pérennité du logiciel."

1..NET Framework et sa Philosophie : ".NET est une plateforme de développement polyvalente et puissante qui prend en charge de nombreux langages, C# étant l'un des plus populaires."

2.Historique et Évolution : "Le .NET Framework a considérablement évolué au fil des ans, avec .NET 6 représentant la dernière version unifiée, regroupant les meilleures fonctionnalités de ses prédécesseurs."

3.Comprendre les Assemblies : "Un Assembly est l'unité de base du déploiement, de la version, des autorisations et de la sécurité dans .NET. Comprendre son fonctionnement est essentiel pour travailler avec .NET."

4.Rôle du CLR : "Le Common Language Runtime (CLR) est le moteur au cœur de .NET, responsable de la compilation Just-In-Time (JIT) et de l'exécution des applications .NET."

5.Pouvoir des Outils .NET : "Les outils fournis avec .NET, tels que Visual Studio, simplifient et accélèrent le processus de développement, de débogage et de déploiement."

6.Visual Studio 2022 vs Visual Studio Code : "Visual Studio 2022 est un IDE complet, tandis que Visual Studio Code est un éditeur de code source léger mais puissant. Chacun a ses propres forces et est adapté à des scénarios d'utilisation spécifiques."

7.Structuration en C# : "Les classes et les espaces de noms sont essentiels pour organiser et structurer votre code en C#. Ils aident à encapsuler la logique et à favoriser la réutilisabilité."

8.Importance de la Documentation : "Documenter votre code, en particulier avec des

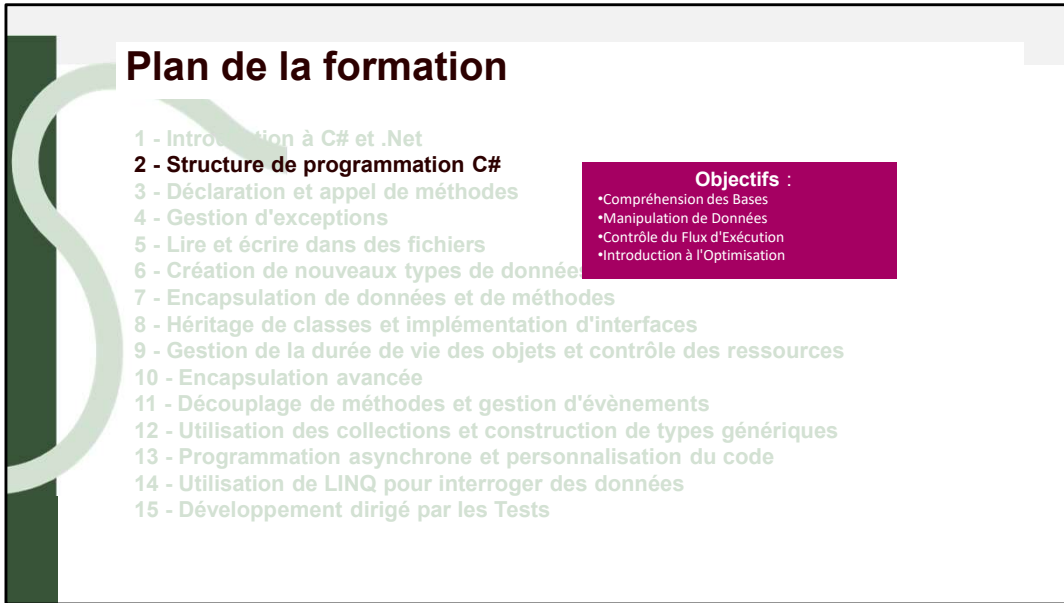
commentaires XML, n'est pas seulement pour les autres développeurs. C'est un investissement pour vous-même, pour l'avenir, facilitant la maintenance et la compréhension du code."

9. Débogage dans Visual Studio 2022 : "Le débogueur de Visual Studio 2022 est un outil puissant qui permet de traquer et de résoudre les problèmes dans le code. Maîtriser les points d'arrêt, le pas à pas et les panneaux de débogage est essentiel pour tout développeur .NET."

10. Bonnes Pratiques : "Outre l'écriture et le débogage de code, adopter de bonnes pratiques, comme commenter correctement son code et comprendre la structure d'une application, est essentiel pour la qualité et la pérennité du logiciel."

Chapitre 2 - Introduction à C# et .Net



A presentation slide titled 'Plan de la formation'. It features a large green 'S' shape on the left side. The title is in bold black text. Below it is a numbered list of 15 topics. To the right of the list is a purple box with the heading 'Objectifs :' and four bullet points.

Plan de la formation

- 1 - Introduction à C# et .Net
- 2 - **Structure de programmation C#**
- 3 - Déclaration et appel de méthodes
- 4 - Gestion d'exceptions
- 5 - Lire et écrire dans des fichiers
- 6 - Création de nouveaux types de données
- 7 - Encapsulation de données et de méthodes
- 8 - Héritage de classes et implémentation d'interfaces
- 9 - Gestion de la durée de vie des objets et contrôle des ressources
- 10 - Encapsulation avancée
- 11 - Découplage de méthodes et gestion d'événements
- 12 - Utilisation des collections et construction de types génériques
- 13 - Programmation asynchrone et personnalisation du code
- 14 - Utilisation de LINQ pour interroger des données
- 15 - Développement dirigé par les Tests

Objectifs :

- Compréhension des Bases
- Manipulation de Données
- Contrôle du Flux d'Exécution
- Introduction à l'Optimisation

Objectifs :

- 1. Compréhension des Bases :** Assimiler les fondamentaux du langage C# pour structurer et organiser le code de manière efficace et lisible.
- 2. Manipulation de Données :** Maîtriser la déclaration de variables, leur typage et l'affectation de valeurs pour stocker et manipuler des données.
- 3. Contrôle du Flux d'Exécution :** Comprendre et utiliser efficacement les structures conditionnelles et de boucle pour diriger le déroulement du programme.
- 4. Introduction à l'Optimisation :** Découvrir les bonnes pratiques pour écrire du code optimisé, maintenable et performant.

Positionnement :

Ce chapitre s'inscrit comme une fondation essentielle pour quiconque souhaite maîtriser le développement en C#. Il aborde les éléments de base nécessaires à la création de n'importe quelle application, des simples scripts aux logiciels les plus complexes. Avant de plonger dans des sujets plus avancés et spécifiques à .NET, il est crucial d'avoir une solide compréhension de ces principes fondamentaux.

Sujets Clés :

- 1. Déclaration de Variables et Typage :** Exploration des types de données intégrés en C#, leur utilisation et les conventions d'affectation.
- 2. Expressions et Opérateurs :** Découverte des différents opérateurs disponibles en C#

(arithmétiques, de comparaison, logiques, etc.) et comment les utiliser dans des expressions.

3. Tableaux : Introduction à la création, l'initialisation et la manipulation de tableaux pour stocker des collections d'éléments.

4. Instructions Conditionnelles : Maîtrise des structures if, else if, et else pour exécuter différents blocs de code en fonction d'une condition.

5. Boucles et Itérations : Exploration des structures for, while, et foreach pour répéter des actions jusqu'à ce qu'une condition soit remplie.

6. Extraits de Code avec Visual Studio 2022 : Démonstration et pratique des concepts abordés à l'aide d'exemples concrets et d'exercices dans l'environnement de Visual Studio 2022.



Commençons
par échanger



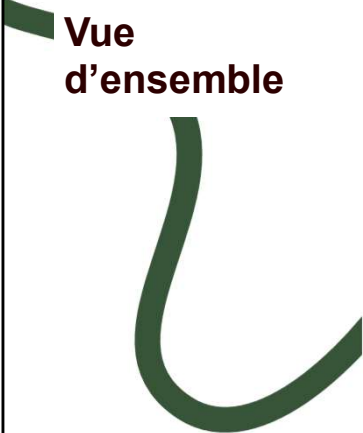
Expérience antérieure avec les langages de programmation :
"Pouvez-vous me donner des exemples de langages de programmation que vous avez utilisés auparavant et comment vous avez structuré votre code dans ces langages ?"

Concept de Variables :
"Dans votre expérience de programmation, comment définiriez-vous une variable et pourquoi est-elle si essentielle dans le développement d'une application ?"

Instructions Conditionnelles :
"Pouvez-vous décrire une situation où vous avez dû utiliser des conditions pour décider de la suite des événements dans une application ou un programme ? Comment avez-vous géré cela ?"

Boucles et Itérations :
"Avez-vous déjà rencontré une situation où vous deviez répéter une tâche plusieurs fois dans votre code ? Comment avez-vous abordé cette répétition et quel type de boucle avez-vous utilisé ?"

Optimisation et Bonnes Pratiques :
"Dans vos projets précédents, comment avez-vous veillé à ce que votre code soit à la fois efficace et lisible ? Avez-vous des exemples de bonnes pratiques que vous avez adoptées ou rencontrées ?"

A large, dark green, stylized bracket or 'C' shape that curves around the text 'Vue d'ensemble' and the list of topics.

Vue d'ensemble

- Déclaration de variables et affectation de valeurs
- Utilisation d'expression et d'opérateurs
- Création et utilisation des tableaux
- Instructions de décision
- Instructions d'itérations
- Extraits de code de Visual Studio 2022

A large green bracket-like graphic on the left side of the slide, spanning the height of the main content area.

Leçon 1: Déclaration de variables et affectation de valeurs

- Définition
- Types de données
- Déclaration et affectation
- Suffixes et Littéraux
- Portée
- Conversion
- Variables lecture seule et constantes
- Typage implicite

Définition

Les variables stockent des valeurs requises par l'application dans des emplacements mémoires temporaires

Les variables sont caractérisées par:

Nom

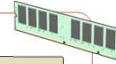
Adresse

Type de données

Valeur

Portée

Durée de vie

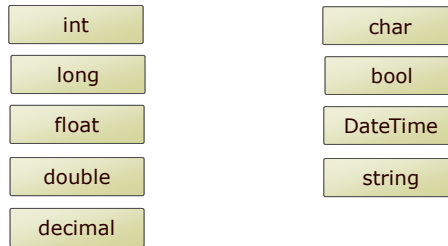


Types de données

C# est un langage typé fort

Le compilateur garantit qu'une valeur stockée dans une variable est du type approprié

Quelques exemples de type de données:



Types de données en C#

1. int :

•**Description** : Un type entier signé qui peut stocker des valeurs comprises entre -2,147,483,648 et 2,147,483,647.

2. char:

•**Description** : Utilisé pour stocker un caractère unique et occupe 2 octets de mémoire.

3. long:

•**Description** : Un type entier signé avec une plage plus large que int, allant de -9,223,372,036,854,775,808 à 9,223,372,036,854,775,807.

4. bool:

•**Description** : Peut stocker l'une des deux valeurs, true ou false.

5. float:

•**Description** : Un nombre à virgule flottante à simple précision. Il peut stocker des nombres fractionnaires. Plus petit en termes de précision par rapport au type double.

6. DateTime:

•**Description** : Utilisé pour représenter une date et une heure.

7. double:

•**Description** : Un nombre à virgule flottante à double précision. Il offre une plus grande précision que le type float.

8. string:

•**Description** : Utilisé pour stocker une série de caractères.

9. decimal:

•**Description** : Utilisé pour stocker des nombres décimaux avec une très grande précision. Idéal pour les calculs financiers.

Conclusion :

La garantie que C# est un langage fortement typé signifie que chaque variable déclarée doit avoir un type de données spécifié. De plus, une fois qu'un type est attribué à une variable, il ne peut pas être modifié. Cela aide à éviter de nombreuses erreurs courantes et garantit que les données sont traitées de manière appropriée et prévisible. La connaissance de ces types de données et de leurs propriétés est essentielle pour tout développeur travaillant en C#.

Déclaration et Affectation

Avant de pouvoir utiliser une variable, il faut la déclarer

```
DataType variableName;  
...  
DataType variableName1, variableName2;  
...  
DataType variableName = new DataType();
```

Après avoir déclaré une variable, il est possible de lui affecter une valeur

```
DataType variableName = value;  
...  
variableName = anotherValue;
```

NOTE: Le nom d'une variable est un identifiant et doit respecter:

- Ne peut contenir que des lettres, chiffres et le caractère _
- Doit débuter par une lettre ou _
- Ne doit pas être un des mots-clés réservés du C# (sauf si préfixé par @)

Suffixes et Littéraux

- Possibilité de typer une valeur numérique par un suffixe

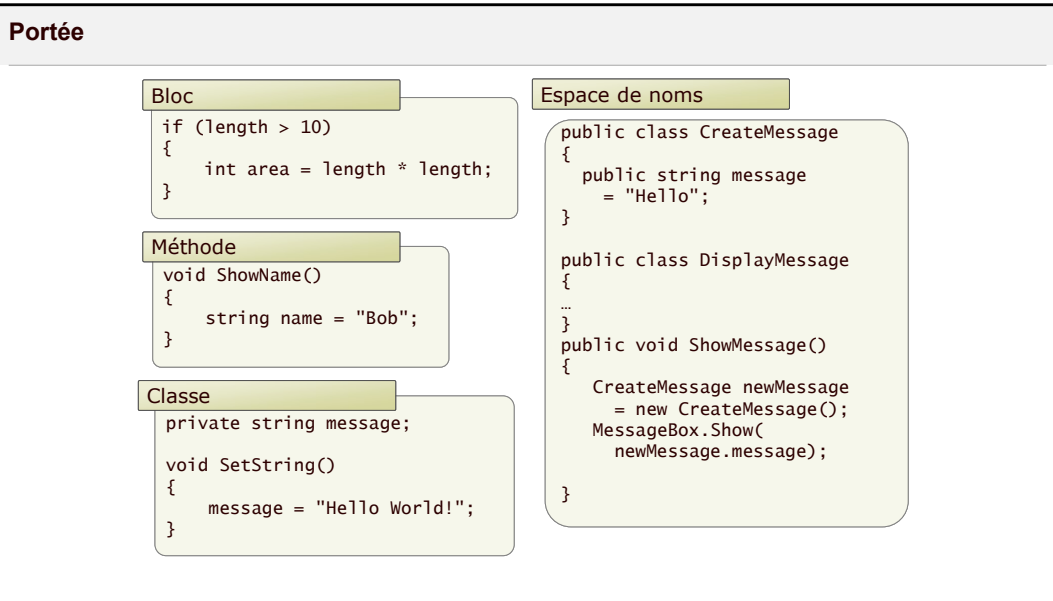
Type :	unsigned int
Caractère :	U
Exemple :	uint x = 100U;
Type :	long
Caractère :	L
Exemple :	long x = 100L;
Type :	unsigned long
Caractère :	UL
Exemple :	ulong x = 100UL;
Type :	float
Caractère :	F
Exemple :	float x = 100F;
Type :	double
Caractère :	D
Exemple :	double x = 100D;
Type :	decimal
Caractère :	M
Exemple :	decimal x = 100M;

- Possibilité d'écrire des littéraux numériques de façon plus lisible

```
byte binaire = 0b01010101;
byte binary = 0b0001_1000;

int hex = 0xAB_BA;

int dec = 1_234_567_890;
```



Conversion

Conversion Implicite

Réalisée automatiquement par le CLR

```
int a = 4;  
long b;  
b = a;           // Conversion implicite int vers long
```

Conversion Explicite

Nécessite d'écrire le code de conversion

```
DataType variableName1 = (castDataType) variableName2;  
...  
int count = Convert.ToInt32("1234");  
...  
int number = int.Parse("1234");  
...  
int number = 0;  
if (int.TryParse("1234", out number)) { // Conversion réussie }
```

Variables lecture seule et constantes

Champs en lecture seule

Déclarés avec le mot clé readonly
Initialisés à l'exécution

```
readonly string currentDateTime = DateTime.Now.ToString();
```

Constantes

Déclarées avec le mot clé const
Initialisées à la compilation

```
const double PI = 3.14159;  
int radius = 5;  
double area = PI * radius * radius;  
double circumference = 2 * PI * radius;
```

Typage implicite

- Grâce au mot-clé var, il est possible de ne pas préciser le type d'une variable lors de sa déclaration
- Obligation d'effectuer une assignation de valeur
- Le type de la valeur affectée est utilisé par le compilateur pour déterminer le type de la variable

```
var myString = "Hello World!"; // myString est typé String  
var data = 5;                  // data est typé int
```

Typage Implicite avec le mot-clé var

Le mot-clé var en C# permet une déclaration de type implicite. C'est-à-dire que vous n'avez pas besoin de spécifier explicitement le type de données d'une variable lors de sa déclaration. Cependant, il y a quelques points clés à noter :

1.Initialisation Obligatoire : Lorsque vous utilisez var, vous devez initialiser la variable au moment de la déclaration. Autrement dit, vous ne pouvez pas simplement déclarer la variable; vous devez également lui attribuer une valeur.

2.Type Déterminé au Moment de la Compilation : Le compilateur détermine le type de la variable en fonction de la valeur que vous lui attribuez. Une fois ce type déterminé, la variable est fortement typée. Cela signifie que vous ne pouvez pas lui attribuer ultérieurement une valeur d'un type différent.

3.Utilisation Locale Seulement : Le mot-clé var ne peut être utilisé que pour les variables locales. Cela signifie que vous ne pouvez pas l'utiliser pour les champs de classe, les propriétés, les paramètres de méthode ou les valeurs de retour.

4.Lisibilité du Code : L'utilisation excessive de var peut rendre le code moins lisible, car il n'est pas immédiatement évident quel est le type de la variable. Il est donc recommandé de l'utiliser judicieusement.

Exemples :

- var name = "John"; // name est implicitement de type string.
- var age = 25; // age est implicitement de type int.
- var isStudent = true; // isStudent est implicitement de type bool.

Conclusion :

Le typage implicite, lorsqu'il est utilisé judicieusement, peut rendre le code C# plus concis et propre. Cependant, il est essentiel de comprendre comment et quand l'utiliser pour garantir la clarté et la robustesse du code.

Leçon 2: Utilisation d'expression et d'opérateurs

- Expressions
- Opérateurs
- Bonnes pratiques de concaténation de chaînes

Expressions

Les expressions sont les structures fondamentales permettant d'évaluer et de manipuler des données

```
a + 1
```

```
(a + b) / 2
```

```
"Answer: " + c.ToString()
```

```
b * System.Math.Tan(theta)
```



Opérateurs

Les opérateurs sont des séquences de caractères utilisés pour définir des opérations réalisées sur des opérandes

Arithmétique +, -, *, /, %	Assignment =, +=, -=, *=, /=
Incrément, décrétement ++, --	Décalage de bit <<, >>
Comparaison ==, !=, <, >, <=, >=, is	Information sizeof, typeof, nameof
Concaténation +	Délégué +, -
Logique/binaire &, , ^, ~, &&,	Dépassement checked, unchecked
Tableaux [], ^, ..	Indirection, Adresse *, ->, [], &
Cast (), as	Conditionnel ?:, ?? (??=), ?.

?: -> (condition)? valeur si vrai : valeur si faux

?? -> variable ?? Valeur si variable null

??= -> assigne la partie droite à la partie gauche si celle-ci est null

? . -> variable ? . Membre invoqué seulement si variable est non null

Bonnes pratiques de concaténation de chaînes

La concaténation de chaînes de caractères peut être effectuée très simplement à l'aide de l'opérateur +

```
string address = "23";  
address = address + ", Oxford Street";  
address = address + ", Thornbury";
```

Cela est toutefois considéré comme une mauvaise pratique car les chaînes de caractères sont immuables (problème de performance)

Bien sûr, poursuivons sur les bonnes pratiques concernant la concaténation de chaînes en C#.

Alternatives pour la Concaténation de Chaînes**1.StringBuilder :**

1. Pour des opérations de concaténation répétées, StringBuilder est souvent plus efficace que la concaténation avec l'opérateur +.
2. Il est particulièrement utile lorsque vous effectuez de nombreuses opérations de concaténation dans une boucle.
3. StringBuilder est mutable, ce qui signifie qu'il ne crée pas une nouvelle instance à chaque modification.

```
StringBuilder sb = new StringBuilder();  
sb.Append("23");  
sb.Append(", Oxford Street");  
sb.Append(", Thornbury");  
string address = sb.ToString();
```

1.String.Concat et String.Join :

1. Pour des opérations de concaténation simples, vous pouvez utiliser String.Concat ou String.Join.
2. String.Join est particulièrement utile si vous avez une collection de chaînes

que vous souhaitez combiner.

```
string address1 = String.Concat("23", " ", "Oxford Street", " ", "Thornbury");  
string[] parts = { "23", "Oxford Street", "Thornbury" };  
string address2 = String.Join(" ", parts);
```

1. Interpolation de Chaînes :

1. À partir de C# 6, vous pouvez utiliser l'interpolation de chaînes pour insérer des variables directement dans une chaîne littérale.
2. C'est une manière élégante et lisible d'assembler des chaînes.

```
string street = "Oxford Street";  
string town = "Thornbury";  
string address3 = $"23, {street}, {town}";
```

À Retenir :

Même si la concaténation de chaînes avec l'opérateur + est simple et directe, elle peut entraîner des problèmes de performance, en particulier dans les boucles. Il est donc essentiel d'évaluer le contexte dans lequel vous travaillez et d'utiliser la méthode la plus appropriée pour vos besoins.

Solutions de concaténation

◦ Première solution: la classe StringBuilder

```
StringBuilder address = new StringBuilder();  
  
address.Append(adresse.Numero);  
address.Append(adresse.Rue);  
address.Append(adresse.Ville);  
  
string concatenatedAddress = address.ToString();
```

Deuxième solution: la méthode String.Format
(on retrouve la même implementation dans Console.WriteLine)

```
string concatenatedAddress = String.Format(  
    "{0}, {1}, {2}",  
    adresse.Numero, adresse.Rue, adresse.Ville);
```

Troisième solution: l'interpolation de chaînes

```
string concatenatedAddress =  
    $"{adresse.Numero}, {adresse.Rue}, {adresse.Ville}";
```

Préfixes de chaînes de caractères

- Le caractère \ est un caractère d'échappement dans une chaîne
 - \ " => ", \ t => tabulation, \ n => nouvelle ligne, \ r => retour, ...
 - Obligation de doubler \ dans un chemin d'accès

- Utilisation du préfixe @ pour que \ ne soit plus interprété

```
string path = "C:\\Users\\Admin\\Documents\\File.txt";
```

- Il est possible de cumuler @ et \$ (@\$"... " ou @\$@"... ")

```
string path = @$"C:\Users\Admin\Documents\File.txt";
```

Leçon 3: Création et utilisation de tableaux

- Définition
- Création et Initialisation
- Quelques propriétés et méthodes
- Accès aux données dans un tableau
- Indices et Plages

Définition

- Un tableau est une séquence d'éléments de même type, groupés ensemble
 - Chaque élément du tableau contient une valeur
 - Les indices sont numérotés à partir de zéro
 - La longueur d'un tableau est le nombre maximum d'éléments qu'il peut contenir
 - Le rang d'un tableau correspond à son nombre de dimensions
 - Les tableaux peuvent être unidimensionnels, multidimensionnels ou imbriqués
 - Les tableaux ont une taille fixe en C#

Manipulation des Tableaux en C#**1. Déclaration et Initialisation:**

•Vous pouvez déclarer un tableau sans l'initialiser, mais vous devez spécifier son type.
int[] numbers;

Vous pouvez aussi déclarer et initialiser un tableau en même temps.

int[] numbers = {1, 2, 3, 4, 5};

2. Accès aux Éléments du Tableau:

•Utilisez l'index pour accéder à un élément spécifique.

int firstNumber = numbers[0]; // Récupère le premier élément

3. Modification des Éléments du Tableau:

•Utilisez l'index pour modifier un élément spécifique.

numbers[0] = 10; // Modifie le premier élément à 10

4. Tableaux Multidimensionnels:

•Les tableaux peuvent avoir plus d'une dimension.

```
int[,] matrix = new int[3, 3]; // Un tableau 3x3  
matrix[0, 0] = 1; // Définit la première cellule
```

5. Parcours des Éléments avec une Boucle:

• Vous pouvez utiliser une boucle for ou foreach pour parcourir tous les éléments.

```
foreach(int number in numbers)  
{  
    Console.WriteLine(number);  
}
```

6. Méthodes Utiles:

- Length: Retourne la longueur totale du tableau.
- Rank: Retourne le nombre de dimensions du tableau.
- GetValue et SetValue: Permettent d'obtenir ou de définir la valeur d'un élément spécifique.
- Array.Sort(): Trie les éléments du tableau.
- Array.Reverse(): Inverse l'ordre des éléments.

À Retenir :

En C#, les tableaux sont des objets qui peuvent stocker plusieurs éléments du même type. Ils ont une taille fixe, ce qui signifie que vous ne pouvez pas ajouter ou supprimer des éléments après leur création. Si vous avez besoin d'une structure plus flexible, vous pourriez envisager d'utiliser des listes ou d'autres collections.

Création et Initialisation

Unidimensionnel

```
Type[] arrayName = new Type[ Size ];  
int[] monTableau = new int[10];
```

Multidimensionnel

```
Type[ , ] arrayName = new Type[ Size1, Size2];  
int[,] maMatrice = new int[10,10];
```

Imbriqué

```
Type[][] JaggedArray = new Type[size][];  
int[][] mesCinqTableaux = new int[5][];
```

Quelques propriétés et méthodes

Length

```
int[] oldNumbers = new int[5]{ 1, 2, 3, 4, 5 };
int numberCount = oldNumbers.Length;
int numberRank = oldNumbers.GetLength(0);
```

Rank

```
int[] oldNumbers = { 1, 2, 3, 4, 5 };
int rank = oldNumbers.Rank;
```

CopyTo()

```
int[] oldNumbers = { 1, 2, 3, 4, 5 };
int[] newNumbers = new int[oldNumbers.Length];
oldNumbers.CopyTo(newNumbers, 0);
```

Sort()

```
int[] oldNumbers = { 5, 2, 1, 3, 4 };
Array.Sort(oldNumbers);
```


Accès aux données dans un tableau

Les indices d'un tableau de N éléments vont de 0 à N-1

Accès à un élément spécifique

```
int[] oldNumbers = { 1, 2, 3, 4, 5 };  
int number = oldNumbers[2];
```

Parcourir tous les éléments

```
int[] oldNumbers = { 1, 2, 3, 4, 5 };  
...  
for (int i = 0; i < oldNumbers.Length; i++)  
{  
    int number= oldNumbers[i];  
    Console.WriteLine(number);  
}  
// OU  
foreach (int number in oldNumbers) {  
    Console.WriteLine(number);  
}
```

Indices et Plages

- System.Index: Parcourir le tableau depuis sa fin à l'aide de l'opérateur ^ (décompte à partir de 1)

```
int[] oldNumbers = { 1, 2, 3, 4, 5 };  
int number = oldNumbers[4];  
int sameNumber = oldNumbers[^1];
```

- System.Range: Considérer une plage de valeurs dans le tableau à l'aide de l'opérateur .. (du type [x,y[)

```
int[] oldNumbers = { 1, 2, 3, 4, 5 };  
int number = oldNumbers[1..2]; // 2  
int[] extract = oldNumbers[..3]; // {1, 2, 3}  
int[] otherExtract = oldNumbers[2..]; // {3, 4, 5}  
...  
Range plage = 1..4;  
int[] morceau = oldNumbers[plage];
```

Leçon 4: Instructions de décision

- Instruction if
- Instruction if else
- Imbrication d'instructions if
- Instruction switch
- Critères Spéciaux

Instructions if

Syntaxe

```
if ([condition]) [code à executer]

// ou

if ([condition])
{
    [code à executer si la condition est vraie]
}
```

Opérateurs Conditionnels:

OU est représenté par ||

ET est représenté par &&

Exemple

```
if ((percent >= 0) && (percent <= 100))
{
    // Ajouter le code à executer si percent est entre 0 et 100.
}
```

Instructions if else

Permet de traiter le cas où la condition du Si est fausse

Exemple

```
if (a > 50)
{
    // a est supérieur à 50
}
else
{
    // a est inférieur ou égal à 50
}

// OU

string carColor = "green";

string response = (carColor == "red") ?
    "You have a red car" :
    "You do not have a red car";
```

Imbrication d'instructions if

Il est tout à fait possible de combiner plusieurs instructions Si-Sinon en une seule

Exemple

```
if (a > 50)
{
    // a est plus grand que 50
}
else if (a > 10)
{
    // a est plus grand que 10 et inférieur ou égal à 50
}
else
{
    // a est inférieur ou égal à 10
}
```

Instruction switch

L'instruction switch permet de simplifier l'écriture de tests multiples sur la même variable

Exemple

```
switch (a)
{
    case 0:
        // a vaut 0
        break;

    case 1:
    case 2:
    case 3:
        // a vaut 1, 2 ou 3
        break;

    default:
        // a est différent de 0, 1, 2 et 3
        break;
}
```

Le *break* est obligatoire
si du code est écrit dans
le *case*



Critères Spéciaux (Pattern Matching)

- Permet de déterminer si une valeur correspond à des cas prédéterminés
- Implémenté par l'expression `is` et une extension de l'instruction `switch`
- Expression `is`: étend les tests de type en ajoutant une déclaration de variable typée
- Expression `switch`: étend les cas du `switch` en validant le type. L'ordre des cas doit être contrôlé avec précaution.

Utilisation de Tuples en C#**1.Définition :**

- Un tuple est une collection ordonnée d'éléments hétérogènes.
- Il permet de regrouper plusieurs valeurs sans avoir à créer un type spécifique.

2.Déclaration et Initialisation :

- Les tuples peuvent être déclarés en utilisant la syntaxe (Type1, Type2,...).
 - `(int, string) person = (1, "John");`

1.Accès aux Éléments :

- Les éléments d'un tuple peuvent être accédés en utilisant des noms d'item par défaut Item1, Item2, etc.
 - `int id = person.Item1; string name = person.Item2;`
- Ils peuvent également être nommés pour plus de clarté.
- `(int Id, string Name) namedPerson = (1, "John"); string personName = namedPerson.Name;`

1.Avantages :

- Les tuples sont immuables, ce qui signifie que leurs valeurs ne peuvent pas être modifiées après l'initialisation.
- Ils sont utiles pour retourner plusieurs valeurs à partir d'une méthode sans avoir à définir un type de retour personnalisé.

2.Limitations :

- Les tuples ne sont pas recommandés pour les scénarios de données publiques, car ils n'offrent pas de sémantique claire.

1. Destructuration :

- Les tuples peuvent être "déstructurés" pour affecter leurs éléments à des variables distinctes.
 - `(int personId, string personName) = person;`

À Retenir :

Les tuples en C# sont un moyen pratique de regrouper un ensemble fixe d'éléments hétérogènes. Cependant, pour les scénarios nécessitant une structure plus formelle ou des opérations supplémentaires sur les données, il peut être préférable d'utiliser des classes ou des structures.

Critères Spéciaux: expression is

```
public static int DiceSum2(object[] values)
{
    int sum = 0;
    foreach(object item in values)
    {
        if (item is int val)
            sum += val;
        else if (item is object[] subList)
            sum += DiceSum2(subList);
    }
    return sum;
}
```

Critères Spéciaux: expression switch

```
public static int DiceSum4(object[] values)
{
    int sum = 0;
    foreach (object item in values)
    {
        switch (item)
        {
            case 0:
                break;
            case int val:
                sum += val;
                break;
            case object[] subList when subList.Length >= 1:
                sum += DiceSum4(subList);
                break;
            case object[] subList:
                break;
            case null:
                break;
            default:
                throw new InvalidOperationException("unknown item type");
        }
    }
    return sum;
}
```

Critères Spéciaux

- D'autres modes d'utilisation sont possibles:
- Réécriture plus concise des switch à retour de valeur
- Modèles de propriétés
 - switch sur un objet avec filtre sur une propriété
- Modèles de tuples
 - Passage d'un tuple à un switch
- Modèles positionnels
 - switch sur un objet par déconstruction en tuple
- Combinaison de modèles
 - Utilisation des mots-clés and et or, notations entre parenthèses

D'accord, approfondissons d'autres aspects de C#.

L'utilisation des Records en C#**1.Définition :**

Introduits dans C# 9.0, les records sont un type de référence immuable qui utilise la valeur sémantique pour l'égalité.

Ils sont conçus pour être des objets de données légers qui ne changent pas après leur création.

2.Déclaration et Initialisation :

```
record Person(string FirstName, string LastName);
```

1.Immuabilité :

Une fois qu'un record est créé, il ne peut pas être modifié. Pour apporter des changements, un nouveau record doit être créé.

C# offre une syntaxe concise pour cela avec le mot-clé with.

```
var john = new Person("John", "Doe"); var jane = john with { FirstName = "Jane" };
```

1.Égalité de valeur :

Contrairement aux classes, les records utilisent l'égalité de valeur. Si deux records ont les mêmes valeurs pour tous leurs membres, ils sont considérés comme égaux.

2.Destructuration :

Tout comme les tuples, les records peuvent être déstructurés en variables.

```
var (firstName, lastName) = john;
```

1.Méthodes intégrées :

Les records fournissent des implémentations par défaut des méthodes ToString, GetHashCode, et Equals.

2.Limitations :

Bien que les records offrent de nombreuses fonctionnalités par défaut, ils peuvent ne pas être adaptés aux scénarios nécessitant une logique métier complexe ou des comportements mutables.

À Retenir :

Les records sont un ajout récent à C# conçu pour simplifier la création d'objets immuables. Ils sont particulièrement utiles dans des scénarios où l'immutabilité et l'égalité de valeur sont importantes, comme la programmation fonctionnelle ou la construction de modèles de données dans les applications.

Lesson 5: Instructions d'itérations

- Les différentes instructions d'itération
- Instruction while
- Instruction do
- Instruction for
- Instructions break et continue

Les différentes instructions d'itération**while**

Représente une itération conditionnée de type Tant Que

do

Représente une itération conditionnée de type Faire/Tant Que

for

Représente une itération dont le nombre d'itérations est connu

Les instructions d'itération avancées en C#**1.for each (ou foreach)**

Utilisé pour itérer sur des collections (comme les tableaux ou les listes) sans avoir besoin de connaître leur taille ou d'utiliser un indice.

```
int[] numbers = { 1, 2, 3, 4, 5 }; foreach (int number in numbers) {  
Console.WriteLine(number); }
```

1.break

Utilisé pour sortir prématurément d'une boucle.

```
while (true) { if (condition) { break; } }
```

1.continue

Passe à la prochaine itération de la boucle, en sautant le reste du code dans la boucle courante.

```
for (int i = 0; i < 10; i++) { if (i % 2 == 0) { continue; } Console.WriteLine(i); }
```

1.goto

Bien que généralement évité en programmation moderne en raison de sa complexité potentielle, goto peut être utilisé pour sauter à un point spécifié dans le code.

```
start: Console.WriteLine("This will loop until a condition is met."); if (!condition) { goto  
start; }
```

À Retenir :

C# offre une variété d'instructions d'itération pour répondre à différents besoins. Il est

essentiel de choisir la bonne instruction pour le bon scénario pour garantir la clarté et l'efficacité du code. Il est également crucial d'utiliser des instructions comme goto avec prudence, car elles peuvent rendre le code difficile à lire et à maintenir.

Instruction while

- Boucle de type 0..N
- La condition est testée en début d'itération
- Le code d'itération est entre accolade s'il est défini de plusieurs instructions

Exemple

```
double balance = 100D;  
double rate = 2.5D;  
double targetBalance = 1000D;  
int years = 0;  
while (balance <= targetBalance)  
{  
    balance *= (rate / 100) + 1;  
    years += 1;  
}
```

Instruction do-while

- Boucle de type 0.1-N.
- Contrairement à while, la condition est testée à la **fin** de l'itération, garantissant ainsi qu'au moins une exécution du corps de la boucle se produit.
- Le code d'itération est placé entre accolades s'il est défini par plusieurs instructions.

À Retenir :

La boucle do-while est utile lorsque vous souhaitez que le corps de la boucle s'exécute au moins une fois avant de tester la condition. C'est souvent le cas pour les menus interactifs ou lorsque vous demandez une entrée utilisateur et que vous souhaitez traiter cette entrée au moins une fois.

Instruction do

- Boucle de type 1..N
- La condition est testée en fin d'itération
- L'instruction se termine par un point-virgule

Exemple

```
string userInput = "";  
do  
{  
    userInput = GetUserInput();  
    if (userInput.Length < 5)  
    {  
        // Traitement si la chaîne fait moins de 5 caractères  
    }  
} while (userInput.Length < 5);
```

Instruction for

- Boucle de type déterministe, généralement utilisée lorsque le nombre d'itérations est connu à l'avance.
- Constituée de trois parties:
 - Initialisation
 - Condition de continuation
 - Incrémentation ou décrémentation
- L'initialisation est effectuée une seule fois, au début de la boucle.

Exemple :

```
for(int i = 0; i < 10; i++)  
{  
    Console.WriteLine($"Valeur actuelle de i: {i}");  
}
```

À Retenir :

La boucle for est utile lorsque vous connaissez à l'avance le nombre d'itérations à effectuer, par exemple pour parcourir un tableau ou une liste. Elle offre un contrôle précis sur la manière dont la boucle évolue à chaque itération.

Instruction for

La boucle est définie à l'aide de trois parties:

for (initialisation; condition; fin d'itération) { ... }

- L'initialisation est exécutée une seule fois en début d'instruction
- La condition est testée à chaque itération (de type tant que)
- La fin d'itération est exécutée en fin de chaque itération

Example

```
for (int i = 0; i < 10; i++)  
{  
    // code à exécuter, pouvant utiliser i  
}
```

Instructions break et continue

Break

```
while (oldNumbers.Length > count)
{
    if (oldNumbers[count] == 5)
    {
        break;
    }
    count++;
}
```

Permet de sortir de la boucle

Continue

```
while (oldNumbers.Length > count)
{
    if (oldNumbers[count]%2==1)
    {
        count++; continue;
    }
    // code pour valeurs paires
    count++;
}
```

Permet d'interrompre la boucle et de reprendre l'exécution à l'itération suivante

Lesson 6: Extraits de code de Visual Studio 2022

- Définition
- Exemples
- Création

Définition

- Les extraits de code (Code Snippets) sont des fragments de code prêt à l'emploi
- Peuvent être appelés par un mot-clé suivi d'une double tabulation ou par un clic-droit
- Certains extraits sont conçus pour encadrer un code existant
- Peuvent être paramétrés, la navigation entre chaque paramètre se faisant par tabulation
- Les extraits de code (*.snippet) sont stockés dans
C:\Program Files\Microsoft Visual Studio\2022\Community

Définition:

Extraits de code (Code Snippets) : Fragments de code préconfigurés dans un environnement de développement qui permettent une écriture rapide en évitant la redondance.

Explication:

Les extraits de code, connus sous le nom de "Code Snippets", sont conçus pour faciliter la vie des développeurs. Ils représentent des portions de code fréquemment utilisées et qui sont prêtes à l'emploi. Par exemple, dans Visual Studio, si un développeur souhaite créer une boucle for, il peut simplement taper un mot-clé spécifique, suivi d'une double tabulation, et le modèle de boucle for apparaîtra automatiquement.

En outre, certains extraits sont spécialement conçus pour encadrer un code existant.

Cela signifie que si vous avez déjà écrit une partie de code et que vous souhaitez l'entourer d'une structure particulière, un extrait de code peut le faire pour vous.

La personnalisation est également un aspect essentiel des extraits de code. Ils peuvent être paramétrés, permettant ainsi une adaptation selon les besoins spécifiques du code.

La navigation entre ces paramètres est aisée et se fait simplement par tabulation.

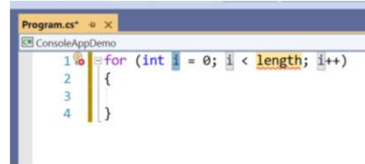
Enfin, pour ceux qui souhaitent explorer davantage ou même ajouter leurs propres extraits de code, ils sont stockés dans un emplacement spécifique sur l'ordinateur, généralement dans le répertoire d'installation de l'environnement de développement, comme Visual Studio.

À retenir:

Les "Code Snippets" sont des outils essentiels pour tout développeur cherchant à accélérer son processus de codage tout en garantissant la cohérence et la conformité du code. Dans Visual Studio, ils peuvent être facilement invoqués, adaptés et même personnalisés, faisant d'eux un atout inestimable pour une programmation efficace.

Exemples

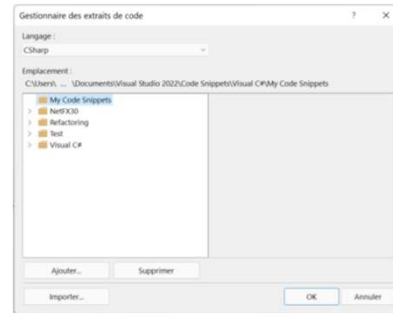
- Structures itératives: for, for, foreach, while, ...
- Structures conditionnelles: if, switch, ...



```
Program.cs*  
ConsoleAppDemo  
1 for (int i = 0; i < length; i++)  
2 {  
3  
4 }
```


Création

- Les extraits personnalisés sont stockés dans
Mes Documents\Visual Studio 2022\Code Snippets
- Pour créer un nouvel extrait de code, il suffit de dupliquer un fichier
.snippet existant et de l'importer dans Visual Studio via le
Gestionnaire d'extrait de code



Création (suite)

```
<?xml version="1.0" encoding="utf-8"?>
<CodeSnippets xmlns="http://schemas.microsoft.com/VisualStudio/2005/CodeSnippet">
  <CodeSnippet Format="1.0.0">
    <Header>
      <Title>for</Title>
      <Shortcut>for</Shortcut>
      <Description>Extrait de code pour une boucle 'for'</Description>
      <Author>Microsoft Corporation</Author>
      <SnippetTypes>
        <SnippetType>Expansion</SnippetType>
        <SnippetType>SurroundsWith</SnippetType>
      </SnippetTypes>
    </Header>
    <Snippet>
      <Declarations>
        <Literal>
          <ID>index</ID>
          <Default>i</Default>
          <ToolTip>Index</ToolTip>
        </Literal>
        <Literal>
          <ID>max</ID>
          <Default>length</Default>
          <ToolTip>Longueur max</ToolTip>
        </Literal>
      </Declarations>
      <Code Language="csharp"><![CDATA[for (int $index$ = 0; $index$ < $max$; $index$++)
{
    $selected$ $end$
}]]>
    </Code>
  </Snippet>
</CodeSnippet>
</CodeSnippets>
```

Atelier 01



Activité
20 min

- Exercice 1: Reconnaître des erreurs de syntaxe en C#
- Exercice 2: Le jeu du plus ou moins



A retenir

Déclaration de Variables :
"En C#, chaque variable a un type, définissant la nature des données qu'elle peut contenir. Bien déclarer une variable est la première étape pour garantir la fiabilité de votre code."

Affectation de Valeurs :
"L'affectation correcte des valeurs aux variables est cruciale pour prévenir les erreurs inattendues à l'exécution."

Expression et Opérateurs :
"Les opérateurs permettent de manipuler les valeurs des variables. Comprendre leur priorité et leur fonctionnement est essentiel pour éviter les erreurs logiques."

Tableaux :
"Un tableau est une collection ordonnée. En C#, ils sont à taille fixe, et il est crucial de savoir les initialiser et y accéder correctement."

Instructions Conditionnelles :
"Les structures if, else if et else sont le cœur du contrôle de flux. Elles dirigent l'exécution du code selon les conditions données."

Boucles et Itérations :
"Les boucles comme for, while et foreach permettent d'exécuter du code de manière répétée. Elles sont fondamentales pour traiter des collections de données."

Optimisation du Code :
"Bien que C# offre plusieurs façons d'aborder un problème, il est essentiel de choisir la structure la plus adaptée pour une exécution optimale et une lisibilité du code."

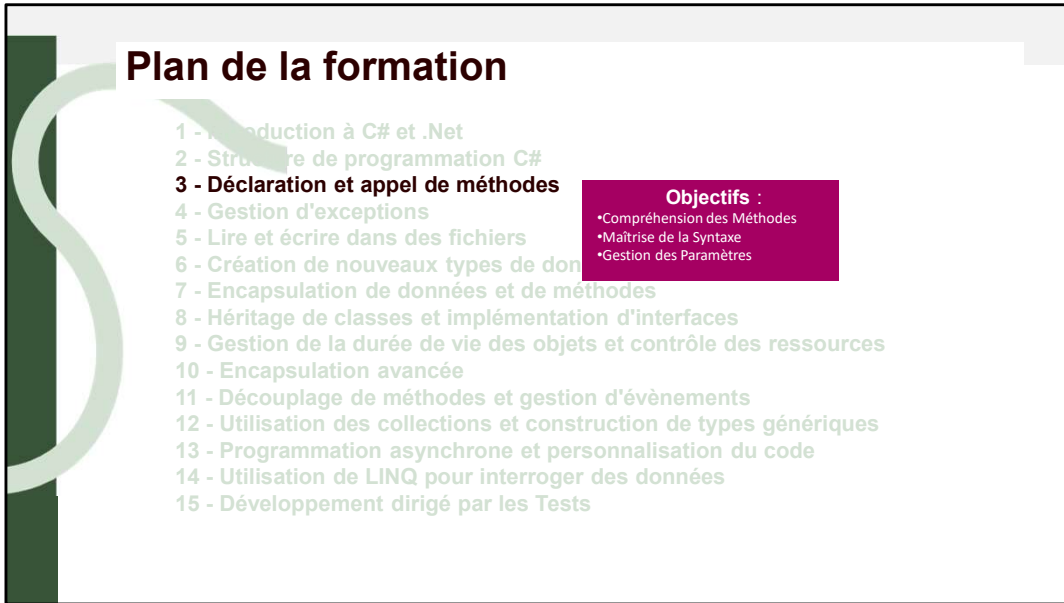
Extraits de Code avec Visual Studio 2022 :
"Visual Studio 2022 est un allié précieux. Il facilite la rédaction, le débogage, et l'optimisation du code C#. Utilisez-le à son plein potentiel."

Éviter les Erreurs Courantes :
"Comprendre ces concepts fondamentaux est la clé pour prévenir des erreurs courantes, comme les dépassements de tableau ou les boucles infinies."

Importance des Bonnes Pratiques :
"Au-delà de la syntaxe, adopter de bonnes pratiques dès le départ permet de garantir un code propre, maintenable, et efficace."

Chapitre 3 - Déclaration et appel de Méthodes





Plan de la formation

- 1 - Introduction à C# et .Net
- 2 - Structure de programmation C#
- 3 - Déclaration et appel de méthodes**
- 4 - Gestion d'exceptions
- 5 - Lire et écrire dans des fichiers
- 6 - Création de nouveaux types de données
- 7 - Encapsulation de données et de méthodes
- 8 - Héritage de classes et implémentation d'interfaces
- 9 - Gestion de la durée de vie des objets et contrôle des ressources
- 10 - Encapsulation avancée
- 11 - Découplage de méthodes et gestion d'événements
- 12 - Utilisation des collections et construction de types génériques
- 13 - Programmation asynchrone et personnalisation du code
- 14 - Utilisation de LINQ pour interroger des données
- 15 - Développement dirigé par les Tests

Objectifs :

- Compréhension des Méthodes
- Maîtrise de la Syntaxe
- Gestion des Paramètres

Objectifs :

1. Compréhension des Méthodes : Faire comprendre aux apprenants l'importance des méthodes dans le développement en C# et comment elles améliorent la modularité et la réutilisabilité du code.

2. Maîtrise de la Syntaxe : Assurer que les apprenants maîtrisent la syntaxe correcte pour déclarer, définir, et appeler des méthodes en C#.

3. Gestion des Paramètres : Apprendre comment passer des informations aux méthodes via des paramètres et comment obtenir des résultats.

Positionnement : Les méthodes sont le fondement de la structuration du code en C#. En apprenant à déclarer et à appeler efficacement des méthodes, les développeurs peuvent créer des programmes modulaires, bien organisés et maintenables. Ce chapitre se positionne comme un élément essentiel pour tout développeur C#, permettant de passer d'une simple écriture de scripts à une véritable conception orientée objet.

Sujets Clés qui seront couverts :

1. Introduction aux Méthodes : Comprendre qu'est-ce qu'une méthode et pourquoi elles sont indispensables en programmation.

2. Déclaration de Méthodes : Apprendre la syntaxe correcte pour déclarer une méthode en C#.

3. Appel de Méthodes : Comment appeler une méthode depuis un autre endroit du code et les conventions associées.

4. Paramètres de Méthodes : Distinguer entre les paramètres en entrée, en sortie et les

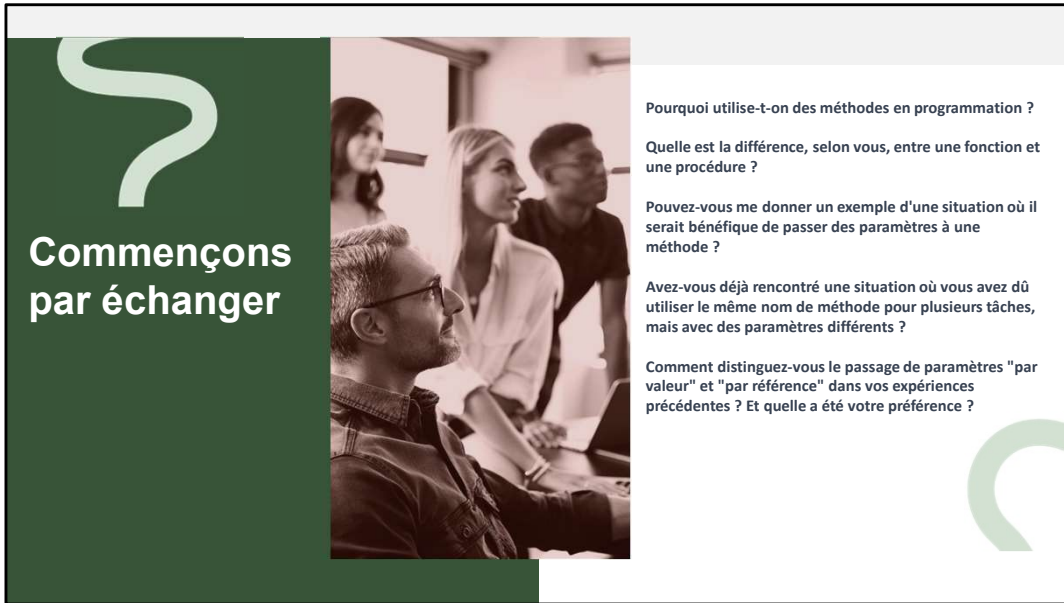
valeurs de retour.

5. Surcharge de Méthodes : Comprendre comment et pourquoi déclarer plusieurs méthodes avec le même nom mais avec des listes de paramètres différentes.

6.Méthodes avec des Paramètres Optionnels et Par Défaut : Comment définir des valeurs par défaut pour les paramètres et les avantages associés.

7.Passage de Paramètres par Valeur et par Référence : La différence entre les deux et quand utiliser chacun.

8.Méthodes Récursives : Introduction à la récursion et ses applications pratiques.



Commençons par échanger

Pourquoi utilise-t-on des méthodes en programmation ?

Quelle est la différence, selon vous, entre une fonction et une procédure ?

Pouvez-vous me donner un exemple d'une situation où il serait bénéfique de passer des paramètres à une méthode ?

Avez-vous déjà rencontré une situation où vous avez dû utiliser le même nom de méthode pour plusieurs tâches, mais avec des paramètres différents ?

Comment distinguez-vous le passage de paramètres "par valeur" et "par référence" dans vos expériences précédentes ? Et quelle a été votre préférence ?

1. Pourquoi utilise-t-on des méthodes en programmation ?

1. Cette question vise à évaluer la compréhension des apprenants quant à l'utilité et à la nécessité des méthodes. Elle encourage la réflexion sur la modularité, la réutilisabilité et l'organisation du code.

2. Quelle est la différence, selon vous, entre une fonction et une procédure ?

1. Une manière d'amener les apprenants à discuter des méthodes qui retournent une valeur versus celles qui effectuent une action sans retourner de valeur.

3. Pouvez-vous me donner un exemple d'une situation où il serait bénéfique de passer des paramètres à une méthode ?

1. Cette question encourage les apprenants à réfléchir à la façon dont les méthodes peuvent être utilisées pour effectuer des actions spécifiques basées sur différentes entrées.

4. Avez-vous déjà rencontré une situation où vous avez dû utiliser le même nom de méthode pour plusieurs tâches, mais avec des paramètres différents ?

1. Cette question met en lumière la surcharge de méthodes et pourquoi elle pourrait être nécessaire.

5. Comment distinguez-vous le passage de paramètres "par valeur" et "par référence" dans vos expériences précédentes ? Et quelle a été votre préférence ?

1. C'est une occasion pour les apprenants d'explorer leurs expériences précédentes avec les différentes manières de passer des paramètres à une

méthode et de discuter des avantages/inconvénients de chacune.

Vue d'ensemble

- Définir et appeler des méthodes
- Passage de paramètres

Leçon 1: Définir et appeler des Méthodes

- Définition
- Création de méthodes
- Invocation de méthodes
- Surcharge
- Paramètres de type tableau
- Refactorisation
- Démonstration: Refactorisation

Définition

- Tout code exécutable doit appartenir à une méthode
- Une application C# doit contenir au minimum une méthode (le point d'entrée du programme Main)
- Une méthode peut retourner un résultat (une fonction) ou simplement effectuer une action (une procédure)
- Une méthode ne peut pas exister en dehors d'une classe mais peut être une méthode d'instance ou statique

Méthode d'instance

```
int count = 99;  
string strCount =  
    count.ToString();
```

Méthode statique

```
string strCount = "99";  
count =  
    Convert.ToInt32(strCount);
```

Création de méthodes

Une méthode contient:

- Une spécification (accès, type de retour et signature)
- Un corps (code)

Signature de méthode:

1 Nom (Pascal case)

2 paramètres (camel case)

La signature de la méthode doit être unique dans la portée où elle est définie

Exemple

```
bool LockReport(string userName)
{
    bool success = false;
    // Traitement
    return success;
}
```

Invocation de méthodes

Pour invoquer une méthode:

- Spécifier le nom de la méthode
- Fournir un argument pour chaque paramètre
- Gérer la valeur de retour

Exemple de méthode

```
bool LockReport(string reportName, string userName)
{
    ...;
}
```

Invocation

```
bool isReportLocked =
    LockReport("Medical Report", "Don Hall");
```

Note: Les arguments sont évalués de gauche à droite. Il s'agit du passage par position

Surcharge

Une méthode surchargée:

- a le même nom qu'une méthode existante
- *devrait* réaliser le même type d'opération
- utilise des paramètres différents

Le compilateur invoque la méthode qui correspond au nombre et au type des paramètres fournis

Exemple

```
void Deposit(decimal amount)
{
    _balance += amount;
}

void Deposit(int dollars, int cents)
{
    _balance += dollars + (cents / 100.0m);
}
```

La signature est
différente pour
chaque
implémentation
de la méthode

Paramètres de type tableau

La surcharge n'est pas toujours appropriée pour traiter des listes d'arguments variables.

Un seul paramètre de type params est autorisé (le dernier).

Exemple

```
int Add(params int[] data)
{
    int sum = 0;
    for (int i = 0; i < data.Length; i++)
    {
        sum += data[i];
    }
    return sum;
}
```

Note: Il n'est pas obligatoire de passer un argument de type tableau même si cela est également possible

Method call

```
int sum = Add(99, 2, 55, -26);
```


Refactorisation

La refactorisation permet d'améliorer la lisibilité et la modularité de son code en réalisant l'extraction de blocs de code répétés sous forme de méthodes.

Visual Studio et Visual Studio Code permettent de sélectionner le code à refactoriser et via la petite ampoule (Ctrl+;), d'Extraire la méthode tout en détectant les potentiels paramètres.

Exemple de résultat de refactorisation

```
LogMessage(messageContents, filePath);  
...  
private void LogMessage(string messageContents, string filePath)  
{  
    ...  
}
```

A noter qu'il ne faut pas utiliser la syntaxe Top-Level Statement pour pouvoir utiliser cette fonctionnalité

Démonstration

- Refactorisation de méthodes

Leçon 2: Passage de paramètres

- Paramètres optionnels
- Passage de paramètres nommés
- Paramètres de sortie
- Variable de sortie
- Tuples
- Discards

Paramètres Optionnels

Les paramètres optionnels permettent de fournir des valeurs par défaut pour certains paramètres d'une méthode:

- Ils assurent une meilleure interopérabilité avec des langages supportant les paramètres facultatifs
- Ils offrent une alternative élégante à la problématique des surcharges multiples

Exemple

```
void MyMethod(  
    int intData, float floatData, int moreIntData = 99)  
{  
    ...  
}
```

```
// Un argument pour chaque paramètre  
MyMethod(1, 123.45, 66);
```

```
// Seulement deux arguments pour les deux premiers paramètres  
MyMethod(100, 54.321);
```

Passage de paramètres nommés

- Alternative au passage par position
- Les arguments sont fournis dans n'importe quel ordre
- Les valeurs manquantes sont fournies par les valeurs par défaut de la méthode
- Les deux types de passage sont cumulables à condition de commencer avec les arguments par position

Exemple

```
void MyMethod(  
    int intData, float floatData = 101.1F, int moreIntData = 99)  
{  
    ...  
}
```

```
MyMethod(moreIntData: 100, intData: 10);  
// floatData recevra la valeur par défaut 101.1F
```

Modification de paramètres

- Par défaut le passage d'un paramètre se fait **par valeur**:
 - Une copie de la variable est passée au paramètre
 - Les modifications apportées au paramètre ne sont pas reportées sur la variable
- Il est possible d'utiliser le mot-clé **ref** pour effectuer un passage **par référence**:
 - La variable est référencée par le paramètre
 - Toute modification du paramètre est appliquée à la variable
 - Le paramètre est considéré comme entrée-sortie
 - La variable doit être initialisée avant son utilisation

Définitions :

Passage par valeur : Lorsqu'une variable est passée à une fonction en tant que paramètre, une copie de cette variable est transmise. Ainsi, tout changement apporté à cette copie à l'intérieur de la fonction n'affecte pas la variable originale à l'extérieur.

Passage par référence (ref) : Une méthode de transmission de variable où le paramètre fait référence à l'adresse mémoire originale de la variable, plutôt qu'à une copie de sa valeur. Cela permet à la fonction de modifier directement la valeur de la variable originale.

Explications :

Passage par valeur : C'est la manière par défaut dont les variables sont passées aux fonctions. Par exemple, si vous passez une variable contenant le nombre 5 à une fonction et que cette fonction change la valeur à 10, la variable originale restera 5 à l'extérieur de la fonction, car c'est la copie qui a été modifiée, pas l'originale.

Passage par référence : En utilisant le mot-clé **ref**, vous indiquez que vous souhaitez passer la variable par référence. Cela signifie que la fonction travaillera directement sur la variable originale, et non sur une copie. Dans cet cas, si la valeur est modifiée à l'intérieur de la fonction, la variable originale reflétera ce changement à l'extérieur de la fonction. Il est important de noter que la variable passée avec **ref** doit être initialisée avant son utilisation.

À retenir :

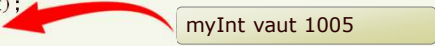
Le passage de variables aux fonctions est une notion fondamentale en programmation. Il

est crucial de comprendre la différence entre le passage par valeur et le passage par référence pour anticiper le comportement du code. Par défaut, les variables sont passées par valeur, ce qui signifie qu'une copie est créée et utilisée dans la fonction. Si vous voulez avoir la capacité de modifier la variable originale à l'intérieur de la fonction, utilisez le mot-clé `ref` pour un passage par référence. Toujours s'assurer que les variables passées par référence soient bien initialisées avant leur utilisation.

Passage de paramètres par référence

```
static void Main(string[] args)
{
    int myInt = 1005;
    ChangeInput(myInt);
}

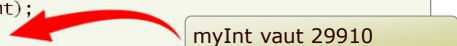
static void ChangeInput(int input)
{
    input = 29910;
}
```



myInt vaut 1005

```
static void Main(string[] args)
{
    int myInt = 1005;
    ChangeInput(ref myInt);
}

static void ChangeInput(ref int input)
{
    input = 29910;
}
```



myInt vaut 29910

Paramètres de sortie

- Les paramètres de sortie permettent de retourner des valeurs par le biais des arguments en entrée par le mot-clé out
- Les paramètres ainsi désignés sont considérés comme des paramètres de sortie
- Il n'est pas obligatoire d'initialiser l'argument passé en paramètre (contrairement à l'utilisation du mot-clé ref)

```
void MyMethod(int first, double second, out int data)
{
    ...
    data = 99;
}
```



```
int value;
MyMethod(10, 101.1F, out value);
// value = 99
```

Variable de sortie

- L'utilisation du mot clé **out** nécessite de déclarer la variable au préalable.
- Une variable de sortie est déclarée lors de l'invocation de la méthode

```
int numericResult;  
if (int.TryParse(input, out numericResult))  
    WriteLine(numericResult);  
else  
    WriteLine("Could not parse input");
```

```
if (int.TryParse(input, out int numericResult))  
    WriteLine(numericResult);  
else  
    WriteLine("Could not parse input");
```



Tuples

- C# permet de développer ses propres types (structures, classes) mais cela peut s'avérer lourd pour une problématique ponctuelle
- Les Tuples permettent de gérer un jeu de valeurs ensemble, en particulier en retour d'une méthode

```
var letters = ("a", "b");  
Console.WriteLine(letters.Item1);  
...  
var alphabetStart = (Alpha: "a", Beta: "b");  
Console.WriteLine(alphabetStart.Alpha);  
...  
(string Alpha, string Beta) namedLetters = ("a", "b");  
Console.WriteLine(namedLetters.Alpha);
```

Tuples (suite)

```
private static (int Max, int Min) Range(int[] numbers)
{
    int min = int.MaxValue;
    int max = int.MinValue;
    foreach(var n in numbers)
    {
        min = (n < min) ? n : min;
        max = (n > max) ? n : max;
    }
    return (max, min);
}
```

```
var range = Range(numbers);
// ou bien
(int max, int min) = Range(numbers)
```

Déconstruction du tuple

Discards

- Utilisation du symbole _
- Permet d'ignorer un membre d'un tuple, ou une variable de sortie

```
private static (string, double, int, int, int, int)
GetCityDetails(string name, int year1, int year2)
{
    /* Récupère nom, superficie, première année, population
    pour année 1, deuxième année, population pour année 2 */
    return (name, area, year1, pop1, year2, pop2);
}
...
var (_, _, _, p1, _, p2)=GetCityDetails("Paris",1960,2010);
```

```
if (int.TryParse(s, out _))
{
    Console.WriteLine("La chaine représente un entier");
}
```

Atelier 01



Activité

20 min

- Exercice 1: Modularisation d'un code existant



- **Modularité** : Les méthodes permettent de découper un programme en blocs logiques, facilitant sa lecture, sa maintenance et sa réutilisabilité.
- **Signature Unique** : Chaque méthode a une signature unique, définie par son nom et la liste des types de ses paramètres, permettant une surcharge efficace.
- **Valeurs de Retour** : Une méthode peut retourner une valeur. Si elle ne retourne rien, elle est définie comme "void" en C#.
- **Paramètres** : Les méthodes peuvent prendre des paramètres qui permettent de transmettre des informations à la méthode lors de son appel.
- **Par Valeur vs Par Référence** : En C#, les paramètres peuvent être passés "par valeur" (où une copie est transmise) ou "par référence" (où l'adresse réelle en mémoire est transmise).
- **Paramètres Par Défaut** : C# permet la définition de valeurs par défaut pour les paramètres, rendant certains arguments optionnels lors de l'appel de la méthode.
- **Réutilisabilité** : En encapsulant des fonctionnalités spécifiques dans des méthodes, on peut facilement réutiliser ces fonctions dans d'autres parties du programme ou dans d'autres programmes.
- **Surcharge de Méthodes** : C# permet la définition de plusieurs méthodes avec le même nom mais des listes de paramètres différentes, offrant une flexibilité dans la façon dont une méthode peut être utilisée.
- **Clarté et Lisibilité** : Nommer correctement les méthodes et utiliser des commentaires appropriés rend le code plus compréhensible pour les autres développeurs.
- **Gestion d'Erreurs** : Grâce au mécanisme d'exceptions de C#, les méthodes peuvent signaler des erreurs d'exécution de manière robuste et contrôlée.

Modularité : Les méthodes permettent de découper un programme en blocs logiques, facilitant sa lecture, sa maintenance et sa réutilisabilité.

Signature Unique : Chaque méthode a une signature unique, définie par son nom et la liste des types de ses paramètres, permettant une surcharge efficace.

Valeurs de Retour : Une méthode peut retourner une valeur. Si elle ne retourne rien, elle est définie comme "void" en C#.

Paramètres : Les méthodes peuvent prendre des paramètres qui permettent de transmettre des informations à la méthode lors de son appel.

Par Valeur vs Par Référence : En C#, les paramètres peuvent être passés "par valeur" (où une copie est transmise) ou "par référence" (où l'adresse réelle en mémoire est transmise).

Paramètres Par Défaut : C# permet la définition de valeurs par défaut pour les paramètres, rendant certains arguments optionnels lors de l'appel de la méthode.

Réutilisabilité : En encapsulant des fonctionnalités spécifiques dans des méthodes, on peut facilement réutiliser ces fonctions dans d'autres parties du programme ou dans d'autres programmes.

Surcharge de Méthodes : C# permet la définition de plusieurs méthodes avec le même nom mais des listes de paramètres différentes, offrant une flexibilité dans la façon dont une méthode peut être utilisée.

Clarté et Lisibilité : Nommer correctement les méthodes et utiliser des commentaires appropriés rend le code plus compréhensible pour les autres développeurs.

Gestion d'Erreurs : Grâce au mécanisme d'exceptions de C#, les méthodes peuvent signaler des erreurs d'exécution de manière robuste et contrôlée.

Chapitre 4 - Gestion d'exceptions



Plan de la formation

- 1 - Introduction à C# et .Net
- 2 - Structure de programmation
- 3 - Déclaration et appel de méthodes
- 4 - Gestion d'exceptions**
- 5 - Lire et écrire dans des fichiers
- 6 - Création de nouveaux types
- 7 - Encapsulation de données
- 8 - Héritage de classes et implémentation
- 9 - Gestion de la durée de vie des objets et contrôle des ressources
- 10 - Encapsulation avancée
- 11 - Découplage de méthodes et gestion d'évènements
- 12 - Utilisation des collections et construction de types génériques
- 13 - Programmation asynchrone et personnalisation du code
- 14 - Utilisation de LINQ pour interroger des données
- 15 - Développement dirigé par les Tests

Objectifs :

- Comprendre l'importance et le rôle de la gestion des exceptions dans le développement logiciel.
- Identifier et gérer les erreurs courantes qui peuvent survenir lors de l'exécution d'un programme.
- Utiliser efficacement les outils fournis par C# pour la gestion des exceptions.
- Écrire du code robuste et résilient qui peut gérer les scénarios d'erreur et reprendre l'exécution.

Objectifs :

À l'issue de ce chapitre, les apprenants seront en mesure de :

1. Comprendre l'importance et le rôle de la gestion des exceptions dans le développement logiciel.
2. Identifier et gérer les erreurs courantes qui peuvent survenir lors de l'exécution d'un programme.
3. Utiliser efficacement les outils fournis par C# pour la gestion des exceptions.
4. Écrire du code robuste et résilient qui peut gérer les scénarios d'erreur et reprendre l'exécution.

Positionnement :

La gestion d'exceptions est une composante essentielle de tout programme robuste. En prévoyant et en gérant correctement les erreurs, les développeurs peuvent garantir que leurs applications restent stables et utiles même en cas de scénarios imprévus. Dans ce chapitre, nous aborderons non seulement les mécanismes techniques de la gestion d'exceptions en C#, mais aussi les meilleures pratiques pour anticiper et répondre efficacement aux erreurs.

Sujets Clés :

- 1. Introduction à la Gestion d'Exceptions :** Comprendre ce qu'est une exception, pourquoi elle se produit et comment elle affecte l'exécution d'un programme.
- 2. Try-Catch-Finally :** Apprendre à utiliser ces blocs pour capturer, gérer et nettoyer après une exception.

3. Lancer des Exceptions : Découvrir comment et pourquoi on peut soulever manuellement une exception dans un programme.

4.Exceptions Prédéfinies et Personnalisées : Explorer les différentes exceptions fournies par le framework .NET et apprendre à créer ses propres exceptions pour des scénarios spécifiques.

5.Meilleures Pratiques : Discuter des stratégies pour décider quand attraper une exception, quand la laisser se propager, et comment informer l'utilisateur ou journaliser l'erreur.

6.Gestion d'Exception Avancée : Aller au-delà des bases pour explorer des scénarios plus complexes, tels que les exceptions filtrées et la gestion d'exceptions dans des scénarios asynchrones.



**Commençons
par échanger**



"Pouvez-vous partager une expérience où une erreur non gérée dans une application vous a causé des problèmes, soit en tant que développeur, soit en tant qu'utilisateur ? Qu'est-ce qui aurait pu être fait différemment ?"

"Comment définiriez-vous une 'exception' en programmation à partir de vos connaissances antérieures ? Et pourquoi pensez-vous que la gestion de ces exceptions est importante ?"

"Avez-vous déjà rencontré une situation où vous auriez souhaité avoir mis en place une gestion d'exceptions mais ne l'avez pas fait ? Quelles ont été les conséquences ?"

"Selon vous, est-il préférable de prévoir toutes les exceptions possibles avant qu'elles ne se produisent ou de réagir aux exceptions lorsqu'elles surviennent ? Pourquoi ?"

"Comment pensez-vous qu'un utilisateur moyen réagit lorsqu'il rencontre une erreur non gérée dans une application ? Quel impact cela pourrait-il avoir sur la perception qu'il a de cette application ?"

Expériences personnelles : "Pouvez-vous partager une expérience où une erreur non gérée dans une application vous a causé des problèmes, soit en tant que développeur, soit en tant qu'utilisateur ? Qu'est-ce qui aurait pu être fait différemment ?"

Compréhension des exceptions : "Comment définiriez-vous une 'exception' en programmation à partir de vos connaissances antérieures ? Et pourquoi pensez-vous que la gestion de ces exceptions est importante ?"

Importance des exceptions : "Avez-vous déjà rencontré une situation où vous auriez souhaité avoir mis en place une gestion d'exceptions mais ne l'avez pas fait ? Quelles ont été les conséquences ?"

Gestion proactive vs réactive : "Selon vous, est-il préférable de prévoir toutes les exceptions possibles avant qu'elles ne se produisent ou de réagir aux exceptions lorsqu'elles surviennent ? Pourquoi ?"

Exceptions et utilisateurs : "Comment pensez-vous qu'un utilisateur moyen réagit lorsqu'il rencontre une erreur non gérée dans une application ? Quel impact cela pourrait-il avoir sur la perception qu'il a de cette application ?"

Vue d'ensemble

- Gestion des exceptions
- Soulever des exceptions

Leçon 1: Gestion des Exceptions

- Définition
- Le bloc try/catch
- Propriétés des exceptions
- Le bloc finally et le catch conditionné
- Les mots clés checked et unchecked

Définition

- Une exception est le résultat d'une erreur d'exécution rencontrée par l'application
- Si l'exception n'est pas gérée par le code l'ayant rencontrée, celle-ci remonte la pile des appels jusqu'à être capturée ou affichée à l'utilisateur
- Il est possible, dans certains Framework de développement, de mettre en place une procédure de gestion d'exceptions non gérées
- Les exceptions sont classées à travers une hiérarchie d'exception de la plus générique (Exception) à la plus spécifique (par exemple FileNotFoundException ou encore DivideByZeroException)

Définitions :

Exception : Une anomalie ou un événement inattendu qui se produit lors de l'exécution d'un programme. C'est le résultat d'une erreur d'exécution rencontrée par l'application.

Gestion des exceptions : Un mécanisme permettant de traiter les exceptions en capturant et en répondant à elles de manière appropriée, plutôt que de permettre à l'application de se terminer de manière imprévue.

Hiérarchie d'exception : Une structure organisée de types d'exceptions, allant de la plus générale à la plus spécifique, pour faciliter leur traitement ciblé.

Explications :

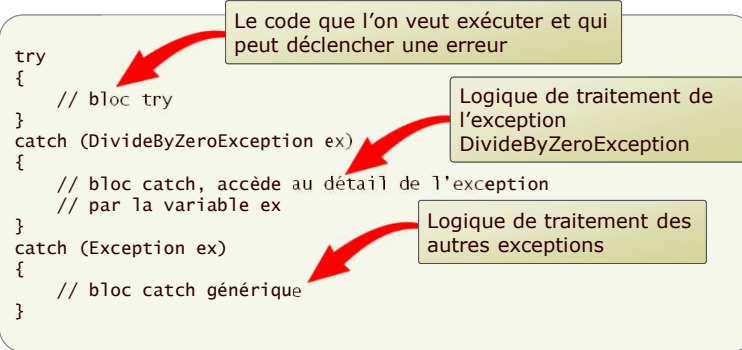
Quand un programme est en cours d'exécution, il peut rencontrer des situations imprévues, comme la tentative de division par zéro ou l'ouverture d'un fichier qui n'existe pas. Ces situations génèrent des exceptions. Si ces exceptions ne sont pas correctement gérées, elles peuvent provoquer un arrêt inattendu de l'application. La plupart des langages de programmation modernes et leurs frameworks associés offrent des mécanismes pour gérer ces exceptions. Cela permet aux développeurs de prévoir et de traiter ces anomalies, assurant ainsi que l'application continue de fonctionner ou qu'elle échoue gracieusement.

Les exceptions peuvent varier en gravité et en type, d'où la nécessité d'une hiérarchie d'exception. Cela permet de gérer des exceptions spécifiques (comme FileNotFoundException) séparément des exceptions plus générales (comme Exception).

À retenir :

La gestion des exceptions est un aspect essentiel de la programmation robuste. Plutôt que de laisser une application échouer silencieusement ou se terminer de manière inattendue, il est préférable de capturer et de gérer les exceptions. La compréhension de la hiérarchie des exceptions aide également à traiter de manière sélective les erreurs spécifiques tout en fournissant une solution de secours pour les exceptions non spécifiées. Assurez-vous toujours de tester minutieusement le traitement des exceptions pour garantir la stabilité de l'application.

Le bloc try/catch



- Il est possible d’imbriquer les blocs try/catch
- Toujours ordonner les blocs catch du plus spécifique au plus générique

Propriétés des exceptions

Quelques propriétés:

Message

Source

StackTrace

TargetSite

InnerException

HelpLink

Data

```
try
{
    // Try block.
}
catch (DivideByZeroException ex)
{
    Console.WriteLine(ex.Message);
}
```

Affichage du
message associé à
cette exception

Le bloc finally et le catch conditionné

- Le bloc finally est toujours exécuté
- Un catch soumis à condition permet d'affiner la gestion d'exception

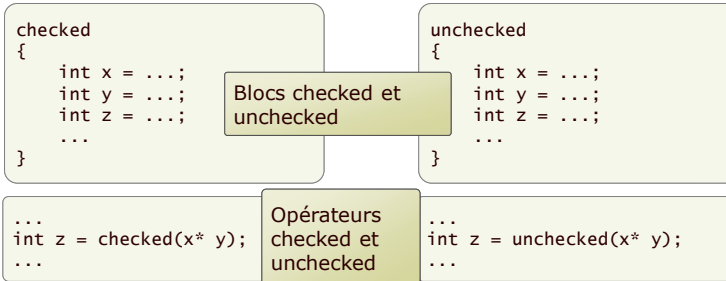
```
try
{
    OpenFile("MyFile");
    WriteToFile(...);
}
catch (IOException ex) when (myCondition)
{
    MessageBox.Show(ex.Message);
}
finally
{
    CloseFile("MyFile"); // Clos le fichier
}
```

Gère l'exception
IOException seulement si
myCondition est vrai

Sera toujours exécuté

Les mots clés checked et unchecked

- Par défaut, les applications C# ne vérifient pas les dépassement de capacités sur les numériques entiers. Vous pouvez changer ce comportement dans les paramètres de votre projet.
- Vous pouvez également contrôler ce comportement localement dans votre code grâce aux mots clés checked et unchecked



Leçon 2: Soulever des Exceptions

- Création d'objets de type Exception
- Lancer une exception
- Bonnes pratiques

Création d'objets de type Exception

System.Exception

System.SystemException

System.FormatException

System.ArgumentException

System.NotSupportedException

+ beaucoup d'autres

```
catch (Exception e)
{
    FormatException ex =
        new FormatException("Argument has the wrong format", e);
}
```

Lancer une exception

Mot clé throw

`throw [exception object];`

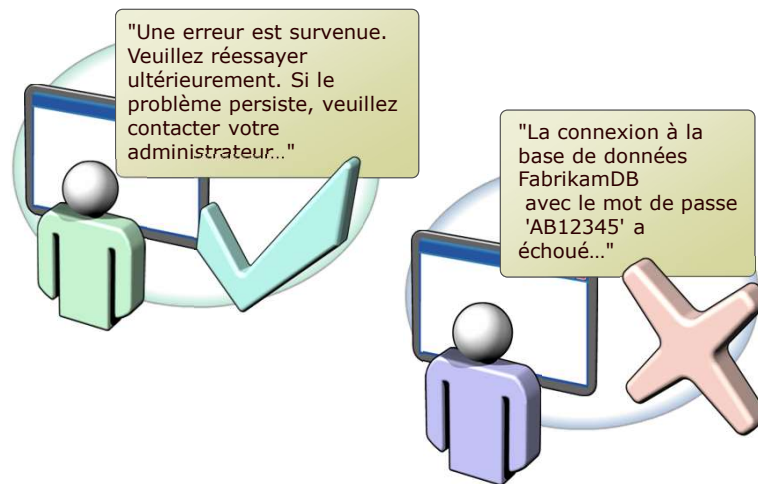
Déclaration de l'objet Exception

Exemple

```
public int GetIntegerRoot(int operand)
{
    double root = Math.Sqrt(operand);
    if (root != (int)root)
    {
        throw new ArgumentException("No integer root found.");
    }
    return (int)root;
}
```

```
name = value ?? throw new ArgumentNullException("Name must not be null");
```

Bonnes pratiques



Atelier 01



Activité

20 min

- Exercice 1: Les dépassements de capacité en C#
- Exercice 2: Levée d'exception
- Exercice 3: Capture d'exception



A retenir

Proactivité : "La gestion des exceptions ne consiste pas simplement à réagir aux erreurs, mais à les anticiper pour garantir une exécution fluide du programme.«

Différenciation : "Il est crucial de différencier une 'erreur' d'une 'exception'. Une erreur est souvent irrécupérable, tandis qu'une exception est une condition que vous pouvez prévoir et gérer.«

Structuration : "Utilisez les blocs try, catch, et finally pour structurer votre code autour de la gestion des exceptions de manière propre et efficace.«

Spécificité : "Il est préférable de gérer les exceptions spécifiques avant les exceptions générales afin d'offrir des retours d'information plus précis et pertinents.«

Personnalisation : "Soulever des exceptions personnalisées peut aider à clarifier l'intention du code et à gérer des situations spécifiques à votre application.«

Messages informatifs : "Lorsque vous soulevez une exception, fournissez toujours un message clair et informatif. Cela facilite le débogage et améliore l'expérience utilisateur.«

Responsabilité : "Ne vous fiez pas uniquement aux exceptions fournies par le framework ; sentez-vous libre de soulever les vôtres pour gérer les cas spécifiques à votre application.«

Évolutivité : "Bien que le code puisse fonctionner sans gestion d'exceptions, une bonne gestion des exceptions rend votre application plus robuste, évolutives, et amicale pour l'utilisateur.«

Documentation : "Documentez toujours les exceptions que votre méthode peut soulever, afin d'informer les autres développeurs et de permettre une meilleure intégration.«

Principe de précaution : "Il vaut mieux prévoir et gérer une exception même si elle semble peu probable. Cela montre que vous avez pris en compte toutes les éventualités, ce qui renforce la robustesse de votre application."

Proactivité : "La gestion des exceptions ne consiste pas simplement à réagir aux erreurs, mais à les anticiper pour garantir une exécution fluide du programme.«

Différenciation : "Il est crucial de différencier une 'erreur' d'une 'exception'. Une erreur est souvent irrécupérable, tandis qu'une exception est une condition que vous pouvez prévoir et gérer.«

Structuration : "Utilisez les blocs try, catch, et finally pour structurer votre code autour de la gestion des exceptions de manière propre et efficace.«

Spécificité : "Il est préférable de gérer les exceptions spécifiques avant les exceptions générales afin d'offrir des retours d'information plus précis et pertinents.«

Personnalisation : "Soulever des exceptions personnalisées peut aider à clarifier l'intention du code et à gérer des situations spécifiques à votre application.«

Messages informatifs : "Lorsque vous soulevez une exception, fournissez toujours un message clair et informatif. Cela facilite le débogage et améliore l'expérience utilisateur.«

Responsabilité : "Ne vous fiez pas uniquement aux exceptions fournies par le framework

; sentez-vous libre de soulever les vôtres pour gérer les cas spécifiques à votre application.»

Évolutivité : "Bien que le code puisse fonctionner sans gestion d'exceptions, une bonne gestion des exceptions rend votre application plus robuste, évolutives, et amicale pour l'utilisateur.»

Documentation : "Documentez toujours les exceptions que votre méthode peut soulever, afin d'informer les autres développeurs et de permettre une meilleure intégration.»

Principe de précaution : "Il vaut mieux prévoir et gérer une exception même si elle semble peu probable. Cela montre que vous avez pris en compte toutes les éventualités, ce qui renforce la robustesse de votre application."

Chapitre 5 - Lire et écrire dans des fichiers



Plan de la formation

- 1 - Introduction à C# et .Net
- 2 - Structure de programmation C#
- 3 - Déclaration et appel de méthodes
- 4 - Gestion d'exceptions
- 5 - Lire et écrire dans des fichiers**
- 6 - Création de nouveaux types de données
- 7 - Encapsulation de données et de méthodes
- 8 - Héritage de classes et implémentation
- 9 - Gestion de la durée de vie des objets
- 10 - Encapsulation avancée
- 11 - Découplage de méthodes et gestion d'événements
- 12 - Utilisation des collections et construction de types génériques
- 13 - Programmation asynchrone et personnalisation du code
- 14 - Utilisation de LINQ pour interroger des données
- 15 - Développement dirigé par les Tests

Objectifs :

- *Comprendre les Fondamentaux : Maîtriser les bases de la lecture et de l'écriture de fichiers en C#.
- *Gérer Efficacement les Fichiers : Apprendre à gérer de manière sécurisée et efficace l'accès aux fichiers et les opérations d'entrée/sortie.
- *Appliquer les Bonnes Pratiques : S'assurer que les fichiers sont traités de manière optimale, tout en gérant les éventuelles erreurs ou exceptions.
- *Manipuler Divers Formats : Se familiariser avec la lecture et l'écriture de différents types de fichiers, des simples fichiers texte aux fichiers binaires ou structurés comme le XML.

Objectifs

1.Comprendre les Fondamentaux : Maîtriser les bases de la lecture et de l'écriture de fichiers en C#.

2.Gérer Efficacement les Fichiers : Apprendre à gérer de manière sécurisée et efficace l'accès aux fichiers et les opérations d'entrée/sortie.

3.Appliquer les Bonnes Pratiques : S'assurer que les fichiers sont traités de manière optimale, tout en gérant les éventuelles erreurs ou exceptions.

4.Manipuler Divers Formats : Se familiariser avec la lecture et l'écriture de différents types de fichiers, des simples fichiers texte aux fichiers binaires ou structurés comme le XML.

Positionnement

Ce chapitre est essentiel pour tout développeur C# souhaitant interagir avec le système de fichiers. La capacité à lire et à écrire des données dans des fichiers est une compétence fondamentale qui trouve son utilité dans de nombreux domaines, que ce soit pour la sauvegarde de données, la génération de rapports, la configuration ou l'échange d'informations. La maîtrise de cette compétence offre une flexibilité et une puissance considérables à un développeur, lui permettant d'étendre l'utilité et la portée de ses applications.

Sujets Clés

1.Fondamentaux de l'Entrée/Sortie : Comprendre comment fonctionne l'entrée/sortie (I/O) en C# et pourquoi elle est importante.

- 2. Manipulation de Chemins** : Utilisation de la classe Path pour travailler avec des chemins de fichiers et des répertoires de manière sécurisée.
- 3. Lecture de Fichiers Texte** : Utilisation des classes StreamReader et File pour lire des fichiers texte.
- 4. Ecriture dans des Fichiers Texte** : Utilisation des classes StreamWriter et File pour écrire dans des fichiers texte.
- 5. Gestion des Fichiers Binaires** : Comment utiliser BinaryReader et BinaryWriter pour lire et écrire des données binaires.
- 6. Travail avec des Fichiers XML** : Introduction à la lecture et à l'écriture de fichiers XML en utilisant XmlReader et XmlWriter.
- 7. Gestion des Exceptions d'Entrée/Sortie** : Comment gérer les exceptions spécifiques à l'entrée/sortie, telles que FileNotFoundException ou IOException.
- 8. Sécurité et Permissions** : Comprendre les permissions nécessaires pour accéder aux fichiers et comment garantir la sécurité lors de l'exécution d'opérations d'entrée/sortie.
- 9. Flux** : Une introduction aux flux (Streams) et à leur importance dans la manipulation de fichiers.
- 10. Fermeture et Libération des Ressources** : L'importance de toujours fermer et libérer les ressources après leur utilisation.

Commençons par échanger



"Pouvez-vous partager une expérience où vous avez dû lire ou écrire des données dans un fichier, que ce soit dans un autre langage ou contexte ? Quels étaient les défis auxquels vous avez été confrontés ?"

"Pourquoi pensez-vous que la capacité de lire et d'écrire dans des fichiers est une compétence si importante pour un développeur ? Dans quelles situations l'avez-vous trouvé utile dans le passé ?"

"Quels types de formats de fichiers avez-vous déjà rencontrés ou utilisés dans vos projets précédents ? Y a-t-il des formats que vous trouvez particulièrement pratiques ou complexes ?"

"Quelle importance accordez-vous à la sécurité lors de la manipulation des fichiers ? Avez-vous des anecdotes ou des préoccupations spécifiques à ce sujet ?"

"Quelle est votre approche lorsque vous rencontrez une erreur ou une exception lors de la lecture ou de l'écriture dans un fichier ? Avez-vous des exemples d'erreurs courantes que vous avez rencontrées et comment les avez-vous résolues ?"

Vue d'ensemble

- Accéder au système de fichier
- Lecture et écriture dans des fichiers en utilisant les flux

Leçon 1: Accéder au système de fichier

- Manipulation des fichiers
- Lecture et écriture de fichiers
- Manipulation des répertoires
- Manipulation des chemins

Manipulation des fichiers

- L'espace de noms System.IO contient de nombreuses classes qui simplifient les interactions avec le système de fichier, notamment les classes File et FileInfo
- La classe File est une classe statique offrant des méthodes simples
- La classe FileInfo est une classe permettant d'obtenir des informations détaillées sur un fichier en particulier

Méthodes de la classe File:

Copy()

Create()

Delete()

Exists()

Méthode de la classe FileInfo:

CopyTo()

Delete()

Length

Open()

Lecture et écriture de fichiers

Lecture en bloc depuis un fichier

```
string filePath = "myFile.txt";  
...  
byte[] data = File.ReadAllBytes(filePath); // données binaires  
string[] lines = File.ReadAllLines(filePath); // lignes  
string data = File.ReadAllText(filePath); // tout le texte
```

Ecriture en bloc vers un fichier

```
string filePath = "myFile.txt";  
...  
string[] fileLines = {"Line 1", "Line 2", "Line 3"};  
File.AppendAllLines(filePath, fileLines); // ajout de lignes  
File.WriteAllLines(filePath, fileLines); // nouveau fichier  
...  
string fileContents = "I am writing this text to a file ...";  
File.AppendAllText(filePath, fileContents); // ajout de texte  
File.WriteAllText(filePath, fileContents); // nouveau fichier
```

Manipulation des répertoires

Les classes Directory et DirectoryInfo de System.IO permettent de gérer les répertoires du système de fichier

Classe Directory

```
string dirPath = @"C:\Users\Student\MyDirectory";  
...  
Directory.CreateDirectory(dirPath);  
Directory.Delete(dirPath);  
string[] dirs = Directory.GetDirectories(dirPath);  
string[] files = Directory.GetFiles(dirPath);
```

Classe DirectoryInfo

```
string dirPath = @"C:\Users\Student\MyDirectory";  
DirectoryInfo dir = new DirectoryInfo(dirPath);  
...  
bool exists = dir.Exists;  
DirectoryInfo[] dirs = dir.GetDirectories();  
FileInfo[] files = dir.GetFiles();  
string fullName = dir.FullName;
```

La classe Path de System.IO permet de créer et gérer les noms d'accès au fichier

La classe Path inclut notamment les méthodes:

GetDirectoryName()

GetExtension()

GetFileName()

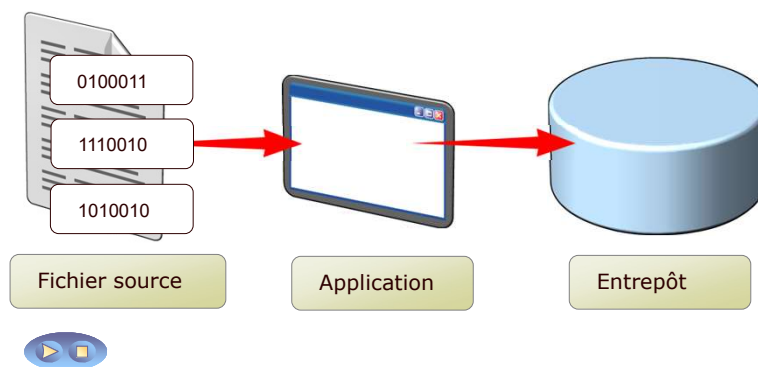
GetFileNameWithoutExtension()

GetRandomFileName()

Leçon 2: Lecture et écriture dans des fichiers en utilisant les flux

- Notion de flux
- Lecture et écriture de données binaires
- Lecture et écriture des types de données primitifs
- Lecture et écriture de texte

Un flux est un mécanisme qui permet de manipuler des données sous forme de morceaux maniables



Lecture et écriture de données binaires

```
FileStream sourceFile = new FileStream(sourceFilePath,...);
BinaryReader reader = new BinaryReader(sourceFile);
int position = 0;
int length = (int)reader.BaseStream.Length;
byte[] dataCollection = new byte[length];
int returnedByte;
while ((returnedByte = reader.Read()) != -1)
{
    dataCollection[position] = (byte)returnedByte;
    position += sizeof(byte);
}
reader.Close();
sourceFile.Close();
```

Classe BinaryReader

```
byte[] dataCollection = { 1, 4, 6, 7, 12, 33, 26, 98, 82, 101 };
FileStream destFile = new FileStream(destinationFilePath,...);
BinaryWriter writer = new BinaryWriter(destFile);
foreach (byte data in dataCollection)
{
    writer.Write(data);
}
writer.Close();
destFile.Close();
```

Classe BinaryWriter

Lecture et écriture des types de données primitifs

```
bool boolValue = reader.ReadBoolean();
byte byteValue = reader.ReadByte();
byte[] byteArrayValue = reader.ReadBytes(4);
char charValue = reader.ReadChar();
...
```

Classe BinaryReader

```
bool boolValue = true;
writer.Write(boolValue);

byte byteValue = 1;
writer.Write(byteValue);

byte[] byteArrayValue = { 1, 4, 6, 8 };
writer.Write(byteArrayValue);

char charValue = 'a';
writer.Write(charValue);
...
```

Classe BinaryWriter

Lecture et écriture de texte

```
FileStream sourceFile = new FileStream(sourceFilePath,...);  
StreamReader reader = new StreamReader(sourceFile);  
StringBuilder fileContents = new StringBuilder();  
while (reader.Peek() != -1)  
{  
    fileContents.Append((char)reader.Read());  
}  
string data = fileContents.ToString();  
reader.Close();  
sourceFile.Close();
```

Classe StreamReader

```
FileStream destFile = new FileStream(destFilePath,...);  
StreamWriter writer = new StreamWriter(destFile);  
  
writer.WriteLine("Hello, this will be written to the file");  
  
writer.Close();  
destFile.Close();
```

Classe StreamWriter

Atelier 01



Activité

20 min

- Exercice 1: Persistance d'informations dans un fichier
- Exercice 2: Persistance du meilleur score
- Exercice 3: Lecture du meilleur score



1. **Accès Essentiel** : Accéder au système de fichiers est une compétence fondamentale pour tout développeur, permettant de lire, écrire et manipuler des données sur le disque.
2. **Flux en .NET** : En C#, les flux (Stream) sont des abstractions essentielles pour traiter les séquences d'octets, que ce soit pour les fichiers, les réseaux ou d'autres sources.
3. **Manipulation Prudente** : Manipuler directement le système de fichiers requiert une attention particulière pour éviter la perte de données, les erreurs d'accès et les problèmes de concurrence.
4. **StreamReader & StreamWriter** : Pour lire et écrire du texte dans des fichiers, les classes StreamReader et StreamWriter sont couramment utilisées et fournissent des méthodes pratiques pour ces opérations.
5. **Gestion des Chemins** : Utilisez les méthodes fournies par la classe Path pour gérer et manipuler les chemins de fichiers de manière sécurisée et compatible.
6. **Fermer les Flux** : Il est crucial de toujours fermer les flux après leur utilisation pour libérer les ressources associées et éviter des comportements indésirables.
7. **Exceptions & Fichiers** : La manipulation des fichiers peut générer diverses exceptions (par exemple, FileNotFoundException). Il est essentiel de les gérer correctement pour assurer la robustesse de votre application.
8. **Accès Concurrent** : Si plusieurs opérations tentent d'accéder au même fichier simultanément, cela peut causer des conflits. Il est important d'être conscient de cela et de gérer l'accès concurrentiel approprié.
9. **Encodage** : Soyez toujours attentif à l'encodage utilisé lors de la lecture ou de l'écriture de texte dans des fichiers, surtout lorsque vous travaillez avec des données internationalisées.
10. **Pratiques de Sécurité** : Lors de l'accès au système de fichiers, gardez toujours à l'esprit les meilleures pratiques de sécurité, en évitant d'exposer des informations sensibles et en utilisant des permissions appropriées.

1.Accès Essentiel : Accéder au système de fichiers est une compétence fondamentale pour tout développeur, permettant de lire, écrire et manipuler des données sur le disque.

2.Flux en .NET : En C#, les flux (Stream) sont des abstractions essentielles pour traiter les séquences d'octets, que ce soit pour les fichiers, les réseaux ou d'autres sources.

3.Manipulation Prudente : Manipuler directement le système de fichiers requiert une attention particulière pour éviter la perte de données, les erreurs d'accès et les problèmes de concurrence.

4.StreamReader & StreamWriter : Pour lire et écrire du texte dans des fichiers, les classes StreamReader et StreamWriter sont couramment utilisées et fournissent des méthodes pratiques pour ces opérations.

5.Gestion des Chemins : Utilisez les méthodes fournies par la classe Path pour gérer et manipuler les chemins de fichiers de manière sécurisée et compatible.

6.Fermer les Flux : Il est crucial de toujours fermer les flux après leur utilisation pour libérer les ressources associées et éviter des comportements indésirables.

7.Exceptions & Fichiers : La manipulation des fichiers peut générer diverses exceptions (par exemple, FileNotFoundException). Il est essentiel de les gérer correctement pour assurer la robustesse de votre application.

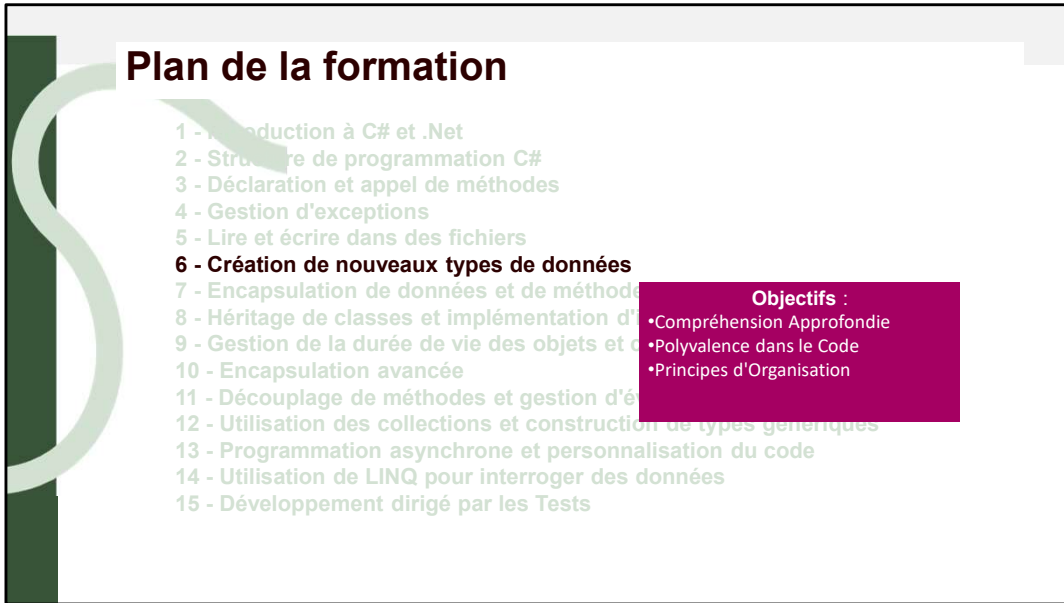
8. Accès Concurrent : Si plusieurs opérations tentent d'accéder au même fichier simultanément, cela peut causer des conflits. Il est important d'être conscient de cela et de gérer l'accès concurrentiel approprié.

9.Encodage : Soyez toujours attentif à l'encodage utilisé lors de la lecture ou de l'écriture de texte dans des fichiers, surtout lorsque vous travaillez avec des données internationalisées.

10.Pratiques de Sécurité : Lors de l'accès au système de fichiers, gardez toujours à l'esprit les meilleures pratiques de sécurité, en évitant d'exposer des informations sensibles et en utilisant des permissions appropriées.

Chapitre 6 - Création de nouveaux types de données





Plan de la formation

- 1 - Introduction à C# et .Net
- 2 - Structure de programmation C#
- 3 - Déclaration et appel de méthodes
- 4 - Gestion d'exceptions
- 5 - Lire et écrire dans des fichiers
- 6 - Création de nouveaux types de données**
- 7 - Encapsulation de données et de méthodes
- 8 - Héritage de classes et implémentation d'interfaces
- 9 - Gestion de la durée de vie des objets et de la mémoire
- 10 - Encapsulation avancée
- 11 - Découplage de méthodes et gestion d'événements
- 12 - Utilisation des collections et construction de types génériques
- 13 - Programmation asynchrone et personnalisation du code
- 14 - Utilisation de LINQ pour interroger des données
- 15 - Développement dirigé par les Tests

Objectifs :

- Compréhension Approfondie
- Polyvalence dans le Code
- Principes d'Organisation

Objectifs :

- 1. Compréhension Approfondie :** Fournir aux apprenants une compréhension solide de la création et de l'utilisation de nouveaux types de données en C# pour étendre et personnaliser la manière dont les données sont stockées et manipulées.
- 2. Polyvalence dans le Code :** Aider les apprenants à devenir plus polyvalents dans leur programmation, leur permettant de définir des types adaptés à des besoins spécifiques.
- 3. Principes d'Organisation :** Inculquer l'importance de l'organisation du code et de la création de structures de données cohérentes pour améliorer la lisibilité et la maintenabilité.

Positionnement :

Ce chapitre occupe une place essentielle dans le parcours d'apprentissage des fondamentaux de C# car il introduit les concepts qui permettent aux développeurs de définir leurs propres structures de données. Alors que les chapitres précédents ont permis aux stagiaires de se familiariser avec les bases de la programmation en C# et l'utilisation des types prédéfinis, ce chapitre les amènera à un niveau supérieur où ils pourront modéliser leurs propres types pour répondre à des besoins spécifiques. Cette capacité est fondamentale pour créer des logiciels robustes et efficaces.

Sujets Clés qui seront couverts :

- 1. Énumérations :** Introduction à la création et à l'utilisation d'énumérations pour définir un ensemble de valeurs nommées.
- 2. Classes :** Fondamentaux de la création, de la définition et de l'instanciation de classes

pour modeler des objets et des comportements associés.

3. Structures : Comprendre la différence entre les classes et les structures, et quand utiliser chacune d'elles.

4.Types Référence vs. Types Valeur : Expliquer la distinction entre les types référence et les types valeur en C#, et leurs implications en matière d'affectation et de gestion de la mémoire.

5.Bibliothèques de Classes sous Visual Studio 2022 : Comment créer, gérer et référencer des bibliothèques de classes pour organiser et réutiliser le code.



Commençons par échanger



"Pouvez-vous me donner un exemple d'une situation où vous auriez besoin de définir un ensemble précis de valeurs possibles pour une variable, comme les jours de la semaine ou les couleurs d'un feu de circulation?"

"Avez-vous déjà travaillé avec des concepts d'orienté objet dans un autre langage de programmation ? Si oui, comment définiriez-vous une 'classe' et un 'objet' d'après votre expérience?"

"Dans quels scénarios pourriez-vous imaginer qu'une structure serait préférable à une classe, étant donné que les deux peuvent être utilisées pour définir des types?"

Types Référence vs. Types Valeur : "Quelle est, d'après vous, la différence principale entre stocker une donnée en tant que référence et la stocker en tant que valeur directe?"

Bibliothèques de Classes : "Avez-vous déjà créé ou utilisé des bibliothèques de fonctions ou de classes dans d'autres projets ou langages de programmation ? Comment avez-vous organisé ou intégré ces bibliothèques ?"

Vue d'ensemble

- Création et utilisation d'énumérations
- Création et utilisation de classes
- Création et utilisation de structures
- Comparaison des types références et types valeurs
- Gestion des références aux bibliothèques de classes sous Visual Studio 2022

Leçon 1: Création et utilisation d'énumérations

- Définition
- Création
- Utilisation

Définition

Une énumération est un ensemble de constantes nommées

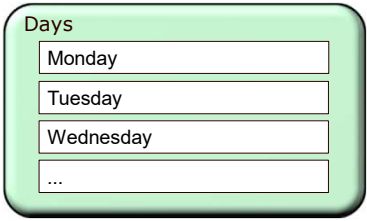
Days

Monday

Tuesday

Wednesday

...



Avantages:

- Le code est plus facile à maintenir car vous affectez des valeurs connues et bien définies aux variables
- Le code est plus facile à lire car vous affectez des valeurs facilement identifiables aux variables
- Le code est plus facile à écrire car l'IntelliSense vous affiche la liste des valeurs utilisables

Définition :

Une énumération est une structure de données spéciale qui permet de définir une collection de constantes nommées, généralement pour faciliter la lecture et l'écriture du code. Dans l'exemple présenté, une énumération "Days" est montrée, contenant les jours de la semaine comme constantes.

Explication :

Les énumérations sont particulièrement utiles pour représenter un ensemble de valeurs associées qui sont connues à l'avance. Par exemple, les jours de la semaine, les mois de l'année ou même des niveaux de priorité (bas, moyen, haut). L'utilisation d'énumérations aide à éviter les erreurs de codage, car elle limite les valeurs possibles à un ensemble défini. Dans certains environnements de développement, comme ceux qui utilisent IntelliSense, lorsqu'un développeur tente d'assigner une valeur à une variable de type énumération, il se voit proposer une liste de constantes valides, ce qui facilite la saisie et réduit les erreurs.

Avantages :

1.Maintenabilité : Les énumérations rendent le code plus robuste en évitant les valeurs incorrectes. En utilisant une énumération, vous êtes sûr que la valeur est toujours l'une des constantes définies.

2.Lisibilité : Utiliser des noms de constantes significatifs améliore la lisibilité du code. Au lieu d'utiliser des chiffres arbitraires, les constantes nommées offrent un contexte clair.

3.Écriture : Les fonctionnalités comme IntelliSense facilitent la saisie en proposant des

suggestions basées sur l'énumération, ce qui accélère le processus de codage et réduit les erreurs.

À retenir :

Les énumérations sont des outils puissants pour améliorer la lisibilité, la maintenabilité et l'exactitude du code. Elles offrent une méthode structurée pour définir et utiliser des ensembles de constantes, garantissant que seules les valeurs valides sont utilisées.

Lorsqu'elles sont utilisées correctement, les énumérations peuvent grandement contribuer à la clarté et à la robustesse d'un programme.

Création

- Une énumération se définit à l'aide du mot clé enum

```
enum Name : DataType { Value1, Value2 . . . };
```

- Il est possible de déclarer des énumérations dans des espaces de noms ou dans des classes, mais pas dans des méthodes
- Sauf indication explicite, les valeurs d'énumérations débutent à 0

```
namespace Fabrikam.CustomEnumerations  
{  
    enum Days : int { Monday = 1, Tuesday = 2, Wednesday = 3 };  
    ...  
}
```

Utilisation

Une variable de type énumération est déclarée comme les autres

```
[EnumType] variableName = [EnumValue]
```

Seules les valeurs définies peuvent être utilisées

```
enum Days : int
{
    Monday = 1, Tuesday = 2, Wednesday = 3, Thursday = 4,
    Friday = 5, Saturday = 6, Sunday = 7
};
static void Main(string[] args)
{
    Days myDayOff = Days.Sunday;
```

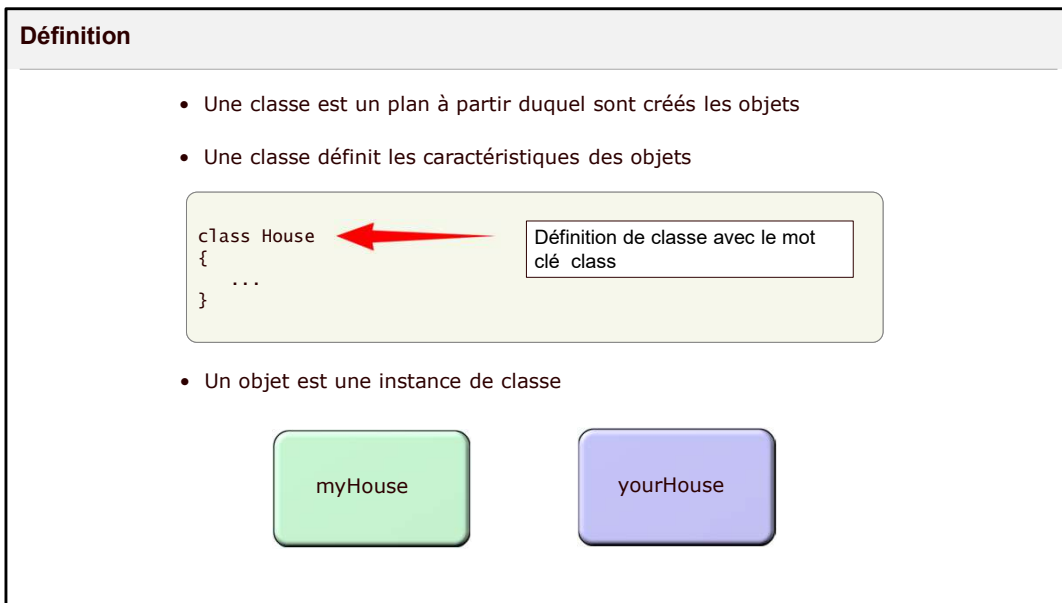
Ou myDayOff = (Days) 7

Certaines opérations simples sont possibles

```
for (Days dayOfWeek = Days.Monday; dayOfWeek <= Days.Sunday;
    dayOfWeek++)
{
    ...
}
```

Leçon 2: Création et utilisation de classes

- Définition
- Ajout de membres
- Constructeurs et initialisation
- Constructeur par défaut
- Création d'objets
- Accès aux membres
- Classes et méthodes partielles

**Définition :**

En programmation orientée objet (POO), une classe est comme un plan ou un modèle à partir duquel des objets sont créés. Elle représente un concept ou une entité et définit les attributs (variables) et les méthodes (fonctions) qui peuvent être utilisés avec cet objet.

Explication :

Pensez à une classe comme à une recette de cuisine. Cette recette décrit les ingrédients et les étapes nécessaires pour préparer un plat. De la même manière, une classe décrit les données qu'elle contiendra et les opérations qui peuvent être effectuées sur ces données.

•**Classe** : Comme illustré, une classe est définie avec le mot-clé "class" suivi du nom de la classe. Dans cet exemple, nous avons une classe nommée "House". Cette classe peut avoir des caractéristiques comme le nombre d'étages, la couleur, la superficie, etc. Elle peut également avoir des méthodes, comme "ouvrirLaPorte" ou "allumerLesLumières".

•**Objet** : Un objet est une occurrence spécifique d'une classe. Si vous créez une instance de la classe "House", vous créez un objet. Dans l'exemple, "myHouse" et "yourHouse" sont deux objets distincts de la classe "House". Chaque objet peut avoir des valeurs différentes pour ses attributs. Par exemple, "myHouse" peut avoir 2 étages tandis que "yourHouse" peut en avoir 3.

Points clés :

1. Classe comme plan : Une classe est un plan à partir duquel de nombreux objets

peuvent être créés. Elle définit la structure et le comportement que ses objets suivront.

2. Instance : Chaque objet est une instance unique de sa classe. Par exemple, bien que "myHouse" et "yourHouse" soient tous deux des instances de la classe "House", ils sont deux objets distincts avec leurs propres attributs.

3.Caractéristiques : Les caractéristiques de l'objet, comme les attributs et les méthodes, sont définies par sa classe.

À retenir :

Les classes et les objets sont des piliers fondamentaux de la programmation orientée objet. Ils permettent d'encapsuler des données et des comportements en une seule entité, rendant le code plus organisé, réutilisable et facile à comprendre.

Ajout de membres

Les membres définissent les données et comportements de la classe

```
public class Residence
{
    public ResidenceType type;
    public int numberOfBedrooms;
    public bool hasGarage;
    public bool hasGarden;

    public int CalculateSalePrice()
    {
        // Code de calcul du prix de
        // la résidence.
    }

    public int CalculateRebuildingCost()
    {
        // Code de calcul du coût de rénovation
        // de la résidence.
    }
}
```

Champs

Méthodes

Constructeurs et initialisation

Un constructeur est une méthode spéciale exécutée implicitement par le CLR

```
public class Residence
{
    public Residence(ResidenceType type, int numberOfBedrooms)
    {
    }
    public Residence(ResidenceType type, int numberOfBedrooms,
        bool hasGarage)
    {
    }
    public Residence(ResidenceType type, int numberOfBedrooms,
        bool hasGarage, bool hasGarden)
    {
    }
}
```

Trois constructeurs

Sans initialisation explicite, les champs seront affectés de la valeur par défaut de leur type

Constructeur par défaut

- Constructeur vide (sans paramètre)
- Généré par le compilateur
- Uniquement créé si aucun constructeur n'est déclaré explicitement
- Assure qu'il soit toujours possible d'instancier les objets

Constructeur par défaut :

Le constructeur par défaut est un type particulier de constructeur en programmation orientée objet (POO). Voici ses caractéristiques et son rôle :

1. Constructeur vide : Il s'agit d'un constructeur qui n'accepte aucun paramètre lors de la création d'un objet. Par conséquent, il est souvent appelé "constructeur sans argument" ou "constructeur par défaut".

2. Généré automatiquement : Dans de nombreux langages de programmation, si vous ne fournissez pas de constructeur pour une classe, le compilateur en génère un pour vous. Ce constructeur généré automatiquement est le constructeur par défaut.

3. Création conditionnelle : Le constructeur par défaut est créé uniquement si aucun autre constructeur n'a été défini explicitement pour la classe. Si vous déclarez un constructeur avec des paramètres (ou même un constructeur sans paramètres), le compilateur ne générera pas automatiquement un constructeur par défaut pour vous.

4. Importance pour l'instanciation : La raison principale de la présence d'un constructeur par défaut est de garantir qu'il sera toujours possible de créer une instance d'une classe. Même si vous n'avez défini aucun constructeur, vous pouvez toujours créer un objet à partir de cette classe grâce au constructeur par défaut.

Exemple :

Prenons l'exemple d'une classe Chien :

```
class Chien {
```

```
        String nom;  
        int age;  
    // Pas de constructeur défini explicitement ici.  
}
```

Dans cet exemple, même si nous n'avons pas défini de constructeur pour la classe Chien, nous pouvons toujours créer une instance de cette classe :

```
Chien monChien = new Chien(); // Utilisation du constructeur par défaut généré  
                                automatiquement.
```

À retenir :

Le constructeur par défaut joue un rôle essentiel dans la POO en garantissant qu'une classe peut toujours être instanciée, même si aucun constructeur n'a été défini explicitement. Cependant, il est toujours bon de se rappeler que si vous définissez votre propre constructeur, vous devrez peut-être aussi définir un constructeur par défaut si vous souhaitez pouvoir créer des objets sans fournir de paramètres.

Création d'objets

- Les objets sont initialement non assignés (*null*)
- Pour utiliser une classe, il faut d'abord créer une instance de la classe
- La création d'une instance implique l'utilisation de l'opérateur new

L'opérateur new permet:

- L'allocation mémoire de l'objet par le CLR
- L'invocation du constructeur pour initialiser l'objet

```
// Appartement avec 2 chambres
Residence myFlat = new Residence(ResidenceType.Flat, 2);

// Maison avec 3 chambres et garage
Residence myHouse = new Residence(ResidenceType.House, 3, true);

// Bungalow avec 2 chambres, garage et jardin
Residence myBungalow =
    new Residence(ResidenceType.Bungalow, 2, true, true);
```

Accès aux membres

L'accès aux membres d'un objet se fait par le nom de l'instance suivi d'un point

InstanceName.MemberName

Exemple

```
// Création d'une maison avec 3 chambres  
Residence myHouse = new Residence(ResidenceType.House, 3);  
  
// La maison a un jardin  
myHouse.hasGarden = true;  
  
// Calcul du prix de vente  
int salePrice = myHouse.CalculateSalePrice();  
  
// Calcul du prix de rénovation  
int rebuildCost = myHouse.CalculateRebuildingCost();
```


Classes et méthodes partielles

- Permet de répartir la définition de la classe sur plusieurs fichiers
- La classe et les méthodes doivent être préfixées du mot clé partial

```
public partial class Residence
{
    partial void SaleResidence(string name);
}
```

Définition de la méthode SaleResidence

```
public partial class Residence
{
    partial void SaleResidence(string name)
    {
        //Logic goes here.
    }
}
```

Implémentation de la méthode SaleResidence

- Compilés en une seule et unique entité
- Généralement mises en oeuvre par des outils de generation de code

Leçon 3: Création et utilisation de structures

- Définition
- Création et utilisation
- Initialisation

Définition

Les structures sont des types valeurs

System.Byte	System.Int64	System.Decimal
System.Int16	System.Single	System.Boolean
System.Int32	System.Double	System.Char

- Les données d'une structure sont stockées sur la pile
- Les structures sont utilisées pour des modèles avec peu de données
- Les structures peuvent contenir des champs et des méthodes

```
int x = 99;
string xAsString = x.ToString();
```

Création et utilisation

- La déclaration d'une structure utilise le mot clé struct
- Les membres sont définis comme dans une classe

```
struct Currency
{
    public string currencyCode;
    public string currencySymbol;
    public int fractionDigits
}
```

- Les structures sont automatiquement allouées en mémoire sur la pile

```
Currency unitedStatesCurrency;

unitedStatesCurrency.currencyCode = "USD";
unitedStatesCurrency.currencySymbol = "$";
unitedStatesCurrency.fractionDigits = 2;
```

- Il n'est pas nécessaire d'utiliser l'opérateur new pour créer une instance de structure

Initialisation

- Il est possible de définir des constructeurs
- Il y a toujours un constructeur vide
- Un constructeur doit initialiser tous les champs

```
struct Currency
{
    public string currencyCode;
    public string currencySymbol;
    public int fractionDigits
    public Currency(string code, string symbol)
    {
        this.currencyCode = code;
        this.currencySymbol = symbol;
        this.fractionDigits = 2;
    }
};
...
Currency unitedKingdomCurrency = new Currency("GBP", "£");
```

Leçon 4: Comparaison des types références et types valeurs

- Comparer types références et types valeurs
- Passage de paramètres par référence
- Boxing et Unboxing
- Types nullable
- Retour de fonctions par référence
- Déconstruction

Comparer types références et types valeurs

Types références (référence sur la pile, données dans le tas)

- Une variable de type référence contient la référence vers un objet en mémoire
- Si deux variables contiennent la même référence, elles pointent vers le même objet
- Si l'objet est modifié, les deux variables permettront de voir cette modification

Types valeurs (valeur sur la pile)

- En copiant un type valeur, les variables ne représentent pas la même donnée
- La modification d'une variable ne sera pas visible par l'autre

Comparer types références et types valeurs :

L'une des distinctions fondamentales dans de nombreux langages de programmation, en particulier les langages orientés objet comme C# ou Java, est celle entre les types référence et les types valeur. Comprendre cette distinction est essentiel pour une gestion efficace de la mémoire et pour éviter des erreurs courantes. Voici une explication détaillée :

Types référence :

Ces types stockent une référence à l'emplacement mémoire de la valeur réelle, plutôt que la valeur elle-même.

•**Stockage** : Une variable de type référence stocke l'adresse mémoire (référence) où l'objet réel est conservé, généralement dans le tas (heap en anglais).

•**Partage de référence** : Si deux variables de type référence pointent vers la même référence, elles pointent vers le même objet en mémoire. Cela signifie que si l'objet est modifié via une variable, les autres variables qui pointent vers la même référence reflèteront également ces modifications.

Exemple :

- `List<int> listeA = new List<int> {1, 2, 3};`
- `List<int> listeB = listeA;`
- `listeB.Add(4);`
- `// listeA et listeB pointent vers la même liste: {1, 2, 3, 4}`

Types valeur :

Contrairement aux types référence, les types valeur contiennent la valeur réelle.

- **Stockage** : Une variable de type valeur stocke la valeur réelle dans la pile (stack en anglais).

- **Copie de valeur** : Lorsqu'une variable de type valeur est affectée à une autre, une copie de la valeur est créée. Les modifications apportées à l'une n'affectent pas l'autre.

- **Exemple** :

- `int a = 10;`

- `int b = a;`

- `b += 5;`

- `// a vaut toujours 10, alors que b vaut 15.`

Pourquoi est-ce important ?

Comprendre la différence entre les types valeur et référence est crucial pour :

1. La gestion de la mémoire : Les types valeur sont stockés dans la pile qui est de taille limitée, tandis que les types référence sont stockés dans le tas, qui est de taille plus grande mais nécessite une gestion manuelle dans certains langages.

2. La prévention d'erreurs : Ne pas comprendre ces différences peut conduire à des erreurs inattendues, en particulier lorsque vous passez des variables à des fonctions ou méthodes ou lorsque vous travaillez avec des structures de données complexes.

3. La performance : La copie de types valeur, en particulier de grandes structures, peut être coûteuse en termes de performances. D'un autre côté, accéder à un objet via une référence peut introduire un niveau d'indirection.

Il est donc essentiel pour tout développeur de bien comprendre ces concepts pour écrire du code efficace et sans erreur.

Passage de paramètres par référence (rappel)

```
static void Main(string[] args)
{
    int myInt = 1005;
    ChangeInput(myInt);
}

static void ChangeInput(int input)
{
    input = 29910;
}
```



myInt vaut 1005

```
static void Main(string[] args)
{
    int myInt = 1005;
    ChangeInput(ref myInt);
}

static void ChangeInput(ref int input)
{
    input = 29910;
}
```



myInt vaut 29910

Boxing et Unboxing

Boxing (type valeur vers type référence)

1. Le CLR alloue la mémoire dans le tas
2. Le CLR copie la valeur de la pile vers le tas

```
Currency myCurrency = new Currency(); // Type valeur  
object o = myCurrency; // Référence sur la valeur
```

Unboxing (type référence vers type valeur)

1. Le CLR vérifie le type de la valeur référencée par la variable
2. Si cela correspond, le CLR copie la valeur depuis le tas

```
Currency myCurrency = new Currency(...);  
object o = myCurrency; // boxing  
...  
Currency anotherCurrency = (Currency)o; // ok à la compilation
```

Types nullable

- Il est possible d'affecter la valeur *null* à une variable de type référence pour indiquer qu'elle n'est pas initialisée
- La valeur *null* est en soi une référence et il n'existe pas de correspondance pour indiquer qu'une variable de type valeur n'est pas initialisée

```
Currency myCurrency = null; // illégal
```

- Le point d'interrogation (?) permet d'indiquer un type valeur nullable

```
Currency? myCurrency = null; // légal  
if (myCurrency == null){...}
```

- Les types nullable exposent les propriétés HasValue et Value

```
Currency? myCurrency = null;  
if (myCurrency.HasValue)  
{  
    Console.WriteLine(myCurrency.Value);  
}
```

Retour de fonctions par référence

```
struct MyBigStruct{  
    public int Id;  
    public string Name;  
}  
  
static MyBigStruct[] _items;  
static ref MyBigStruct FindItem(int id){  
    for (int i = 0; i < _items.Length; i++)  
    {  
        if (_items[i].Id == id)  
        {  
            return ref _items[i];  
        }  
    }  
    throw new Exception("Item not found");  
}  
  
static void Test(){  
    ref MyBigStruct item = ref FindItem(42);  
    item.Name = "test";  
    FindItem(42) = new MyBigStruct();  
}
```

Utile dans des
scénarios
d'optimisation

Déconstruction

- A ne pas confondre avec la notion de Destructeur
- Permet de déconstruire un type (classe ou structure) vers un Tuple

```
public class Point
{
    ...
    public void Deconstruct(out double x, out double y)
    {
        x = this.X;
        y = this.Y;
    }
}
```

```
var p = new Point(3.14, 2.71);
(double X, double Y) = p;
...
(double horizontalDistance, double verticalDistance) = p;
```

Gestion des références aux bibliothèques de classes sous Visual Studio 2022

- Rôle des références
- Ajout d'une référence

Rôle des références

- Permet de spécifier au programme où trouver les définitions des types utilisés
- Peut-être précoce (liaison dès le développement) ou tardive (liaison dynamique lors de l'exécution)
- A ne pas confondre avec la clause *using* qui permet de définir des alias pour simplifier la syntaxe

Rôle des références :

Les références jouent un rôle crucial dans le développement de logiciels, en particulier dans les langages orientés objet et les environnements tels que .NET. Voici une explication détaillée des points mentionnés :

1. Spécification des définitions de types :

Lorsque vous développez un programme, il est fréquent d'utiliser des types définis dans d'une bibliothèque ou d'un autre module. Pour que le programme comprenne ces types, vous devez lui indiquer où trouver leurs définitions. C'est là que les références entrent en jeu.

•Exemple :

Si vous développez une application .NET et que vous souhaitez utiliser une fonctionnalité d'une bibliothèque externe, vous devez ajouter une référence à cette bibliothèque.

2. Liaison précoce vs. Liaison tardive :

La manière dont un programme est lié à une référence peut varier :

•Liaison précoce (liaison statique) : Elle se produit au moment de la compilation. Les types et les méthodes que vous utilisez sont vérifiés et liés à ce moment-là. Cela offre des avantages en termes de performances et garantit que tout est en place avant l'exécution.

•Liaison tardive (liaison dynamique) : Elle se produit au moment de l'exécution. Cela permet une plus grande flexibilité (par exemple, le remplacement de bibliothèques à la

volée), mais comporte des risques tels que l'absence de certaines méthodes ou types attendus au moment de l'exécution.

3. Distinction avec la clause using :

Il est essentiel de ne pas confondre les références avec la clause using (ou équivalent dans d'autres langages).

•**Clause using** : Elle permet de définir des alias pour simplifier la syntaxe. Au lieu de taper le nom complet d'un type à chaque fois, vous pouvez utiliser la clause using pour raccourcir la manière dont vous accédez à ce type.

•**Exemple :**

En C#, au lieu de taper `System.Console.WriteLine()`, vous pouvez ajouter `using System;` en haut de votre fichier et simplement taper `Console.WriteLine()`.

En conclusion, les références jouent un rôle essentiel pour indiquer au programme où trouver les ressources nécessaires. Bien les gérer et comprendre leur fonctionnement peut grandement influencer la qualité et l'efficacité de votre code.

Ajout d'une référence

- Types de références:
 - Assembly (.NET Framework uniquement)
 - Projet
 - Projet partagé
 - COM
- Dans Visual Studio, via l'explorateur de solutions:
 - Clic droit sur le projet puis Ajouter...
 - Clic droit sur le noeud Références (.NET Framework) ou Dépendances (.NET)

Ajout d'une référence :

L'ajout de références est essentiel lors de la création d'applications dans des environnements tels que .NET, car cela permet à votre code d'accéder à des fonctionnalités et des types définis ailleurs. Dans Visual Studio, l'outil de développement le plus couramment utilisé pour .NET, l'ajout de références est une tâche courante.

Types de références :**1.Assembly (.NET Framework uniquement) :**

Il s'agit de bibliothèques compilées qui contiennent du code que vous pouvez utiliser dans votre application. Ces assemblies peuvent être des bibliothèques standard fournies par Microsoft ou des bibliothèques tierces.

2.Projet :

Si vous avez plusieurs projets dans une seule solution Visual Studio, vous pouvez faire référence à l'un d'entre eux dans un autre. Cela est utile pour diviser une grande application en composants gérables.

3.Projet partagé :

Il s'agit d'un type de projet qui permet à plusieurs autres projets de partager le même code. Au lieu de compiler en une assembly, le code est directement intégré dans chaque projet qui y fait référence.

4.COM :

Le modèle d'objet composant (COM) est une interface de programmation d'application

utilisée pour créer des composants logiciels réutilisables. Bien que plus ancien et largement remplacé par .NET, il est toujours utilisé dans certains scénarios, en particulier pour la compatibilité avec d'anciens systèmes.

Ajouter une référence via Visual Studio :

1. Via l'explorateur de solutions :

L'Explorateur de solutions est la fenêtre dans Visual Studio où vous pouvez voir tous vos projets, fichiers et dépendances.

1. Faites un clic droit sur votre projet dans l'Explorateur de solutions.
2. Sélectionnez "Ajouter".
3. Puis, selon ce que vous voulez ajouter, sélectionnez soit "Référence" pour ajouter une assembly, un projet ou COM, soit "Dépendance" pour les packages NuGet ou d'autres dépendances du .NET Core et .NET 5 et versions ultérieures.

En résumé, ajouter des références dans Visual Studio est une étape essentielle pour accéder à d'autres fonctionnalités et types, que ce soit des bibliothèques standard, des bibliothèques tierces, d'autres projets ou même des composants COM. Bien gérer ces références est crucial pour la réussite de votre projet.

Atelier 01



Activité
20 min

- Exercice 1: Création d'une classe
- Exercice 2: Mise en oeuvre d'une bibliothèque de classes



A retenir

1. Énumérations : Les énumérations offrent un moyen efficace de représenter un ensemble fixe de valeurs connexes, augmentant la lisibilité et réduisant les erreurs dans le code.
2. Polyvalence des Classes : Les classes sont au cœur de la programmation orientée objet en C# et permettent de modéliser des objets du monde réel avec des propriétés et des comportements.
3. Structures vs Classes : Contrairement aux classes, les structures sont des types valeur. Elles sont plus légères, idéales pour des objets simples et de petite taille, mais ne supportent pas l'héritage.
4. Choix de Type : Le choix entre type référence (comme les classes) et type valeur (comme les structures) doit être basé sur les besoins de l'application et la manière dont les données sont utilisées.
5. Avantage des Types Valeur : Les types valeur, comme les structures, sont généralement plus rapides à accéder et ne nécessitent pas de gestion de la mémoire via le garbage collector, contrairement aux types référence.
6. Héritage : Seules les classes permettent l'héritage, offrant des mécanismes pour créer de nouveaux types basés sur des types existants.
7. Bibliothèques de Classes : Les bibliothèques de classes facilitent la modularité et la réutilisabilité, permettant de regrouper des types et des fonctionnalités couramment utilisés.
8. Référencement dans Visual Studio : Visual Studio 2022 facilite la gestion des références aux bibliothèques de classes, assurant que les bonnes versions des bibliothèques sont utilisées.
9. Cohérence : L'utilisation cohérente des types, que ce soit des énumérations, des classes ou des structures, améliore la lisibilité du code et réduit les chances d'erreurs.
10. Adaptabilité : Grâce à la richesse des types de données en C#, les développeurs ont la flexibilité de choisir le type le plus approprié pour chaque situation, assurant ainsi une performance et une maintenance optimales du code.

Chapitre 7 - Encapsulation de données et de méthodes



Plan de la formation

- 1 - Introduction à C# et .Net
- 2 - Structure de programmation C#
- 3 - Déclaration et appel de méthodes
- 4 - Gestion d'exceptions
- 5 - Lire et écrire dans des fichiers
- 6 - Création de nouveaux types de données
- 7 - Encapsulation de données et de méthodes**
- 8 - Héritage de classes et implémentation d'interfaces
- 9 - Gestion de la durée de vie des objets
- 10 - Encapsulation avancée
- 11 - Découplage de méthodes et gestion
- 12 - Utilisation des collections et constru
- 13 - Programmation asynchrone et perso
- 14 - Utilisation de LINQ pour interroger d
- 15 - Développement dirigé par les Tests

Objectifs :

- Comprendre le concept d'encapsulation et son importance dans la programmation orientée objet (POO).
- Maîtriser les mécanismes offerts par C# pour implémenter l'encapsulation de manière efficace.
- Savoir comment protéger l'intégrité des données d'une classe tout en fournissant un accès contrôlé à ces données.
- Évaluer l'importance de l'encapsulation pour assurer la modularité, la maintenance, et la sécurité du code.

Objectifs :

1. Comprendre le concept d'encapsulation et son importance dans la programmation orientée objet (POO).
2. Maîtriser les mécanismes offerts par C# pour implémenter l'encapsulation de manière efficace.
3. Savoir comment protéger l'intégrité des données d'une classe tout en fournissant un accès contrôlé à ces données.
4. Évaluer l'importance de l'encapsulation pour assurer la modularité, la maintenance, et la sécurité du code.

Positionnement : L'encapsulation est l'une des trois principales caractéristiques de la POO (avec l'héritage et le polymorphisme). Elle est fondamentale pour garantir que les données internes d'un objet sont protégées des accès non autorisés ou non intentionnels. En contrôlant l'accès aux données et en les regroupant avec les méthodes qui opèrent sur ces données, l'encapsulation favorise une conception plus propre et plus robuste du logiciel.

Sujets clés qui seront couverts :

- 1. Concept d'Encapsulation :** Comprendre ce qu'est l'encapsulation et pourquoi elle est essentielle dans la POO.
- 2. Membres Privés et Publics :** Apprendre comment utiliser les niveaux d'accès pour contrôler la visibilité et l'accès aux membres d'une classe.
- 3. Propriétés en C# :** Découvrir comment les propriétés offrent une méthode flexible

pour accéder, modifier, et valider les données d'un objet.

4. Accesseurs (getters) et Mutateurs (setters) : Comprendre leur rôle dans la fourniture d'un accès contrôlé aux données d'une classe.

5. Avantages de l'Encapsulation : Évaluer comment l'encapsulation peut conduire à un code plus sécurisé, modulable, et maintenable.

6. Méthodes d'Encapsulation : Techniques et meilleures pratiques pour encapsuler efficacement les données et les comportements dans une classe.

7. Auto-implemented Properties : Découvrir les propriétés auto-implémentées en C# et quand les utiliser.



Commençons
par échanger



"Pouvez-vous partager une expérience où vous avez senti le besoin de protéger ou de cacher certaines informations à l'intérieur d'une classe ou d'un objet ? Comment avez-vous géré cela ?"

"Quels sont, selon vous, les principaux avantages de l'utilisation des principes de la programmation orientée objet, en particulier l'encapsulation, dans la conception de logiciels ?"

"Si vous avez déjà travaillé avec d'autres langages orientés objet, comment l'encapsulation y est-elle traitée par rapport à ce que vous savez de C# ?"

"Dans quelles situations pourrait-il être essentiel de limiter l'accès direct aux données d'un objet ? Pouvez-vous donner un exemple concret ?"

"Selon votre compréhension actuelle, pourquoi pensez-vous qu'il est crucial d'associer étroitement les données d'un objet et les méthodes qui opèrent sur ces données ?"

Expériences précédentes : "Pouvez-vous partager une expérience où vous avez senti le besoin de protéger ou de cacher certaines informations à l'intérieur d'une classe ou d'un objet ? Comment avez-vous géré cela ?"

Concepts de la POO : "Quels sont, selon vous, les principaux avantages de l'utilisation des principes de la programmation orientée objet, en particulier l'encapsulation, dans la conception de logiciels ?"

Comparaison avec d'autres langages : "Si vous avez déjà travaillé avec d'autres langages orientés objet, comment l'encapsulation y est-elle traitée par rapport à ce que vous savez de C# ?"

Accès aux données : "Dans quelles situations pourrait-il être essentiel de limiter l'accès direct aux données d'un objet ? Pouvez-vous donner un exemple concret ?"

Compréhension générale : "Selon votre compréhension actuelle, pourquoi pensez-vous qu'il est crucial d'associer étroitement les données d'un objet et les méthodes qui opèrent sur ces données ?"

Vue d'ensemble

- Contrôler la visibilité des membres
- Partager méthodes et données

Leçon 1: Contrôler la visibilité des membres

- Définition de l'encapsulation
- Membres publics et membres privés
- Types publics et types internes

Contrôler la visibilité des membres

Dans le contexte de la programmation orientée objet, l'encapsulation est le concept de restriction de l'accès direct aux composants d'un objet et est souvent considérée comme le regroupement des données (attributs) et des méthodes (fonctions) qui agissent sur les données au sein d'une même unité ou classe. Pour renforcer l'encapsulation, nous devons contrôler la visibilité des membres d'une classe.

Niveaux de visibilité:**1.Public:**

1. Les membres déclarés comme publics sont accessibles de n'importe où, que ce soit à l'intérieur ou à l'extérieur de la classe.
2. Ce sont généralement les méthodes et les propriétés qui doivent être exposées pour que les autres classes ou objets interagissent avec eux.

2.Private:

1. Les membres privés sont uniquement accessibles à l'intérieur de la classe où ils sont définis.
2. Ils sont utilisés pour masquer les détails d'implémentation et protéger l'intégrité des données.

3.Protected:

1. Les membres protégés sont accessibles à l'intérieur de la classe où ils sont définis et également dans ses sous-classes.

2. Ils sont généralement utilisés dans le cadre de l'héritage pour permettre une extension ou une surcharge de comportement.

1.Internal (spécifique à certains langages comme C#):

1. Les membres internes sont accessibles uniquement à l'intérieur de l'assembly où ils sont définis.
2. Ils sont utiles pour exposer des fonctionnalités à d'autres classes du même assembly tout en les cachant à l'extérieur.

2.Protected Internal (également spécifique à certains langages):

1. Une combinaison des niveaux protégé et interne. Le membre est accessible au sein de l'assembly ou à partir de sous-classes.

Bonnes pratiques:

•**Masquer par défaut:**

- Lors de la création d'une nouvelle classe, il est judicieux de rendre les membres privés par défaut et de n'exposer que ce qui est absolument nécessaire.

•**Utiliser des propriétés:**

- Plutôt que d'exposer directement un attribut, utilisez des propriétés pour contrôler l'accès et éventuellement valider les données lors de la définition.

•**Éviter les "setters" publics inutiles:**

- Si un attribut n'a pas besoin d'être modifié en dehors de la classe, il est préférable de ne fournir qu'un "getter" public et de rendre le "setter" privé ou protégé.

En conclusion, le contrôle de la visibilité des membres est essentiel pour maintenir l'intégrité des données, masquer les détails d'implémentation et garantir que les consommateurs de la classe l'utilisent comme prévu. En maîtrisant ces concepts, vous pouvez créer des applications robustes et maintenables.

Définition de l'encapsulation**Définition**

- Principe essentiel de la programmation orientée-objet
- Masque les données et traitements internes
- Assure des opérations publics bien définies

Avantages

- Code externe plus simple et plus consistant
- Evolution interne du code possible sans modification du code externe

Utiliser l'encapsulation pour contrôler la visibilité des membres

L'encapsulation est un pilier fondamental de la programmation orientée objet (POO) et se réfère à la capacité de masquer les détails internes d'une classe, tout en exposant une interface claire et sécurisée à l'utilisateur. Cette interface est généralement constituée de méthodes publiques et de propriétés.

Contrôler la visibilité à l'aide de modificateurs d'accès:**1.Modificateur private:**

1. Un membre déclaré comme private est accessible uniquement au sein de la classe où il est défini.
2. Utilisé pour cacher des détails d'implémentation ou des données sensibles de la classe.

2.Modificateur public:

1. Un membre déclaré comme public est accessible de partout, que ce soit à l'intérieur ou à l'extérieur de la classe.
2. Idéal pour exposer les méthodes et propriétés essentielles pour l'interaction avec la classe.

3.Modificateur protected:

1. Un membre déclaré comme protected est accessible à la fois dans la classe où il est défini et dans ses sous-classes.
2. Utile pour les scénarios d'héritage où une sous-classe doit accéder à

certaines membres de la classe parente.

4. Modificateur internal (spécifique à certains langages):

1. Accessible uniquement à l'intérieur de l'assembly courant.
2. Utilisé pour définir une portée au niveau de l'assembly.

5. Modificateur protected internal:

1. Combinaison des niveaux protected et internal. Accessible soit par les classes du même assembly, soit par les sous-classes.

Exemples pratiques:

• **Propriétés:**

- Au lieu d'exposer directement une variable d'instance (par exemple, un attribut), utilisez des propriétés pour fournir un accès contrôlé. Cela vous permet également d'ajouter une logique supplémentaire, comme la validation, si nécessaire.

• **Méthodes d'assistance privées:**

- Une classe peut avoir des méthodes qui sont utilisées en interne pour effectuer diverses tâches, mais qui ne devraient pas être exposées à l'utilisateur de la classe. Ces méthodes peuvent être rendues privées.

Importance de l'encapsulation:

• **Sécurité:** L'encapsulation protège l'intégrité des données en empêchant l'accès direct aux variables d'instance.

• **Flexibilité et maintenance:** Avec l'encapsulation, les détails internes peuvent être modifiés sans affecter les parties du code qui utilisent la classe.

• **Clarté:** Une interface clairement définie rend la classe plus facile à comprendre et à utiliser.

En contrôlant la visibilité des membres d'une classe, vous garanteez non seulement la sécurité des données, mais aussi une meilleure structuration et modularité du code, tout en facilitant la maintenance et l'évolution du programme à long terme.

Membres publics et membres privés

private Niveau d'accès le plus strict

```
class Sales
{
  private double monthlyProfit;
  private void SetMonthlyProfit(double monthlyProfit)
  {
    this.monthlyProfit = monthlyProfit;
  }
}
```

Uniquement
accessibles
dans la classe
Sales

public Niveau d'accès le moins strict

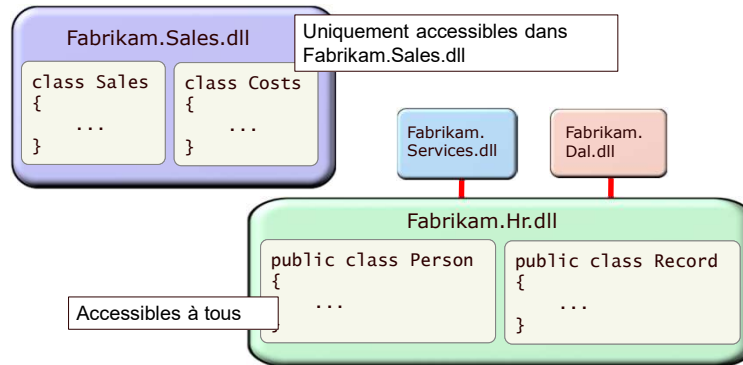
```
class Sales
{
  private double monthlyProfit;
  public void SetMonthlyProfit(double monthlyProfit)
  {
    this.monthlyProfit = monthlyProfit;
  }
}
```

Accessible
depuis
n'importe quel
type

Types publics et types internes

Par défaut les types sont Internes

- Internal: n'est accessible qu'aux autres types du même assembly
- Public: est accessible par des types d'autres assemblys



Leçon 2: Partager méthodes et données

- Création et utilisation de champs statiques
- Création et utilisation de méthodes statiques
- Création de types statiques et constructeur statique
- Création et utilisation de méthodes d'extension
- Import statique

Partager méthodes et données

Dans le cadre de la programmation orientée objet (POO), le partage de méthodes et de données entre classes et objets est un concept clé pour favoriser la réutilisation du code et pour établir des relations entre différents types d'objets. Voici une exploration de plusieurs façons de partager des méthodes et des données :

1. Héritage:

- Le mécanisme le plus courant pour partager du code en POO est l'héritage, qui permet à une classe (la sous-classe) d'hériter des caractéristiques d'une autre classe (la classe de base ou la superclasse).
- La sous-classe peut réutiliser (ou "hériter") des attributs et méthodes de la superclasse, et elle peut également introduire de nouveaux attributs et méthodes ou surcharger (réécrire) ceux de la superclasse.

Exemple :

```
public class Vehicule
{
    public void Demarrer()
    {
        Console.WriteLine("Le véhicule démarre");
    }
}
```



```
public class Voiture : Vehicule
{
    public void Klaxonner()
    {
        Console.WriteLine("La voiture klaxonne");
    }
}
```

Dans cet exemple, la classe Voiture hérite de la classe Vehicule et a donc accès à la méthode Demarrer() ainsi qu'à sa propre méthode Klaxonner().

2. Composition:

- Au lieu d'hériter de la superclasse, une classe peut inclure une instance d'une autre classe. C'est ce qu'on appelle la composition.
- La composition est souvent considérée comme une alternative flexible à l'héritage car elle permet à une classe de réutiliser du code sans être liée structurellement à la classe dont elle réutilise le code.

Exemple :

```
public class Moteur
{
    public void Demarrer()
    {
        Console.WriteLine("Le moteur démarre");
    }
}

public class Voiture
{
    private Moteur moteur = new Moteur();

    public void Demarrer()
    {
        moteur.Demarrer();
    }
}
```

Dans cet exemple, au lieu d'hériter de la fonctionnalité de démarrage, la classe Voiture utilise la composition pour inclure un Moteur et démarrer le moteur via la méthode Demarrer().

3. Aggrégation:

- Semblable à la composition, mais avec une association plus faible.
- Par exemple, une classe Université peut avoir plusieurs objets Etudiant, mais les étudiants peuvent exister indépendamment de l'université.

4. Static:

- Les classes peuvent avoir des méthodes et des variables static, ce qui signifie qu'elles appartiennent à la classe elle-même plutôt qu'à une instance de la classe.
- Elles sont partagées entre toutes les instances de la classe.

Exemple :

```
public class Calculateur
```

```
    {  
    public static int Additionner(int a, int b)  
    {  
        return a + b;  
    }  
    }
```

Dans cet exemple, Additionner est une méthode static et peut être appelée sans créer une instance de la classe Calculateur.

En conclusion, partager des méthodes et des données est essentiel pour créer un code modulaire et réutilisable en POO. La méthode de partage que vous choisissez (héritage, composition, aggrégation, etc.) dépendra des besoins spécifiques de votre conception et de votre application.

Création et utilisation de champs statiques

- Les champs d'instance contiennent des données spécifiques à une instance d'un type donné
- Les champs statiques contiennent des données spécifiques à un type

```
class Sales
{
    public static double salesTaxPercentage = 20;
}
```

- Un champ statique est accédé en le préfixant du nom du type dans lequel il est défini

```
Sales.salesTaxPercentage = 32;
```

Création et utilisation de méthodes statiques

- Les méthodes statiques offrent des fonctionnalités pertinentes pour un type et non pour une instance en particulier

- Les méthodes statiques ne peuvent accéder qu'aux champs statiques de la classe

```
class Sales
{
    public static double GetMonthlySalesTax(double monthProfit)
    {
        ...
    }
}
```

- Il n'est pas nécessaire de créer une instance pour pouvoir utiliser des méthodes statiques

```
double monthlySalesTax = Sales.GetMonthlySalesTax(34267);
```

Création de types statiques et constructeur statique

- Un type statique est déclaré en utilisant le modificateur static

- Les types statiques ne peuvent contenir que des membres statiques (sorte de boîte à outils, ex. System.Math)

```
static class Sales
{
}
```

- Les types statiques peuvent avoir un constructeur

- Les constructeurs statiques sont invoqués implicitement par le CLR

```
static class Sales
{
    static Sales()
    {
    }
}
```

Défini avec le modificateur static

Défini sans paramètres

Défini sans modificateur d'accès

Création et utilisation de méthodes d'extension

Ajouter des méthodes à une classe dans les cas suivants:

- Aucun accès au code source de la classe
- Impossible de recompiler le code de la classe
- Impossible d'hériter de la classe

```
namespace Fabrikam.Extens
{
    static class IntExtension
    {
        internal static int NextRand(this int seed, int
            maxValue)
        {
            Random randomNumberGenerator = new Random(seed);
            return randomNumberGenerator.Next(maxValue);
        }
    }
}
```

Classe statique

Méthode statique

Type à étendre

```
using Fabrikam.Extensions;
...
int i = 8;
int j = i.NextRand(20);
```

Aucune référence à la classe IntExtension

Import statique

En plus de pouvoir définir des alias d'espace de noms, la clause *using* peut également permettre de se passer de préfixer les méthodes statiques

```
// Définition d'alias
using MyAlias = Society.Package.Tools;
...
MyAlias.MyType t = new MyAlias.MyType();
```

```
// Import statique
using static System.Math;
...
double value = Sqrt(data);
```

Atelier 01



Activité
20 min

- Exercice 1: Encapsulation de données
- Exercice 2: Les membres statiques
- Exercice 3: Méthodes d'extension



1. **Définition fondamentale** : L'encapsulation est un pilier de la programmation orientée objet, permettant de regrouper les données d'un objet et les méthodes qui opèrent sur ces données.
2. **Contrôle d'accès** : En C#, les modificateurs d'accès (comme private, public, protected et internal) déterminent comment les membres d'une classe peuvent être accessibles depuis d'autres classes ou assemblages.
3. **Protection des données** : L'encapsulation aide à protéger l'intégrité des données en empêchant un accès direct ou non autorisé à certaines parties d'un objet.
4. **Simplicité et flexibilité** : En cachant les détails d'implémentation, l'encapsulation rend le code plus lisible et facilite la maintenance.
5. **Principe de moindre privilège** : Par défaut, il est recommandé de rendre les membres private sauf si une bonne raison justifie une autre visibilité.
6. **Réutilisabilité** : Grâce à l'encapsulation, les classes peuvent être plus facilement réutilisées dans d'autres contextes sans craindre que leur fonctionnement interne ne soit perturbé.
7. **Interdépendance** : En partageant méthodes et données à l'intérieur d'une classe, nous pouvons garantir que toutes les opérations sur ces données sont cohérentes et sécurisées.
8. **Évolution du code** : L'encapsulation permet d'apporter des modifications à l'implémentation d'une classe sans affecter les parties du code qui utilisent cette classe.
9. **Conception robuste** : Une bonne encapsulation contribue à créer des systèmes robustes et évolutifs en isolant les différentes parties du code les unes des autres.
10. **Cohésion et couplage** : L'encapsulation favorise une forte cohésion à l'intérieur des classes et un faible couplage entre elles, ce qui est une pratique souhaitable en conception logicielle.

1.Définition fondamentale : L'encapsulation est un pilier de la programmation orientée objet, permettant de regrouper les données d'un objet et les méthodes qui opèrent sur ces données.

2.Contrôle d'accès : En C#, les modificateurs d'accès (comme private, public, protected et internal) déterminent comment les membres d'une classe peuvent être accessibles depuis d'autres classes ou assemblages.

3.Protection des données : L'encapsulation aide à protéger l'intégrité des données en empêchant un accès direct ou non autorisé à certaines parties d'un objet.

4.Simplicité et flexibilité : En cachant les détails d'implémentation, l'encapsulation rend le code plus lisible et facilite la maintenance.

5.Principe de moindre privilège : Par défaut, il est recommandé de rendre les membres private sauf si une bonne raison justifie une autre visibilité.

6.Réutilisabilité : Grâce à l'encapsulation, les classes peuvent être plus facilement réutilisées dans d'autres contextes sans craindre que leur fonctionnement interne ne soit perturbé.

7.Interdépendance : En partageant méthodes et données à l'intérieur d'une classe, nous pouvons garantir que toutes les opérations sur ces données sont cohérentes et sécurisées.

8.Évolution du code : L'encapsulation permet d'apporter des modifications à l'implémentation d'une classe sans affecter les parties du code qui utilisent cette classe.

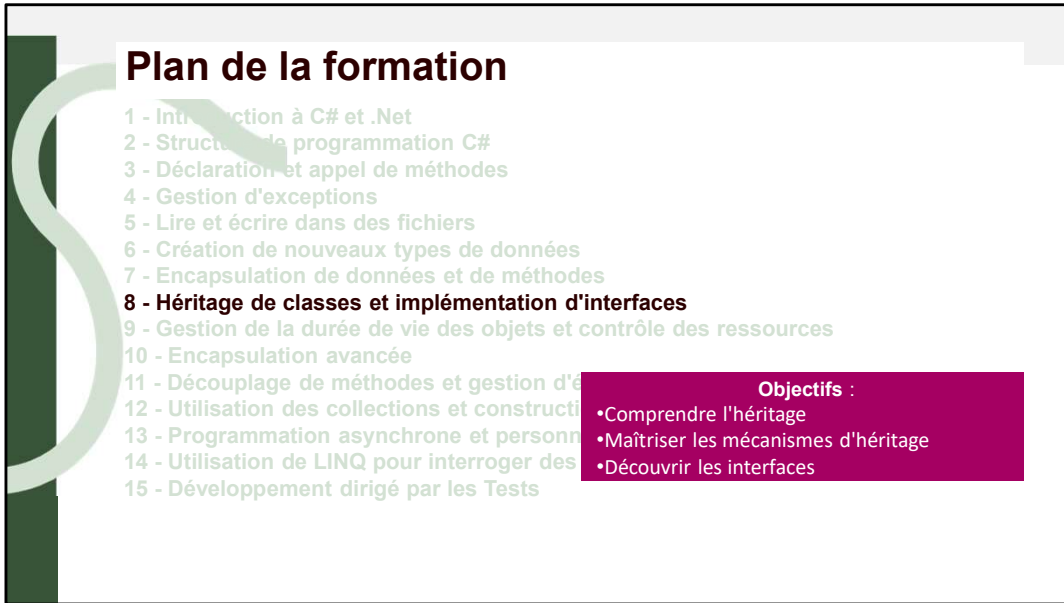
9.Conception robuste : Une bonne encapsulation contribue à créer des systèmes

robustes et évolutifs en isolant les différentes parties du code les unes des autres.

10. Cohésion et couplage : L'encapsulation favorise une forte cohésion à l'intérieur des classes et un faible couplage entre elles, ce qui est une pratique souhaitable en conception logicielle.

Chapitre 8 - Héritage de classes et implémentation d'interfaces





Plan de la formation

- 1 - Introduction à C# et .Net
- 2 - Structure de programmation C#
- 3 - Déclaration et appel de méthodes
- 4 - Gestion d'exceptions
- 5 - Lire et écrire dans des fichiers
- 6 - Création de nouveaux types de données
- 7 - Encapsulation de données et de méthodes
- 8 - Héritage de classes et implémentation d'interfaces**
- 9 - Gestion de la durée de vie des objets et contrôle des ressources
- 10 - Encapsulation avancée
- 11 - Découplage de méthodes et gestion d'événements
- 12 - Utilisation des collections et constructions
- 13 - Programmation asynchrone et personnalisation
- 14 - Utilisation de LINQ pour interroger des données
- 15 - Développement dirigé par les Tests

Objectifs :

- Comprendre l'héritage
- Maîtriser les mécanismes d'héritage
- Découvrir les interfaces

Objectifs :

1. Comprendre l'héritage : Fournir aux apprenants une solide compréhension de l'héritage en programmation orientée objet (POO) et son importance dans la création d'applications évolutives et maintenables.

2. Maîtriser les mécanismes d'héritage : Permettre aux participants d'utiliser l'héritage pour créer des classes dérivées en héritant des propriétés et des méthodes d'une classe de base.

3. Découvrir les interfaces : Introduire le concept d'interface, son rôle et son importance dans la POO, et montrer comment l'utiliser pour ajouter de la flexibilité au code.

4. Concevoir des solutions POO robustes : Aider les apprenants à comprendre comment l'héritage et les interfaces peuvent être utilisés ensemble pour créer des solutions solides et polyvalentes.

Positionnement : L'héritage et les interfaces sont des piliers fondamentaux de la programmation orientée objet. Ils permettent une abstraction, une modularité et une polyvalence accrues, tout en offrant une structure claire et maintenable. Ce chapitre sert à ancrer ces concepts essentiels, après avoir abordé les bases de la création de classes et de l'encapsulation, pour permettre aux apprenants de créer des architectures logicielles plus sophistiquées et flexibles.

Sujets clés qui seront couverts :

1. Principes de base de l'héritage : Introduction à l'héritage, à la classe de base, à la classe dérivée et aux relations "est un".

- 2. Surcharge et substitution** : Comment redéfinir ou étendre les méthodes et les propriétés héritées.
- 3.Modificateurs d'accès et héritage** : Comprendre les niveaux d'accès protected et private en relation avec l'héritage.
- 4.Polymorphisme** : Utiliser l'héritage pour permettre à plusieurs classes dérivées d'être traitées comme des instances d'une classe de base.
- 5.Introduction aux interfaces** : Définition, avantages et différences par rapport à l'héritage.
- 6.Implémentation d'interfaces** : Comment définir et mettre en œuvre une interface dans C#.
- 7.Interfaces vs Classes abstraites** : Comprendre les différences et quand utiliser l'une plutôt que l'autre.
- 8.Composition** : Comment et pourquoi combiner l'héritage et la composition pour obtenir une conception plus flexible et maintenable.



Commençons par échanger

Quelle est votre expérience avec la notion d'héritage en programmation?

Pouvez-vous citer un exemple, dans la vie réelle ou dans la programmation, où la relation "est un" est évidente ?

Avez-vous déjà utilisé des interfaces dans votre code? Si oui, dans quel contexte?

Quels avantages voyez-vous à définir des comportements à l'aide d'interfaces plutôt que d'hériter directement d'une classe?

Pouvez-vous décrire une situation où le polymorphisme pourrait être utile dans une application ?

1. Quelle est votre expérience avec la notion d'héritage en programmation? Cette question permet de sonder la familiarité des apprenants avec l'héritage et de voir s'ils ont déjà une base sur laquelle construire.

2. Pouvez-vous citer un exemple, dans la vie réelle ou dans la programmation, où la relation "est un" est évidente ? Cela peut aider les apprenants à établir un lien entre la notion d'héritage et des situations concrètes qu'ils ont déjà rencontrées.

3. Avez-vous déjà utilisé des interfaces dans votre code? Si oui, dans quel contexte? C'est une excellente manière de voir qui a une expérience pratique avec les interfaces et dans quelles situations elles ont été utilisées.

4. Quels avantages voyez-vous à définir des comportements à l'aide d'interfaces plutôt que d'hériter directement d'une classe? Cette question stimule la réflexion sur les différences et les avantages potentiels des interfaces par rapport à l'héritage.

5. Pouvez-vous décrire une situation où le polymorphisme pourrait être utile dans une application ? En demandant aux apprenants de réfléchir à l'application du polymorphisme, cela les aide à consolider leur compréhension de ce concept clé et à voir son potentiel pratique.

Vue d'ensemble

- Utiliser l'héritage pour définir de nouveaux types références
- Définir et implémenter des interfaces
- Définir des classes abstraites

Leçon 1: Utiliser l'héritage pour définir de nouveaux types références

- Définition
- Hiérarchie d'héritage dans le .NET Framework
- Transtypage et héritage
- Polymorphisme et masquage
- Comprendre le polymorphisme
- Appel de méthode et de constructeur de la classe de base
- Classes et méthodes scellées
- Récapitulatif des niveaux d'accessibilité

Définition

L'héritage permet de créer de nouveaux types à partir de types existants:

- Par exemple, les classes Manager et ManualWorker peuvent hériter de la classe Employee
- Les champs et les méthodes de la classe Employee sont hérités par Manager et ManualWorker
- Manager et ManualWorker peuvent définir leur propre champs et comportements
- protected: public pour les classes héritantes, privées pour les autres

```
// Classe de base
class Employee
{
    public string empNum;
    public string empName;
    public void DoWork() { ... }
}

// Classes héritantes
class Manager : Employee
{
    public void DoManagementWork()
    { ... }
}

class ManualWorker : Employee
{
    public void DoManualWork()
    { ... }
}
```

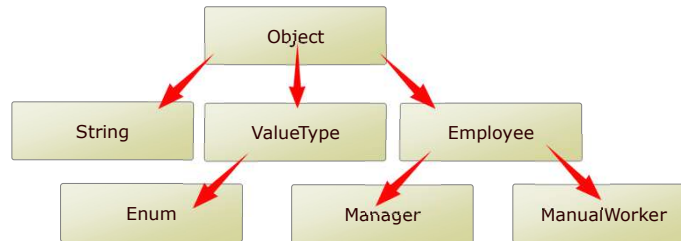


C# ne permet d'hériter que d'une seule classe

Hiérarchie d'héritage dans le .NET Framework

Tous les types héritent directement ou non de la classe System.Object

- Inutile de spécifier :Object dans la définition
- Structures et Enumérations héritent de ValueType



Il n'est pas possible de définir sa propre hiérarchie d'héritage par rapport aux types valeurs



Transtypage et héritage

Le typage fort de C# vous
empêche de référencer un type
dans une variable d'un autre type
...



```
Manager myManager = new
Manager(...);
ManualWorker myWorker =
myManager;
```

... mais vous pouvez affecter une
référence à un type différent plus
haut dans la hiérarchie d'héritage



```
Manager myManager = new
Manager(...);
Employee myEmployee =
myManager;
```

Utiliser les opérateurs is et as
pour assurer la sécurité des
conversions inverses



```
Manager myManagerAgain =
myEmployee as Manager;
// OU
if(myEmployee is Manager)
    Manager myManagerAgain =
    (Manager) myEmployee;
```



Polymorphisme et masquage



Polymorphisme: Remplacer ou étendre des fonctionnalités de la classe de base avec un comportement sémantiquement équivalent (intentionnel)

```
class Employee
{
    protected void DoWork()
    { ... }
}

class Manager : Employee
{
    public new void DoWork()
    { ... }
}
```

```
class Employee
{
    protected virtual void
    DoWork()
    { ... }
}

class Manager : Employee
{
    protected override void
    DoWork()
    { ... }
}
```



Masquage: Remplacer des fonctionnalités de la classe de base par un nouveau comportement (potentiellement une erreur)



Comprendre le polymorphisme

Dans une hiérarchie d'héritage, la même instruction peut appeler différentes implémentations de la même méthode

```
Employee myEmployee;
Manager myManager =
    new Manager(...);
ManualWorker myWorker =
    new ManualWorker(...);

myEmployee = myManager;
Console.WriteLine(
    myEmployee.GetTypeName());

myEmployee = myWorker;
Console.WriteLine(
    myEmployee.GetTypeName());
```

```
class Employee
{
    public virtual string
    GetTypeName()
    {
        return "This is an
        Employee";
    }
}

class Manager : Employee
{
    public override string
    GetTypeName()
    {
        return "This is a Manager";
    }
}

class ManualWorker : Employee
{
    // Pas de polymorphisme
}
```

Appel de méthode et de constructeur de la classe de base

Utilisation du mot clé base

```
class Employee
{
    protected string empName;
    public Employee(string name)
    { this.empName = name; }
}

class Manager : Employee
{
    protected string empGrade;
    public Manager(string name,
                    string grade)
    {
        : base(name)
        {
            this.empGrade = grade;
        }
    }
}
```

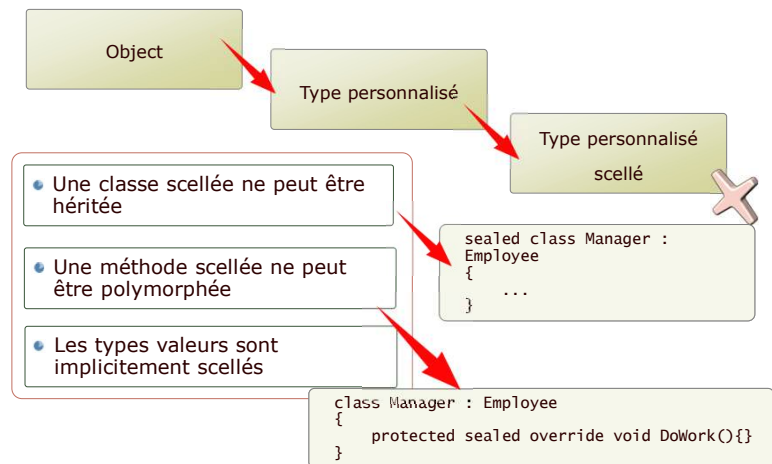
```
class Employee
{
    protected virtual void
    DoWork()
    { ... }
}

class Manager : Employee
{
    protected override void
    DoWork()
    {
        ...
        base.DoWork();
    }
}
```

Les constructeurs invoquent
automatiquement le constructeur
par défaut de la classe mère sauf
indication explicite



Classes et méthodes scellées



Récapitulatif des niveaux d'accessibilité

- Une classe peut être
 - public
 - internal (par défaut)

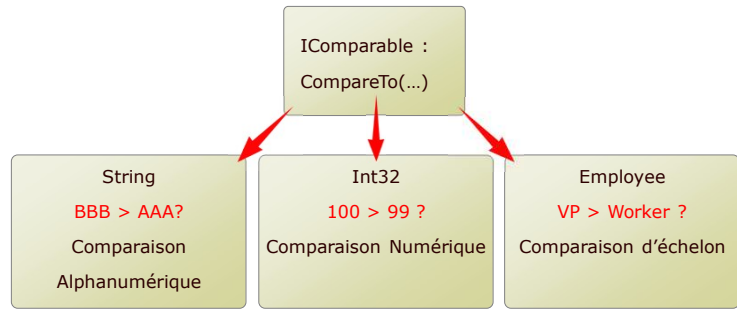
- Les membres d'une classe peuvent être
 - public (partout)
 - protected (par héritage)
 - internal (même assembly)
 - private (par défaut – uniquement dans la classe)
 - protected internal (même assembly ou par héritage)
 - private protected (par héritage d'une classe du même assembly)

Leçon 2: Définir et implémenter des interfaces

- Définition
- Création et implémentation d'interface
- Transtypage et interface
- Implémentation explicite et implicite
- Méthodes d'interface par défaut

Définition

- Un contrat qui spécifie les méthodes que doit exposer la classe qui l'implémente (impose un certain comportement)
- Indépendante de l'implémentation
- Facilite l'injection de dépendance / inversion de contrôle



Création et implémentation d'interface

Utilisation du mot clé interface

Modificateur d'accès facultatif
(si défini, plus délicat à gérer)

```
interface ICalculator
{
    double Add();
    double Subtract();
    double Multiply();
    double Divide();
}
```

```
class Calculator : ICalculator
{
    public double Add(){}
    public double Subtract(){}
    public double Multiply(){}
    public double Divide(){}
}
```

Même syntaxe que l'héritage

Attention à l'accessibilité des
méthodes



C# autorise l'implémentation de plusieurs interfaces



Transtypage et interface

Vous pouvez définir une classe
qui implémente l'interface ...

```
class Calculator : ICalculator
{
    ...
}
```

... et utiliser l'interface pour
référer une instance de cette
classe

```
Calculator myCalculator =
    new Calculator();
ICalculator iMyCalculator =
    myCalculator;
```

You pouvez également utiliser les
interfaces comme paramètres ...

```
public int PerformAnalysis(ICalculator c)
{
    ...
}
```

... , vérifier qu'une instance
implémente une interface, ou
encore qu'une variable de type
interface contient le bon objet

```
Calculator calc =
    iMyCalculator as Calculator;
bool isCalc =
    iMyCalculator is Calculator;
```

```
ICalculator iCalc =
    myCalculator as ICalculator;
```



Implémentation explicite et implicite

Implémentation implicite:

- Scénarios simples
- Peut engendrer des ambiguïtés
- Méthodes visibles pour l'objet en tant qu'instance de classe

```
class Calculator : ICalculator
{
    double Add()
    {
    }

    double Subtract()
    {
    }

    ...
}
```

Implémentation explicite:

- Scénarios complexes
- Evite les ambiguïtés
- Solution recommandée pour les interfaces
- Impose de référencer l'objet en tant qu'interface pour voir les méthodes

```
class Calculator : ICalculator
{
    double ICalculator.Add()
    {
    }

    double ICalculator.Subtract()
    {
    }

    ...
}
```



Méthodes d'interface par défaut

- Proposer une implémentation de méthode dans une interface
- Faciliter l'ajout de méthodes à une interface sans rupture avec le code existant
 - La nouvelle méthode proposera une implémentation par défaut en cas d'absence dans la classe
- Peut référencer tout membre de l'interface

Leçon 3: Définir des classes abstraites

- Classes Abstraites
- Méthodes abstraites

Classes Abstraites

- Contiennent du code comme toute classe
- Doivent être héritées
- Ne peuvent pas être instanciées
- Peuvent contenir des champs, des méthodes...
- Utilise le mot clé abstract

```
abstract class SalariedEmployee : Employee, ISalaried
{
    void ISalaried.PaySalary()
    {
        Console.WriteLine("Pay salary: {0}", currentSalary);
        // Logique métier
    }
    int currentSalary;
}

class ManualWorker : SalariedEmployee, ISalaried
{
    ...
}

class Manager : SalariedEmployee, ISalaried
{
    ...
}
```


Méthodes abstraites

Ajoutées à une classe abstraite

```
abstract class SalariedEmployee : Employee, ISalariedEmployee
{
    abstract void PayBonus();
    ...
}
```

Définie avec le modificateur abstract

Pas de corps de méthode



Les méthodes abstraites sont utiles lors du développement d'une classe abstraite qui implémente une interface ou dépend d'une méthode dont l'implémentation par défaut ne serait pas appropriée



Les classes qui héritent d'une classe abstraite doivent polymorpher toutes les méthodes abstraites ou être également abstraites



Atelier 01



Activité

20 min

- Exercice 1: Syntaxe d'héritage et d'interface en C#
- Exercice 2: Héritage de classe



A retenir

1. **Héritage en C#** : L'héritage est un pilier de la programmation orientée objet. Il permet à une classe (classe fille) d'hériter des membres d'une autre classe (classe mère).
2. **"Est-un" vs. "A-un"** : L'héritage établit une relation "est-un" entre la classe fille et la classe mère. C'est différent d'une composition où l'on a une relation "a-un".
3. **Les classes de base** : En C#, toutes les classes héritent implicitement de la classe Object, qui est la classe de base ultime du .NET Framework.
4. **L'héritage simple** : C# ne supporte que l'héritage simple, ce qui signifie qu'une classe ne peut hériter que d'une seule autre classe. Cependant, elle peut implémenter plusieurs interfaces.
5. **Les interfaces** : Une interface définit un contrat que les classes implémentantes doivent respecter. Une interface peut être considérée comme une classe entièrement abstraite qui ne contient que des déclarations de méthodes, propriétés, indexeurs et événements.
6. **Avantages des interfaces** : Les interfaces offrent une grande flexibilité en permettant à une classe d'hériter de plusieurs comportements sans les contraintes de l'héritage multiple direct.
7. **Classes abstraites** : Une classe abstraite est une classe qui ne peut pas être instanciée directement. Elle sert de base pour d'autres classes et peut contenir des méthodes abstraites (sans implémentation) que les classes dérivées doivent implémenter.
8. **Différence entre interfaces et classes abstraites** : Alors que les interfaces définissent un contrat sans fournir d'implémentation, les classes abstraites peuvent fournir une implémentation partielle tout en imposant un certain contrat à leurs classes dérivées.
9. **Override et New** : Lors de l'héritage, si une classe dérivée souhaite modifier le comportement d'une méthode héritée, elle peut utiliser le mot-clé override. Si elle souhaite masquer la méthode d'origine, elle utilise new.
10. **Protection contre la modification** : L'utilisation du mot-clé sealed sur une classe empêche d'autres classes d'hériter de celle-ci, garantissant ainsi que ses comportements et propriétés ne seront pas modifiés de manière inattendue.

Chapitre 9 - Gestion de la durée de vie des objets et contrôle des ressources



Plan de la formation

- 1 - Introduction à C# et .Net
- 2 - Structure de programmation C#
- 3 - Déclaration et appel de méthodes
- 4 - Gestion d'exceptions
- 5 - Lire et écrire dans des fichiers
- 6 - Création de nouveaux types de données
- 7 - Encapsulation de données et de méthodes
- 8 - Héritage de classes et implémentation d'interfaces
- 9 - Gestion de la durée de vie des objets et contrôle des ressources
- 10 - Encapsulation avancée
- 11 - Découplage de méthodes et gestion des ressources
- 12 - Utilisation des collections et des itérateurs
- 13 - Programmation asynchrone et tâches
- 14 - Utilisation de LINQ pour interroger des données
- 15 - Développement dirigé par les tests

Objectifs :

- Comprendre la gestion de la mémoire en C# et comment elle impacte la durée de vie des objets.
- Apprendre à gérer explicitement la mémoire et à libérer les ressources.
- Acquérir des connaissances sur les bonnes pratiques pour éviter les fuites de mémoire et assurer une utilisation efficace des ressources.

Objectifs :

1. Comprendre la gestion de la mémoire en C# et comment elle impacte la durée de vie des objets.
2. Apprendre à gérer explicitement la mémoire et à libérer les ressources.
3. Acquérir des connaissances sur les bonnes pratiques pour éviter les fuites de mémoire et assurer une utilisation efficace des ressources.

Positionnement : Dans un environnement de développement moderne, la gestion efficace des ressources est cruciale pour assurer des performances optimales et éviter les fuites de mémoire. Ce chapitre offre une perspective approfondie sur la manière dont C# gère la durée de vie des objets et comment les développeurs peuvent intervenir pour optimiser et contrôler l'utilisation des ressources.

Sujets clés qui seront couverts :

- 1. Le ramasse-miettes (Garbage Collector)** : Comprendre son rôle, son fonctionnement et comment il gère la libération de mémoire.
- 2. Finaliseurs et Dispose** : Différence entre eux, quand et comment les utiliser pour nettoyer les ressources.
- 3. La classe IDisposable** : Son importance, comment l'implémenter et pourquoi elle est essentielle pour la gestion des ressources non gérées.
- 4. Utilisation du mot-clé 'using'** : Comment il facilite la gestion des ressources et garantit que les objets sont correctement disposés.
- 5. Weak References** : Comprendre ce qu'elles sont, comment elles fonctionnent et dans

quelles situations elles peuvent être utiles.

6. Patterns et bonnes pratiques : Approfondir les modèles couramment utilisés pour gérer les ressources et assurer une libération de mémoire efficace.

7. Détecter les fuites de mémoire : Outils et techniques pour identifier et résoudre les fuites de mémoire dans les applications .NET.



Commençons par échanger

Pouvez-vous expliquer comment fonctionne le ramasse-miettes en C# et pourquoi il est essentiel pour la gestion de la mémoire ?

Avez-vous déjà rencontré les méthodes `Dispose` ou `Finalize` lors de votre expérience de codage ? Si oui, pouvez-vous nous dire dans quel contexte ?

Quelle est l'importance de l'interface `IDisposable` et dans quelles situations l'avez-vous trouvée utile dans le développement d'applications ?

Avez-vous déjà utilisé le mot-clé `'using'` dans vos codes précédents ? Comment cela a-t-il influencé la gestion des ressources de votre application ?

Avez-vous déjà rencontré des problèmes de fuites de mémoire dans une application ? Comment avez-vous identifié et résolu ces problèmes ?

1. Garbage Collector (GC) : Pouvez-vous expliquer comment fonctionne le ramasse-miettes en C# et pourquoi il est essentiel pour la gestion de la mémoire ?

2. Dispose vs Finalise : Avez-vous déjà rencontré les méthodes `Dispose` ou `Finalize` lors de votre expérience de codage ? Si oui, pouvez-vous nous dire dans quel contexte ?

3. IDisposable : Quelle est l'importance de l'interface `IDisposable` et dans quelles situations l'avez-vous trouvée utile dans le développement d'applications ?

4. Using : Avez-vous déjà utilisé le mot-clé `'using'` dans vos codes précédents ? Comment cela a-t-il influencé la gestion des ressources de votre application ?

5. Fuites de mémoire : Avez-vous déjà rencontré des problèmes de fuites de mémoire dans une application ? Comment avez-vous identifié et résolu ces problèmes ?

Vue d'ensemble

- Introduction au Garbage Collection
- Gestion des Ressources

Leçon 1: Introduction au Garbage Collection

- Cycle de vie des objets
- Ressources managés dans .NET
- Fonctionnement du Garbage Collector
- Définir un destructeur
- La classe GC

Cycle de vie des objets

◦ Bloc de mémoire alloué

1

2

Mémoire convertie en objet (constructeur)

3

Objet utilisé dans l'application

4

Objet nettoyé (destructeur)

5

Mémoire réclamée

Ressources managées dans .NET

Types Valeurs

- Créés sur la pile
- Détruits automatiquement quand hors de portée
- Jamais fragmentés
- Rapides à allouer et désallouer

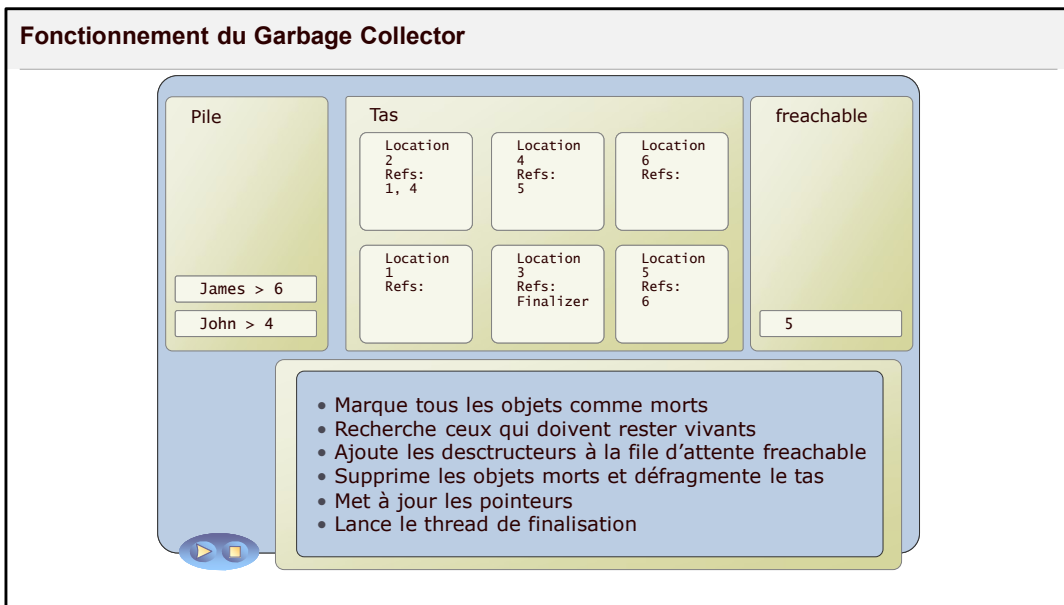
```
private void MyMethod(...)
{
    MyStruct a = ...;
    DateTime z = ...;
    double y = ...;
    int x = ...;
    ...
    // x, y, z et a disparaissent ici
}
```

Types références

- Créés dans le tas
- Supportent plusieurs références au même objet
- Détruits après la disparition de toutes les références
- Fragmentation possible
- Gérés par le garbage collector
- Plus coûteux à gérer

```
string s = ...;
MyClass c = ...;

private void MyMethod2(...)
{
    string t = s;
    MyClass d = c;
    ...
    // les références t et d
    disparaissent
    // les objets s et c existent encore
}
```



Définir un destructeur

```
class Employee
{
    ...
    ~Employee()
    {
        // Code Nettoyage
    }
}
```

- Le destructeur est défini par un tilde (~) suivi du nom de la classe

- Les types valeurs ne peuvent pas avoir de destructeur

- Les destructeurs n'ont pas de paramètres

- Le compilateur convertit le destructeur en méthode polymorphe Finalize



```
class Employee
{
    ...
    protected override void Finalize()
    {
        try
        {
            // Code Nettoyage
        }
        finally
        {
            base.Finalize();
        }
    }
}
```

La classe GC

Méthode	Description
Collect	Force le garbage collection
WaitForPendingFinalizers	Suspend le thread en cours jusqu'à la fin de traitement de la file freachable
SuppressFinalize	Désactive l'invocation du destructeur sur l'objet passé en paramètre
ReRegisterForFinalize	Réactive l'invocation du destructeur pour l'objet passé en paramètre
AddMemoryPressure	Informe le système de l'allocation d'un bloc volumineux de mémoire non managé
RemoveMemoryPressure	Informe le système de la libération d'un bloc volumineux de mémoire non managé

Leçon 2: Gestion des Ressources

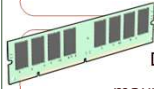
- Pourquoi gérer les ressources dans un environnement managé?
- Modèle Dispose
- Gestion des ressources dans une application
- Déclaration using

Pourquoi gérer les ressources dans un environnement managé?

Les ressources non managées ne sont pas du ressort du garbage collection:

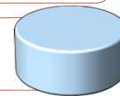
- Vous devez vous charger de libérer manuellement ces ressources lorsqu'un objet est détruit
- Ne pas libérer proprement des objets non managés peut être source de problèmes (perte de données, fuite mémoire...)

Fichiers verrouillés empêchant d'autres applications d'y accéder



Données mises en cache mémoire, perdues suite à une mauvaise gestion des objets

Les connexions aux bases de données sont souvent limitées



Modèle Dispose

```
class LogFileWriter : ..., IDisposable
{
    private bool isDisposed = false;

    ~LogFileWriter()
    {
        Dispose(false);
    }

    public void Dispose()
    {
        Dispose(true);
        GC.SuppressFinalize(this);
    }

    protected virtual void
        Dispose(bool isDisposing)
    {
        if (!isDisposed)
        {
            if (isDisposing)
            {
                // Nettoyage ressources managées;
            }
            // Nettoyage ressources non managées;
            isDisposed = true;
            base.Dispose(isDisposing);
        }
    }
}
```

Implémenter l'interface IDisposable

Garder une trace de l'état de l'objet

Ajouter un destructeur qui assurera
le nettoyage automatique de l'objet

Ajouter la méthode d'interface
Dispose qui assurera le nettoyage
manuel de l'objet et désactivera le
destructeur

Ajouter une méthode virtuelle
Dispose qui par le paramètre
booléen, effectuera le nettoyage
manuel (toutes les ressources) ou
automatique (seulement les
ressources non managées)

Gestion des ressources dans une application

- Implémentez l'interface *IDisposable* sur les classes qui gèrent des données que le CLR ne peut pas récupérer
- Beaucoup de classes du Framework son *IDisposable*
- Il est généralement conseillé d'utiliser ces classes avec la déclaration **using**
- Si le **using** n'est pas possible, utiliser un try/catch

```
DisposableObject disp = new  
    DisposableObject();  
try  
{  
    // Utiliser l'objet  
}  
finally  
{  
    disp.Dispose();  
}
```

Déclaration using

- Ne fonctionne qu'avec des types *IDisposable*
- A ne pas confondre avec l'instruction **using** d'import des espaces de noms

```
using(DisposableObject disp = new DisposableObject())
{
    // Utiliser l'objet
} // Dispose
```

```
DisposableObject disp2 = new DisposableObject();
using(disp2)
{
    // Utiliser l'objet
} // Dispose
```

```
Using DisposableObject disp = new DisposableObject();
// Utiliser l'objet
// Dispose quand la variable est hors de portée
```



Atelier 01

Activité
20 min

- Exercice 1: Implémentation de l'interface IDisposable

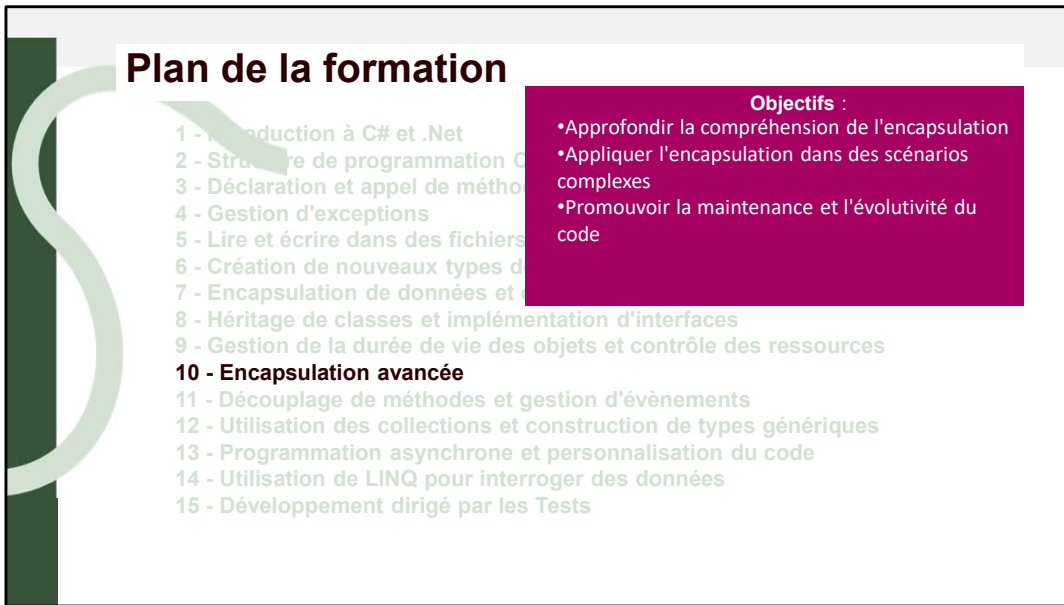


A retenir

1. **Automatisation de la gestion de la mémoire** : En C#, le Garbage Collector (GC) gère automatiquement la libération de la mémoire qui n'est plus utilisée, ce qui élimine la nécessité pour le développeur de le faire manuellement.
2. **Principe du Garbage Collection** : Le GC fonctionne en traquant et libérant la mémoire des objets qui ne sont plus référencés dans l'application, assurant ainsi une utilisation efficace de la mémoire.
3. **Eviter les fuites de mémoire** : Même avec le GC, il est possible d'avoir des fuites de mémoire si les objets sont toujours référencés mais inutilisés. Il est crucial de surveiller et de déréférencer les objets lorsque nécessaire.
4. **L'importance de IDisposable** : L'interface IDisposable fournit le mécanisme pour libérer les ressources non gérées telles que les fichiers, les flux et les connexions. Lorsqu'une classe implémente cette interface, elle doit fournir une méthode Dispose pour la libération des ressources.
5. **Utilisation du mot-clé using** : En C#, le mot-clé using garantit que les objets IDisposable sont correctement disposés, ce qui est particulièrement utile pour s'assurer que les ressources sont libérées immédiatement après leur utilisation.
6. **Finaliseurs et destructeurs** : En C#, les destructeurs (ou finaliseurs) peuvent être utilisés pour libérer des ressources non gérées, mais ils ne devraient être utilisés que lorsque c'est absolument nécessaire car le GC gère cela automatiquement pour la plupart des cas.
7. **Gestion des ressources non gérées** : Toutes les ressources, comme les connexions à la base de données ou les fichiers, ne sont pas gérées par le GC. Il est crucial de comprendre quand et comment libérer ces ressources pour éviter des problèmes de performance ou d'accès.
8. **Performance et GC** : Bien que le GC soit conçu pour être efficace, une utilisation excessive de la mémoire peut entraîner des performances médiocres. Les développeurs doivent être conscients de l'impact de la création excessive d'objets et de la rétention de références inutiles.
9. **Les générations du GC** : Le GC utilise le concept de générations pour optimiser la collecte des objets. Comprendre comment les différentes générations fonctionnent peut aider à écrire un code plus performant.
10. **Responsabilité du développeur** : Bien que le GC soit puissant, le développeur doit toujours être attentif à la gestion de la mémoire et des ressources pour garantir des applications performantes et robustes.-

Chapitre 10 - Encapsulation avancée





Plan de la formation

- 1 - Introduction à C# et .Net
- 2 - Structure de programmation C#
- 3 - Déclaration et appel de méthodes
- 4 - Gestion d'exceptions
- 5 - Lire et écrire dans des fichiers
- 6 - Création de nouveaux types d'objets
- 7 - Encapsulation de données et méthodes
- 8 - Héritage de classes et implémentation d'interfaces
- 9 - Gestion de la durée de vie des objets et contrôle des ressources
- 10 - Encapsulation avancée**
- 11 - Découplage de méthodes et gestion d'événements
- 12 - Utilisation des collections et construction de types génériques
- 13 - Programmation asynchrone et personnalisation du code
- 14 - Utilisation de LINQ pour interroger des données
- 15 - Développement dirigé par les Tests

Objectifs :

- Approfondir la compréhension de l'encapsulation
- Appliquer l'encapsulation dans des scénarios complexes
- Promouvoir la maintenance et l'évolutivité du code

Objectifs :

1.Approfondir la compréhension de l'encapsulation : Aller au-delà des concepts de base pour maîtriser les techniques avancées d'encapsulation et savoir quand et comment les utiliser.

2.Appliquer l'encapsulation dans des scénarios complexes : Apprendre à utiliser l'encapsulation pour résoudre des problèmes de conception spécifiques et améliorer la robustesse du code.

3.Promouvoir la maintenance et l'évolutivité du code : Grâce à une utilisation judicieuse de l'encapsulation avancée, garantir que le code est non seulement sécurisé mais aussi facile à maintenir et à étendre.

Positionnement : Après avoir abordé les bases de l'encapsulation, ce chapitre vise à équiper les apprenants des techniques avancées pour améliorer la modularité, la sécurité et la maintenabilité de leurs applications. L'encapsulation avancée est essentielle pour concevoir des systèmes évolutifs et robustes qui peuvent facilement s'adapter aux exigences changeantes.

Sujets clés qui seront couverts :

1.Propriétés avancées : Utilisation des propriétés pour fournir un contrôle d'accès granulaire, y compris les propriétés en lecture seule, les propriétés indexées et les propriétés auto-implementées.

2.Méthodes d'extension : Comment étendre les types existants avec de nouvelles méthodologies sans les modifier directement.

3. Attributs et métadonnées : Utilisation des attributs pour ajouter des métadonnées aux éléments de code, et comment interroger ces métadonnées à l'exécution.

4. Classes et méthodes partielles : Découvrir comment diviser la définition d'une classe ou d'une méthode sur plusieurs fichiers pour une meilleure organisation.

5. Types imbriqués : Comprendre comment et pourquoi définir un type à l'intérieur d'un autre type.

6. Accès aux membres via des méthodes d'accès : L'utilisation de getters et setters pour contrôler l'accès et la modification des données encapsulées.



Commençons par échanger

Pourquoi pensez-vous que l'encapsulation est considérée comme un pilier fondamental de la programmation orientée objet ?

Quels avantages voyez-vous à limiter l'accès direct aux données d'une classe ?

Avez-vous déjà rencontré des situations où un changement dans une partie du code a eu des effets inattendus sur d'autres parties du système ? Comment l'encapsulation pourrait-elle aider à prévenir de tels problèmes ?

Quelle est votre expérience avec les propriétés dans d'autres langages de programmation ou frameworks ? Comment les utilisez-vous généralement ?

Est-ce que l'idée de "types imbriqués" vous évoque quelque chose ? Dans quels contextes pourriez-vous imaginer que cela soit utile ?

- 1. Pourquoi pensez-vous que l'encapsulation est considérée comme un pilier fondamental de la programmation orientée objet ?**
- 2. Objectif : Amener les stagiaires à réfléchir à l'importance de l'encapsulation et à ses avantages en matière de conception de logiciels.*
- 3. Quels avantages voyez-vous à limiter l'accès direct aux données d'une classe ?**
- 4. Objectif : Encourager les stagiaires à reconnaître les dangers potentiels d'un accès non contrôlé et les avantages de l'encapsulation pour la sécurité et la robustesse du code.*
- 5. Avez-vous déjà rencontré des situations où un changement dans une partie du code a eu des effets inattendus sur d'autres parties du système ? Comment l'encapsulation pourrait-elle aider à prévenir de tels problèmes ?**
- 6. Objectif : Mettre en évidence l'importance de l'encapsulation pour assurer la maintenabilité et réduire les effets secondaires du code.*
- 7. Quelle est votre expérience avec les propriétés dans d'autres langages de programmation ou frameworks ? Comment les utilisez-vous généralement ?**
- 8. Objectif : Se baser sur les connaissances préexistantes des stagiaires concernant les propriétés et les amener à comparer et contraster avec les techniques avancées d'encapsulation qu'ils vont apprendre.*
- 9. Est-ce que l'idée de "types imbriqués" vous évoque quelque chose ? Dans quels contextes pourriez-vous imaginer que cela soit utile ?**
- 10. Objectif : Stimuler la curiosité des stagiaires sur les concepts avancés et les préparer à voir comment ils s'insèrent dans des scénarios réels de programmation.*

- Création et utilisation des propriétés
- Création et utilisation des indexeurs
- Surcharge d'opérateurs

- Définition des propriétés
- Création d'une propriété
- Propriétés automatiques
- Instanciation d'objet avec propriétés
- Propriétés en initialisation uniquement
- Propriétés et Interface
- Bonnes pratiques

Formation: Les bases fondamentales du développement .Net en C# sous Visual Studio - Manuel - Stagiaire

Comportement d'une
méthode

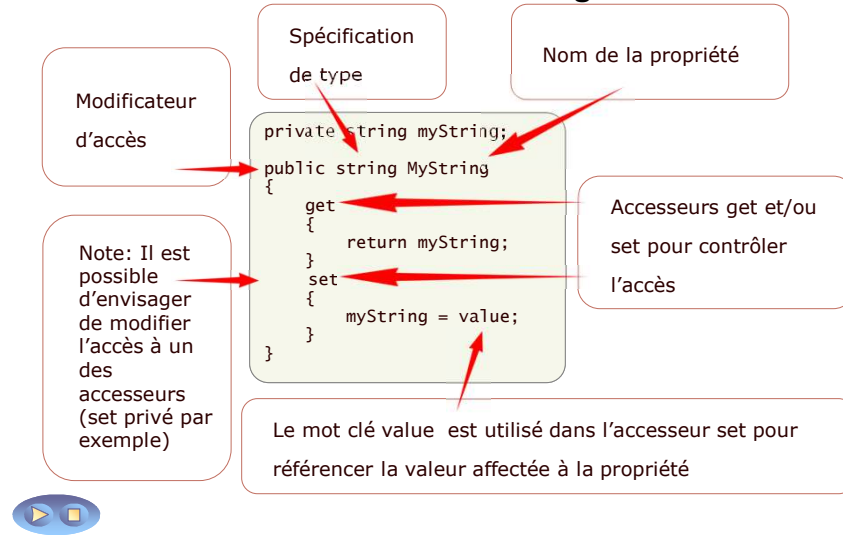
Syntaxe d'un champ

Property:
get and set accessors

Les propriétés permettent:

- Le contrôle de l'accès aux données
- La validation
- Le contrôle Lecture/Ecriture

Formation des fondamentaux du développement .Net en C# sous Visual Studio - Manuel Stagiaire



Formation aux bases fondamentales du développement .Net en C# sous Visual Studio - Manuel - Stagiaire

```
public string Name { get; set; }
```

A la compilation, la propriété automatique est remplacée par le code suivant

```
private string _name;
public string Name
{
    get
    {
        return _name;
    }
    set
    {
        this._name = value;
    }
}
```

- Permet d'assurer l'encapsulation des données rapidement et simplement
- Permet l'évolutivité du code tout en assurant la compatibilité descendante
- Peut être défini par get/set ou get uniquement
- Peut être initialisé à la déclaration

```
public string Name { get; set; } = "John";
```



Formation des développeurs du développement .Net en C# sous Visual Studio - Manuel - Stagiaire

```
Employee john = new Employee { Name = "John" };  
Employee louisa = new Employee() { Department = "Technical" };  
Employee mike = new Employee  
{  
    Name = "Mike",  
    Department = "Technical"  
};
```

- Les initialiseurs d'objets évitent le problème de création d'un grand nombre de constructeurs
- Un constructeur par défaut devrait assurer les initialisations aux valeurs par défaut
- Le constructeur est toujours appelé d'abord, les propriétés ensuite
- Il est possible d'invoquer un constructeur vide (avec ou sans parenthèses) ou bien un constructeur avec paramètres (bien évidemment avec les valeurs appropriées)

Propriétés en initialisation uniquement

- L'utilisation des initialiseurs d'objets nécessite une propriété disposant de l'accesseur *set*
 - Il est donc envisageable de modifier cette valeur ensuite
- En remplaçant le mot-clé *set* par le mot-clé *init*, la propriété n'autorise plus la modification sauf:
 - Dans un initialiseur d'objet
 - Dans un constructeur
 - Dans l'accesseur *init*

```
public string Name { get; init; }
```

```
private string _name;
public string Name
{
    get
    {
        return _name;
    }
    init
    {
        this._name = value;
    }
}
```


Formation Les fondamentaux du développement .Net en C# sous Visual Studio - Manuel - Stagiaire

Rappel: une interface est un contrat entre un type et une application.
Elle ne contient pas de détails d'implémentation (sauf méthode par défaut).

- Les champs sont considérés comme détail d'implémentation et sont donc interdits dans une déclaration d'interface.
- Les propriétés sont avant tout des méthodes d'accès à des données, et sont donc autorisées dans les interface (il en est de même avec les indexeurs)

```
interface IPerson
{
    string Name { get; set; }
    int Age { get; }
    DateTime DateOfBirth { set; }
}
```

Syntaxe identique à celle des propriétés automatiques, mais s'agissant de prototypes, il est possible de choisir les accesseurs

Formation Les fondamentaux du développement .Net en C# sous Visual Studio - Manuel - Stagiaire

N'utilisez des propriétés uniquement que lorsque cela a un sens

BankAccount

- ☒ double Balance (get, ~~set~~)
- ☒ WithdrawMoney(double Amount)
- ☒ DepositMoney (double Amount)



Une application peut-elle directement définir le solde?



Le code de l'accesseur get ne doit pas modifier de données



Attention aux problèmes de récursivité dans les propriétés

- Définition
- Création
- Comparaison entre Indexeur et Tableau
- Initialisation d'indexeur

Formation : Les fondamentaux du développement .Net en C# sous Visual Studio - Manuel - Stagiaire

- Fournit une syntaxe type-tableau pour accéder aux membres d'un ensemble
- Peut utiliser n'importe quel type de données pour accéder à un élément
- Peut être surchargé

```
CustomerAddressBook addressBook = ...;  
Address customerAddress = addressBook["a2332"];  
...  
customerAddress = addressBook[99];
```

Formation : Les fondamentaux du développement .Net en C# sous Visual Studio - Manuel - Stagiaire

Devrait être
public

Type de retour

Paramètre(s) de l'indexeur

```
public Customer this[string CustomerID]
{
    get
    {
        return database.FindCustomer(CustomerID);
    }
    set
    {
        database.UpdateCustomer(CustomerID, value);
    }
}
```

Tous les indexeurs s'appellent
this

Ajout des accesseurs get
et/ou set



	Tableaux	Indexeurs
Indexation	Uniquement numérique entier	Numérique, Chaîne de caractères, voire multiple
Surcharge	Pas de surcharge possible	Possibilité de surcharge
Passage en tant que paramètre de méthode	Peut être utilisé avec ou sans ref / out	Ne peut pas être utilisé en tant que paramètre ref ou out

Initialisation d'indexeur

- Basée sur les initialiseurs d'objet
- Initialise chaque membre de l'ensemble sous-jacent
- Egalement disponible sur les collections

```
CustomerAddressBook addressBook =  
    new CustomerAddressBook {  
        ["A1001"] = new Address { ... },  
        ["A1002"] = new Address { ... },  
        ...  
    };
```

- Définition
- Surcharge d'un opérateur
- Restrictions
- Bonnes pratiques
- Opérateurs de Conversion

Formation : Les fondamentaux du développement .Net en C# sous Visual Studio - Manuel - Stagiaire

-

/

==

+

!=

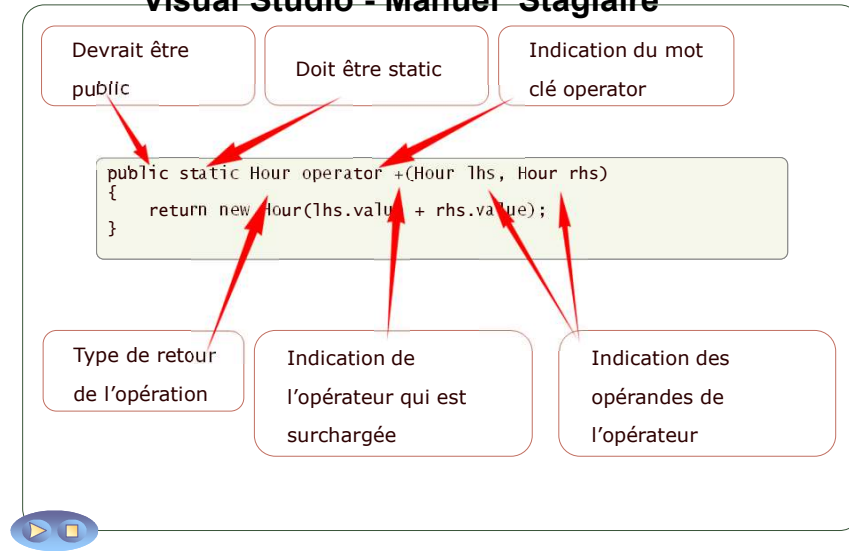
```
Class MyType { ... }
```

```
...  
MyType var1 = ...;  
MyType var2 = ...;  
...
```

```
? = var1 + var2;
```



Formation des fondamentaux du développement .Net en C# sous Visual Studio - Manuel Stagiaire



Formation : Les fondamentaux du développement .Net en C# sous Visual Studio - Manuel - Stagiaire

- 1 Il est impossible de changer précedence et associativité d'un opérateur
- 2 Il est impossible de changer la multiplicité d'un opérateur
- 3 Il est impossible d'inventer de nouveaux opérateurs (rôle des méthodes)
- 4 Il est impossible de modifier les opérateurs pour les types intégrés
- 5 Tous les opérateurs ne sont pas surchargeables, par exemple le point (.)
- 6 Il est obligatoire de définir certains opérateurs par pair (==, !=, etc)

Formation Les fondamentaux du développement .Net en C# sous Visual Studio - Manuel - Stagiaire

- Définissez les opérateurs de manière symétrique

```
public static Salary operator +(Salary salary, double number)
{
    return new Salary(salary.amount + number);
}
public static Salary operator +(double number, Salary salary)
{
    return salary + number;
}
```

- Ne modifiez pas les opérandes

- Ne définissez des opérateurs que lorsque cela a un sens

```
public static BankAccount operator +(BankAccount account, decimal amount)
{ ... }
public static BankAccount operator -(BankAccount account, decimal amount)
{ ... }
```

Formation des fondamentaux du développement .Net en C# sous Visual Studio - Manuel - Stagiaire

Conversion
implicite:
Pas de perte
de données

```
public static implicit operator int (Hour from)
{
    return from.value;
}
```

```
public static void MyMethod(int parameter) { ... }
public static void Main()
{
    Hour lunch = new Hour(12);
    Example.MyMethod(lunch); // implicite
}
```

Conversion
explicite:

Risque de
perte de
données,
nécessite un
cast

```
public static explicit operator Hour (int from)
{
    return new Hour(from);
}
```

```
public static void MyOtherMethod(Hour parameter) { ... }
public static void Main()
{
    int lunch = 12;
    Example.MyOtherMethod((Hour)lunch); // explicite
}
```

Atelier 01



Activité
20 min

- Exercice 1: Création de propriétés
- Exercice 2: Création d'une propriété Indexeur
- Exercice 3: Surcharge d'opérateur (facultatif)

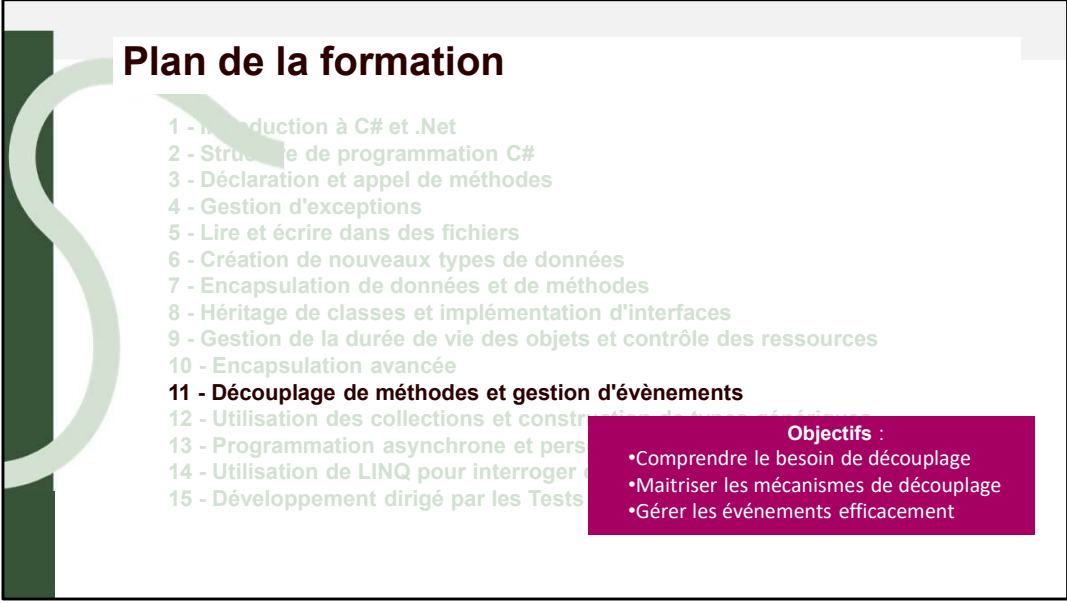


A retenir

- Propriétés** : Les propriétés en C# permettent d'encapsuler les données d'un objet en offrant une méthode élégante pour lire, écrire ou calculer la valeur d'un champ privé.
- Sécurité des données** : Grâce aux propriétés, il est possible de valider les données avant de les affecter à un champ, ce qui garantit l'intégrité et la sécurité des données.
- Indexeurs** : Les indexeurs permettent à une classe d'être indexée comme un tableau, offrant une manière intuitive d'accéder à ses éléments.
- Flexibilité avec les indexeurs** : Vous pouvez définir plusieurs indexeurs pour une seule classe, chacun ayant un type d'index différent.
- Surcharge d'opérateurs** : En C#, vous pouvez redéfinir le comportement des opérateurs existants pour vos propres types, offrant une expression naturelle pour vos classes.
- Opérations intuitives** : La surcharge d'opérateurs permet de rendre certaines opérations plus intuitives, par exemple, en ajoutant deux objets ensemble avec l'opérateur +.
- Respect des conventions** : Bien que la surcharge d'opérateurs offre une grande flexibilité, il est essentiel de s'assurer que le comportement redéfini reste intuitif et ne surprend pas les utilisateurs de la classe.
- Encapsulation avancée** : L'encapsulation ne concerne pas uniquement la protection des données. Elle englobe également la manière dont les utilisateurs de la classe interagissent avec elle, rendant les interactions plus naturelles et intuitives.
- Cohérence** : L'utilisation de propriétés, d'indexeurs et de surcharge d'opérateurs doit être cohérente pour éviter la confusion.
- Meilleures pratiques** : Tout en exploitant ces fonctionnalités avancées, il est essentiel de se rappeler des meilleures pratiques en matière de conception d'objet pour garantir que la classe reste maintenable et compréhensible.

Chapitre 11 - Découplage de méthodes et gestion d'évènements





Plan de la formation

- 1 - Introduction à C# et .Net
- 2 - Structure de programmation C#
- 3 - Déclaration et appel de méthodes
- 4 - Gestion d'exceptions
- 5 - Lire et écrire dans des fichiers
- 6 - Création de nouveaux types de données
- 7 - Encapsulation de données et de méthodes
- 8 - Héritage de classes et implémentation d'interfaces
- 9 - Gestion de la durée de vie des objets et contrôle des ressources
- 10 - Encapsulation avancée
- 11 - Découplage de méthodes et gestion d'évènements**
- 12 - Utilisation des collections et constructions de données
- 13 - Programmation asynchrone et persistance
- 14 - Utilisation de LINQ pour interroger des données
- 15 - Développement dirigé par les Tests

Objectifs :

- Comprendre le besoin de découplage
- Maîtriser les mécanismes de découplage
- Gérer les événements efficacement

Objectifs :

1. Comprendre le besoin de découplage : Introduire l'importance de concevoir des composants logiciels modulaires et indépendants, et comment le découplage aide à réaliser cette vision.

2. Maîtriser les mécanismes de découplage : Apprendre à utiliser les délégués, les événements et les lambdas pour créer des applications flexibles et extensibles.

3. Gérer les événements efficacement : Apprendre à créer, soulever et gérer les événements dans les applications pour répondre aux changements d'état ou à d'autres conditions importantes.

Positionnement : Après avoir abordé des concepts avancés comme l'encapsulation, ce chapitre s'oriente vers une perspective plus architecturale, où la manière dont les différentes parties d'une application interagissent entre elles est aussi importante que leur fonctionnement interne. Le découplage est essentiel pour garantir que les systèmes soient modulaires, maintenables et évolutifs.

Sujets clés couverts :

1. Délégation et délégués : Introduction aux délégués en C# et comment ils permettent de référencer et d'appeler des méthodes de manière dynamique.

2. Lambdas et expressions lambda : Exploration des expressions lambda, leur syntaxe, et comment elles offrent une façon concise de représenter des méthodes ou des fonctions anonymes.

3. Événements en C# : Comment déclarer, soulever et s'abonner à des événements dans

les classes, et leur importance dans les modèles de conception orientés événements.

4. Principe de responsabilité unique : Discussion sur la manière dont le découplage aide à garantir que chaque classe ou module ait une seule responsabilité, renforçant l'architecture logicielle.

5.Scénarios courants et meilleures pratiques : Analyse de scénarios réels où le découplage et la gestion des événements sont cruciaux, et discussion des meilleures pratiques pour leur mise en œuvre.



Commençons par échanger

"Pouvez-vous me donner un exemple d'une situation où deux composants d'une application doivent interagir entre eux, mais sans dépendre fortement l'un de l'autre ?"

"Dans un contexte de développement logiciel, pourquoi pensez-vous qu'il est important que les différents composants d'une application soient indépendants et modulaires ?"

"Avez-vous déjà utilisé une application ou un service qui vous notifie lorsqu'un événement spécifique se produit ? Comment imaginez-vous que cela fonctionne techniquement en arrière-plan ?"

"Pourquoi est-il essentiel que chaque composant ou classe d'une application ait sa propre responsabilité claire ? Comment cela pourrait-il influencer la qualité du code et la maintenance ?"

"Dans quelles situations pourriez-vous avoir besoin de décider à la volée quelle fonction ou méthode appeler, plutôt que de l'avoir codée en dur ?"

1.Interactions des Composants : "Pouvez-vous me donner un exemple d'une situation où deux composants d'une application doivent interagir entre eux, mais sans dépendre fortement l'un de l'autre ?"

2.Principe d'architecture : "Dans un contexte de développement logiciel, pourquoi pensez-vous qu'il est important que les différents composants d'une application soient indépendants et modulaires ?"

3.Notification & Réaction : "Avez-vous déjà utilisé une application ou un service qui vous notifie lorsqu'un événement spécifique se produit ? Comment imaginez-vous que cela fonctionne techniquement en arrière-plan ?"

4.Principe de Responsabilité : "Pourquoi est-il essentiel que chaque composant ou classe d'une application ait sa propre responsabilité claire ? Comment cela pourrait-il influencer la qualité du code et la maintenance ?"

5.Référence Dynamique : "Dans quelles situations pourriez-vous avoir besoin de décider à la volée quelle fonction ou méthode appeler, plutôt que de l'avoir codée en dur ?"

Formation : Les fondamentaux du développement .Net en C# sous Visual Studio - Manuel - Stagiaire

vue d'ensemble

- Déclaration et utilisation de délégués
- Utilisation des expressions Lambda
- Gestion d'événements

- Découplage opération/méthode
- Définition d'un délégué
- Appel d'un délégué
- Méthodes anonymes

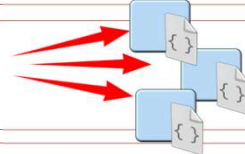
- Choisir la méthode au moment de l'exécution
 - Méthode A ou méthode B ou méthode C



- Méthodes de rappel (Callback)
 - Fournir une méthode à exécuter lorsqu'un appel asynchrone se termine



- Opérations à multidiffusion (Multicast)
 - Méthode A et méthode B et méthode C



Implémentés par les délégués, que l'on peut considérer comme des références vers des méthodes



Formation : Les fondamentaux du développement .Net en C# sous Visual Studio - Manuel Stagiaire

Définir un delegate

Développeur de Classe

- Déclaration de la signature du délégué à l'aide du mot clé delegate

```
public delegate bool isValidDelegate();
```

- Création de l'instance qui stockera les références de méthodes

```
public isValidDelegate isValid = null;
```

Développeur Applicatif

- Ajout des références par l'opérateur +=

```
isValid += CheckStateValid;  
isValid += new isValidDelegate(CheckControl);
```

- Suppression de références par l'opérateur -=

```
isValid -= CheckStateValid;
```



Formation : Les fondamentaux du développement .Net en C# sous Visual Studio - Manuel Stagiaire

Appel d'un délégué

Toujours vérifier que le délégué soit non null avant de l'invoquer!

- Appel d'un délégué de façon synchrone

```
if (isValid != null)
{
    isValid();
}
```

- Appel d'un délégué de façon asynchrone à l'aide des méthodes BeginInvoke et EndInvoke

```
if (isValid != null)
{
    isValid.BeginInvoke(...);
}
```


Formation : Les fondamentaux du développement .Net en C# sous Visual Studio - Manuel Stagiaire

Méthodes anonymes

Ajout de la méthode anonyme aux références d'une instance de délégué

Utilisation du mot clé delegate

Indiquer si besoin les paramètres nécessaires

```
myDelegate instance += new
myDelegate(
delegate (int parameter)
{
// Perform operation.
return 10;
});
```

Mettre en oeuvre le corps de la méthode

Il n'est pas nécessaire d'indiquer le type de retour car il sera déterminé par le contexte, à savoir par la signature du délégué



- Définition
- Syntaxe
- Portée des variables dans les expressions Lambda
- Réécriture de méthodes et de propriétés
- Alternative par les fonctions locales

Formation : Les fondamentaux du développement .Net en C# sous Visual Studio - Manuel Stagiaire

Définition

- Une expression Lambda est une expression qui retourne une méthode
- Les expressions Lambda sont définies en utilisant l'opérateur =>
- Les expressions Lambda utilisent une syntaxe naturelle et concise
- Les types des paramètres et valeur de sortie sont déterminés par le contexte

$x \Rightarrow x * x$

Cet exemple peut être lu ainsi: Pour x, calculer x * x

Formation : Les fondamentaux du développement .Net en C# sous Visual Studio - Manuel Stagiaire

Syntaxe

Expression simple où le type du paramètre x est déterminé par le contexte

```
x => x * x
```

Expression mettant en oeuvre un bloc de code C#

```
x => { return x * x ; }
```

Expression sans paramètre d'entrée et invoquant une méthode

```
() => myObject.MyMethod(0)
```

Expression avec typage explicite du paramètre

```
(int x) => x / 2
```

Expression avec paramètres multiples, dont un passé par référence

```
(ref int x,int y) => {x++; return x/y;}
```



Formation : Les fondamentaux du développement .Net en C# sous Visual Studio - Manuel Stagiaire

Portée des Variables dans les Expressions Lambda

- Une expression Lambda peut utiliser toutes les variables accessibles dans la portée où elle est déclarée
- Une expression Lambda peut définir ses propres variables

```
string name;           // Champs de niveau classe
MyDelegate del = null;
...
void myMethod() {
    int count = 0;
    del += new MyDelegate(() =>
    {
        int temp = 10;
        CheckName(name);
        count++; // Résultat imprévisible
    });
}
```

- Utiliser des variables externes dans une expression Lambda peut étendre leur durée de vie: A utiliser avec précaution!
- Il est possible d'interdire ce comportement en préfixant la lambda par *static*

Réécriture de méthodes et de propriétés

Il est possible de simplifier l'écriture des méthodes et des propriétés en réutilisant la syntaxe des expressions Lambda

```
public string GetData(int index){
    return String.Format("Text: {0}",data[index]);
}
```



```
public string GetData(int index) => return String.Format("Text: {0}",data[index]);
```

```
// Constructeur
public ExpressionMembersExample(string label) => this.Label = label;
// Destructeur
~ExpressionMembersExample() => Console.Error.WriteLine("Finalized!");

private string label;
// Propriétés
public string Label
{
    get => label;
    set => this.label = value ?? "Default label";
}
```

Alternative par les fonctions locales

- Utile pour la gestion de la récursivité
- Plus performant que les expressions Lambda pour des fonctions appelées uniquement dans le contexte d'une méthode

```
public static int LambdaFactorial(int n)
{
    MonDelegate nthFactorial = (number) => (number < 2) ?
        1 : number * nthFactorial(number - 1);

    return nthFactorial(n);
}
```

```
public static int LocalFunctionFactorial(int n)
{
    return nthFactorial(n);

    int nthFactorial(int number) => (number < 2) ?
        1 : number * nthFactorial(number - 1);
}
```

- Définition
- Syntaxe
- Utilisation
- Bonnes pratiques

Formation : Les fondamentaux du développement .Net en C# sous Visual Studio - Manuel Stagiaire

Définition

- Basés sur les délégués, les événements permettent à un composant (fournisseur) de signaler à une application (consommateur) que quelque chose s'est produit
- Contrairement aux délégués, les événements ne peuvent être déclenchés que dans la classe qui l'a défini
- L'application consommatrice ajoute les références de méthodes qu'elle veut voir exécuter lors du déclenchement de l'événement. A noter que seuls les opérateurs += et -= sont autorisés
- Les événements sont beaucoup utilisés en .NET et en particulier dans les applications graphiques

Formation : Les fondamentaux du développement .Net en C# sous Visual Studio - Manuel Stagiaire

Syntaxe

Définir le délégué sur lequel sera basé l'événement; le délégué doit être au moins aussi visible que l'événement

```
public delegate void MyEventDelegate(object sender, EventArgs e);  
public event MyEventDelegate MyEvent = null;
```

Utiliser le mot
clé event

Spécifier le type du
délégué

Donner un nom à
l'événement



Il est possible de définir un événement
dans une interface

Formation : Les fondamentaux du développement .Net en C# sous Visual Studio - Manuel Stagiaire

Utilisation

Inscription à un événement par l'opérateur += comme cela aurait été le cas avec un simple délégué

```
MyEvent += new MyEventDelegate(myHandlingMethod);
```

Utilisation de l'opérateur -= pour se désinscrire

```
MyEvent -= myHandlingMethod;
```

Pour soulever/déclencher l'événement, il suffit de l'invoquer comme s'il s'agissait d'une méthode (en fait, nous invoquons les méthodes référencées)

```
if (MyEvent != null)
{
    EventArgs args = new EventArgs();
    MyEvent(this, args);
}
```

Toujours vérifier que l'événement est différent de null avant de le déclencher!





Utiliser la signature standard

```
void MyEvent(object sender, EventArgs e)
```



Utiliser une méthode *protected virtual* pour lancer l'événement



Ne jamais passer de valeurs *null* à l'événement

Il devient rare de créer soi-même ses propres déclarations de délégués.

En effet, il existe en .NET des délégués prédéfinis: les *délégués génériques*

- Action, Action<T>, Action<T1,T2>,...
- Func<TResult>, Func<T,Tresult>, Func<T1,T2,Tresult>,...

Les types T, T1, T2, ... sont définis au moment où le code est écrit.

Nous en reparlerons dans le module 12.



Atelier 01

Activité
20 min

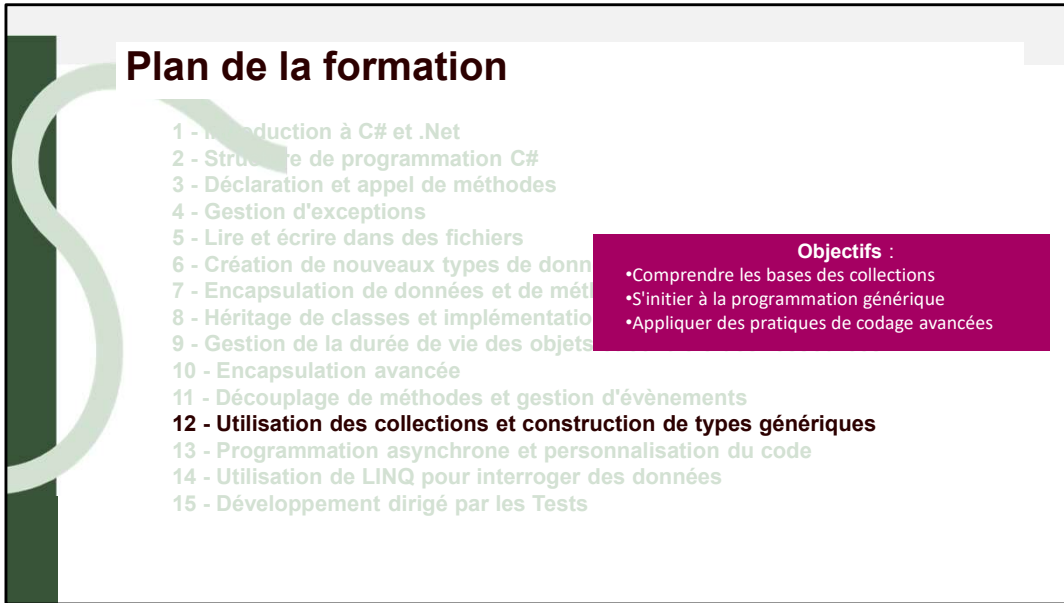
- Exercice 1: Application graphique et événement
- Exercice 2: Mise en œuvre d'un délégué (facultatif)
- Exercice 3: Création et gestion d'un événement (facultatif)



1. **Délégués comme Types** : Les délégués sont des types qui représentent des références à des méthodes avec un type de paramètre particulier et un type de retour. Ils peuvent référencer à la fois des méthodes statiques et des méthodes d'instance.
2. **Flexibilité des Délégués** : Grâce aux délégués, vous pouvez définir des opérations comme des paramètres et créer des méthodes plus flexibles et réutilisables.
3. **Multicast Délégués** : Un délégué peut référencer plusieurs méthodes; on parle alors de délégué multicast. Cela permet de déclencher plusieurs méthodes via une seule invocation de délégué.
4. **Expressions Lambda** : Les expressions lambda offrent une syntaxe concise pour écrire des méthodes anonymes. Elles sont particulièrement utiles pour écrire des fonctions courtes qui sont passées à des méthodes d'ordre supérieur.
5. **Syntaxe des Lambdas** : La syntaxe générale d'une expression lambda est (paramètres) => expression ou (paramètres) => { instructions; }.
6. **Les événements comme Communication** : Les événements sont un mécanisme par lequel un objet peut notifier d'autres objets que quelque chose s'est produit. Ils permettent une communication propre et découpée entre composants.
7. **Mécanisme Sous-Jacent** : Les événements utilisent en réalité des délégués en arrière-plan. Chaque événement est basé sur un délégué qui définit la signature des méthodes de rappel (handlers).
8. **Inscription et Désinscription** : Avec les opérateurs += et -=, les méthodes peuvent s'inscrire ou se désinscrire d'un événement et être appelées lorsque l'événement est déclenché.
9. **Principe de Sécurité** : Grâce aux événements, vous pouvez empêcher d'autres classes d'émettre l'événement de votre classe, garantissant ainsi que seul l'objet qui a défini l'événement peut déclencher ses handlers.
10. **Utilisation Judicieuse** : Bien que puissants, les délégués, les lambdas et les événements doivent être utilisés judicieusement pour garantir la lisibilité du code et éviter des complications inutiles.

Chapitre 12 - Utilisation des collections et construction de types génériques





Plan de la formation

- 1 - Introduction à C# et .Net
- 2 - Structure de programmation C#
- 3 - Déclaration et appel de méthodes
- 4 - Gestion d'exceptions
- 5 - Lire et écrire dans des fichiers
- 6 - Création de nouveaux types de données
- 7 - Encapsulation de données et de méthodes
- 8 - Héritage de classes et implémentation
- 9 - Gestion de la durée de vie des objets
- 10 - Encapsulation avancée
- 11 - Découplage de méthodes et gestion d'événements
- 12 - Utilisation des collections et construction de types génériques**
- 13 - Programmation asynchrone et personnalisation du code
- 14 - Utilisation de LINQ pour interroger des données
- 15 - Développement dirigé par les Tests

Objectifs :

- Comprendre les bases des collections
- S'initier à la programmation générique
- Appliquer des pratiques de codage avancées

Objectifs :

1. Comprendre les bases des collections : Acquérir une connaissance fondamentale des différents types de collections disponibles en C# et savoir quand et comment les utiliser efficacement.

2. S'initier à la programmation générique : Découvrir les avantages des types génériques pour écrire un code réutilisable et type-safe.

3. Appliquer des pratiques de codage avancées : Grâce aux concepts de collections et de génériques, les participants développeront des compétences pour écrire un code plus structuré, plus sûr et optimisé.

Positionnement : Après avoir abordé les fondamentaux de la programmation orientée objet, des méthodes et de la gestion d'événements, il est maintenant temps de se plonger dans les outils qui vous permettront de gérer et d'organiser des ensembles de données de manière plus efficace. Les collections et les génériques sont des éléments essentiels de la boîte à outils de tout développeur C#, permettant une manipulation efficace des données tout en maintenant la flexibilité et la sécurité du typage.

Sujets clés couverts :

1. Introduction aux Collections :

1. Les listes, piles, files d'attente et dictionnaires.
2. Avantages et inconvénients des différents types de collections.
3. Exploration de la bibliothèque de classes de collections.

2. Programmation Générique :

1. Comprendre les types génériques et leur avantage en termes de réutilisabilité et de sécurité du typage.
2. Création de classes, interfaces et méthodes génériques.
3. Contraintes génériques et variance.

3. Collections Génériques :

1. Avantages des collections génériques par rapport aux collections non génériques.
2. Exploration des classes telles que List<T>, Dictionary<TKey, TValue>, et d'autres collections génériques.

4. Bonnes pratiques dans l'utilisation des collections et des génériques :

1. Conseils pour choisir le bon type de collection en fonction du besoin.
2. Recommandations pour une utilisation efficace et optimisée des génériques.



Commençons par échanger

"Pouvez-vous me donner un exemple d'une situation où vous auriez besoin de stocker et d'organiser plusieurs éléments ou données dans votre code ? Comment l'avez-vous géré jusqu'à présent ?"

"Avez-vous déjà rencontré une situation où vous aviez à réutiliser une certaine fonctionnalité pour différents types de données ? Comment avez-vous abordé ce défi ?"

"Quels sont, selon vous, les défis associés à la gestion de grandes quantités de données dans une application ? Comment pensez-vous que des structures de données bien conçues pourraient aider à améliorer les performances ?"

"Pouvez-vous expliquer pourquoi il est important d'avoir un contrôle strict sur les types de données avec lesquels vous travaillez dans une application ? Avez-vous déjà rencontré des problèmes dus à un mélange involontaire de types ?"

"Dans vos expériences précédentes de programmation, combien de fois avez-vous senti le besoin de réutiliser certaines portions de votre code pour différentes tâches ? Quels défis avez-vous rencontrés en essayant de le faire ?"

1.Question sur les bases des structures de données : "Pouvez-vous me donner un exemple d'une situation où vous auriez besoin de stocker et d'organiser plusieurs éléments ou données dans votre code ? Comment l'avez-vous géré jusqu'à présent ?"

2.Question sur la flexibilité du code : "Avez-vous déjà rencontré une situation où vous aviez à réutiliser une certaine fonctionnalité pour différents types de données ? Comment avez-vous abordé ce défi ?"

3.Question sur les performances : "Quels sont, selon vous, les défis associés à la gestion de grandes quantités de données dans une application ? Comment pensez-vous que des structures de données bien conçues pourraient aider à améliorer les performances ?"

4.Question sur la sécurité du typage : "Pouvez-vous expliquer pourquoi il est important d'avoir un contrôle strict sur les types de données avec lesquels vous travaillez dans une application ? Avez-vous déjà rencontré des problèmes dus à un mélange involontaire de types ?"

5.Question générale sur la réutilisabilité : "Dans vos expériences précédentes de programmation, combien de fois avez-vous senti le besoin de réutiliser certaines portions de votre code pour différentes tâches ? Quels défis avez-vous rencontrés en essayant de le faire ?"

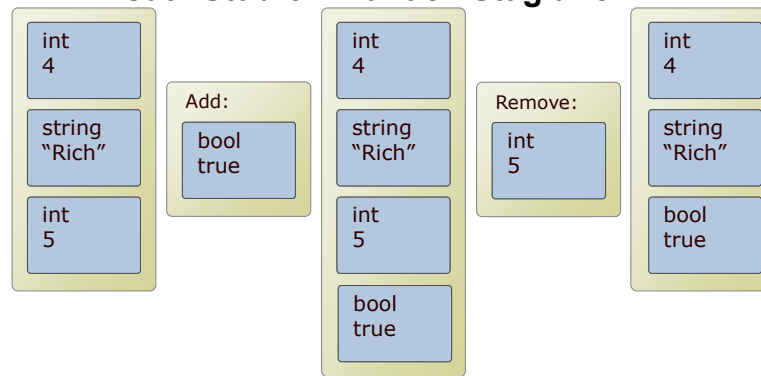
Formation : Les fondamentaux du développement .Net en C# sous Visual Studio - Manuel - Stagiaire

Vue d'ensemble

- Utilisation des collections
- Création et utilisation des types génériques
- Définir des interfaces génériques et comprendre la variance
- Utilisation de méthodes génériques et des délégués

- Définition
- Utilisation
- Itération
- Collections courantes
- Initialiseurs de collections

Formation : Les fondamentaux du développement .Net en C# sous Visual Studio - Manuel - Stagiaire



- Les éléments d'une collection sont référencés à travers le type System.Object
- Les collections gèrent l'espace automatiquement

Formation : Les fondamentaux du développement .Net en C# sous Visual Studio - Manuel - Stagiaire

Interface ICollection:
CopyTo
GetEnumerator
Count
Implémentée par toutes classes de collection

Interface IList:
Add
Remove

Implémentée par certaines collections

Les éléments sont stockés en tant qu'objet, nécessitant un cast pour les récupérer.
Add et Remove peuvent être remplacés par d'autres méthodes, voire accompagnées d'autres solutions

```
ArrayList list = new ArrayList();  
list.Add(6);  
list.Remove(6);  
list.RemoveAt(1);  
int temp = (int)list[0];
```

Formation : Les fondamentaux du développement .Net en C# sous Visual Studio - Manuel - Stagiaire

La boucle foreach récupère chaque élément les uns après les autres

```
foreach(<type> <control_variable> in <collection>)  
{  
    <foreach_statement_body>  
}
```

L'utilisation de la boucle foreach n'est possible que sur les classes implémentant la méthode GetEnumerator de l'interface IEnumerable

Il est également possible d'implémenter GetEnumerator via une méthode d'extension pour les classes dont le code n'est pas modifiable

```
ArrayList list = new ArrayList();  
list.Add(99);  
list.Add(10001);  
list.Add(25);  
...  
foreach (int i in list)  
{  
    Console.WriteLine(i);  
}  
// Sortie : 99  
//          10001  
//          25
```


Formation : Les fondamentaux du développement .Net en C# sous Visual Studio - Manuel - Stagiaire

Collections courantes

ArrayList

Collection non ordonnée, similaire à un tableau. Accès par indice

Queue

Collection FIFO avec méthodes Enqueue et Dequeue

Stack

Collection FILO avec méthodes Push et Pop

Hashtable

Collection de paires clé-valeur

SortedList

Collection de paires clé-valeur, triée sur la clés

Formation : Les fondamentaux du développement .Net en C# sous Visual Studio - Manuel - Stagiaire

Initialiseurs de collections

Il est possible d'utiliser la méthode Add pour ajouter des éléments...

```
ArrayList a1 = new ArrayList();
a1.Add("Value");
a1.Add("Another Value");
```

...ou encore utiliser les initialiseurs de collection, similaires aux initialiseurs d'objets

```
// person1 contient une référence de Person
ArrayList a12 = new ArrayList()
{
    person1,
    new Person() {Name="Tom", Age =31}
};
```

Il est également possible de tirer partie des initialiseurs d'indexeur

```
Hashtable data = new Hashtable() {
    ["A01"]=person1,
    ["A02"]=new Person() {Name="Tom", Age =31}
};
```



- Définition
- Compilation
- Définir un type générique
- Contraintes de généricité
- Collections génériques

Formation : Les fondamentaux du développement .Net en C# sous Visual Studio - Manuel - Stagiaire

- Un type générique est un type qui déclare un ou plusieurs paramètres de type
- Un paramètre de type est un paramètre comme les autres, à la différence qu'il permet d'indiquer un type et non une instance d'un type
- Les paramètres de type sont indiqués entre inférieur et supérieur

```
public class List<T>
```

Cet exemple présente la définition d'une classe nommée List acceptant un paramètre de type T. T est un type que l'on pourra utiliser dans la définition de List comme tout autre type

Formation : Les fondamentaux du développement .Net en C# sous Visual Studio - Manuel - Stagiaire

Compilation

List<T> est ici utilisé plusieurs fois avec des types différents

```
List<string> names = new List<string>();
names.Add("John");
...
string name = names[0];

List<List<string>> listOfLists = new List<List<string>>();
listOfLists.Add(names);
...
List<string> data = listOfLists[0];
```

Le compilateur génère une version fortement typée de la classe générique, en fonction des types passés en paramètre

```
public void Add(string item)
...
public void Add(List<string> item)
```

Ces classes et méthodes sont générés à la compilation et donc, vous ne pouvez les accéder directement dans votre code

Formation : Les fondamentaux du développement .Net en C# sous Visual Studio - Manuel - Stagiaire

Définir la classe en ajoutant derrière son nom les paramètres de type

```
class PrintableCollection<TItem>
{
    TItem[] data;
    int index;
    ...
    public void Insert(TItem item)
    {
        ...
        data[index] = item;
        ...
    }
}
```

Le paramètre de type est utilisé tel un alias dans tout le code, que ce soit pour des champs, des propriétés, des paramètres de méthode...



Formation : Les fondamentaux du développement .Net en C# sous Visual Studio - Manuel - Stagiaire

Contraintes de générique

Contraintes	Description
where T: struct	T doit être un type valeur
where T : class	T doit être un type référence
where T : new()	T doit avoir un constructeur public par défaut
where T : <classe de base>	T doit hériter de la classe indiquée
where T : <interface>	T doit implémenter l'interface indiquée
where T : U	T doit hériter de U

Collections génériques

List<T>
Equivalent de ArrayList

Queue<T>
Equivalent de Queue

Stack<T>
Equivalent de Stack

Dictionary<Tkey,Tvalue>
Equivalent de Hashtable

Mais aussi: LinkedList<T>, SortedList<Tkey,Tvalue>,
SortedDictionary<Tkey,Tvalue>, ...

- Interfaces génériques
- Problème de l'invariance
- Covariance et contravariance

Interfaces génériques

- Une interface générique peut être implémentée

- Par une classe typée

```
interface IPrinter<T> {  
    ...  
}  
  
class Printer: IPrinter<WordDocument> {  
    ...  
}
```

- Par une classe générique

```
interface IPrinter<T> {  
    ...  
}  
  
class Printer<T>: IPrinter<T> {  
    ...  
}
```

- L'invariance est une sécurité empêchant de caster des paramètres de type vers d'autres types de la même hiérarchie d'héritage
- Les classes génériques sont toujours invariantes
- Les interfaces génériques sont invariantes par défaut
- Covariance et contravariance permettent de changer le comportement des interfaces génériques

Définition de l'invariance (suite)

```
string myString = "Hello";
object myObject = myString;
```

```
interface IData<T>
{
    void SetData(T data);
    T GetData();
}

class Truc<T> : IData <T>
{
    private T storedData;

    void IData<T>.SetData(T data)
    {
        this.storedData = data;
    }

    T IData<T>.GetData()
    {
        return this.storedData;
    }
}
```

```
Truc<string> strTruc = new Truc<string>();
IData<string> dataStr = strTruc;
```

```
dataStr.SetData("Hello");
Console.WriteLine("Stored value is {0}",
    dataStr.GetData());
```

```
// Ne compile pas
IData<object> dataObj = strTruc;
```

```
// Compile mais Erreur d'exécution
IData<object> dataObj =
    (IData<object>) strTruc;
```

```
// Ce qu'empêche l'invariance
IData<object> dataObj =
    (IData<object>) strTruc;
Window myWindow = new Window();
objData.SetData(myWindow);
```

Covariance et Contravariance

- Mécanismes permettant d'autoriser les conversions entre interface générique et classe générique
- Utilisation des mots-clés *out* et *in* dans les déclarations d'interface

Bonne pratique:

Implémenter les interfaces génériques en tant que classe concrète!

- Définir une méthode générique
- Utiliser une méthode générique
- Utilisation des délégués génériques

Formation : Les fondamentaux du développement .Net en C# sous Visual Studio - Manuel - Stagiaire

Définir une méthode et spécifier le ou les types à utiliser après le nom de la méthode entre <>

```
ResultType MyMethod<Parameter1Type, ResultType>(Parameter1Type param1)
{
    where ResultType : new()
    {
        ResultType result = new ResultType();
        return result;
    }
}
```

Si besoin, indiquer les contraintes à appliquer aux paramètres de type

Utiliser les paramètres de type là où cela est nécessaire

Créer une méthode générique permet d'éviter la multiplication des surcharges pour prendre en compte différents types



Formation : Les fondamentaux du développement .Net en C# sous Visual Studio - Manuel - Stagiaire

Utiliser une méthode générique

Lors de l'appel de la méthode, spécifiez le type à utiliser

```
T PerformUpdate<T>(T input)
{
    T output = ...;
    return output;
}
...

string result = PerformUpdate<string>("Test");
int result2 = PerformUpdate<int>(1);
```

Le compilateur génère une version de la méthode pour chaque type de paramètre

```
string PerformUpdate(string input)
{
    string output = ...;
    return output;
}

int PerformUpdate(int input)
{
    int output = ...;
    return output;
}
```



Ces méthodes ne sont pas accessibles dans votre code

Visual Studio - Manuel - Stagiaire

Action<T>

Le délégué générique Action permet de ne pas avoir à déclarer la signature d'un délégué sans valeur de retour

Func<T, TResult>

Le délégué générique Func<Tresult> permet de ne pas avoir à déclarer la signature d'un délégué à retour de valeur, le paramètre de type TResult représentant la type de cette valeur

.NET inclut des surcharges des délégués Action et Func jusqu'à 16 paramètres



Atelier 01

Activité
20 min

- Exercice 1: Utilisation d'une collection



A retenir

1. **Les Collections** : Les collections sont essentielles pour stocker et gérer des groupes d'objets. Elles offrent une manière flexible et efficace de travailler avec des ensembles de données dans des applications.
2. **L'importance des types génériques** : Les types génériques permettent de créer des structures de données et des méthodes flexibles tout en conservant la sécurité du typage, réduisant ainsi les risques d'erreurs.
3. **La puissance de la réutilisabilité** : Grâce aux types génériques, vous pouvez concevoir des classes, interfaces et méthodes qui peuvent être réutilisées avec n'importe quel type de données, offrant une adaptabilité énorme.
4. **Sécurité du typage avec les génériques** : Utiliser des génériques garantit que vous travaillez toujours avec le bon type de données, ce qui minimise les erreurs de typage et renforce la robustesse de votre code.
5. **Interfaces génériques** : Les interfaces génériques définissent des contrats pour des classes qui travailleront avec des types spécifiés au moment de l'implémentation, ajoutant une couche supplémentaire d'abstraction et de flexibilité.
6. **Comprendre la variance** : En .NET, la variance permet d'assigner un type générique moins dérivé (par exemple, une liste de base) à un type générique plus dérivé (par exemple, une liste dérivée). Cela offre plus de flexibilité tout en conservant la sécurité du typage.
7. **Méthodes génériques et délégués** : Les méthodes génériques vous permettent de définir une méthode une fois et de l'utiliser pour une variété de types de données. Les délégués génériques étendent cette capacité aux méthodes de rappel.
8. **Performance des génériques** : Les génériques offrent souvent de meilleures performances que les structures non génériques car ils éliminent le besoin de conversions de type coûteuses.
9. **Extensibilité des collections et des génériques** : En comprenant et en maîtrisant les collections et les génériques, vous pouvez étendre leurs fonctionnalités pour répondre aux besoins spécifiques de votre application.
10. **Les génériques renforcent les bonnes pratiques** : En encourageant la réutilisabilité, la sécurité du typage, et l'abstraction, les génériques favorisent les bonnes pratiques de programmation et aident à créer un code propre et maintenable.

Chapitre 13 - Programmation asynchrone et personnalisation du code



Plan de la formation

- 1 - Introduction à C# et .Net
- 2 - Structure de programmation C#
- 3 - Déclaration et appel de méthodes
- 4 - Gestion d'exceptions
- 5 - Lire et écrire dans des fichiers
- 6 - Création de nouveaux types de données
- 7 - Encapsulation de données et de méthodes
- 8 - Héritage de classes et implémentation
- 9 - Gestion de la durée de vie des objets
- 10 - Encapsulation avancée
- 11 - Découplage de méthodes et gestion
- 12 - Utilisation des collections et constantes
- 13 - Programmation asynchrone et personnalisation du code**
- 14 - Utilisation de LINQ pour interroger des données
- 15 - Développement dirigé par les Tests

Objectifs :

- Comprendre la Programmation Asynchrone :
- Maîtriser les Mécanismes Asynchrones
- Personnaliser le Comportement des Applications
- Gérer les Erreurs en Asynchrone
- Optimiser les Performances des Applications

Objectifs :

1. Comprendre la Programmation Asynchrone : Démystifier les concepts et les pratiques de la programmation asynchrone, et montrer comment elle peut améliorer la réactivité et l'efficacité des applications.

2. Maîtriser les Mécanismes Asynchrones : Apprendre à utiliser les mécanismes asynchrones tels que les mots-clés `async` et `await`, et comprendre comment ils fonctionnent sous le capot.

3. Personnaliser le Comportement des Applications : Montrer comment personnaliser le comportement des applications en utilisant diverses techniques de programmation avancées.

4. Gérer les Erreurs en Asynchrone : Apprendre à gérer correctement les erreurs qui peuvent survenir lors de l'exécution asynchrone.

5. Optimiser les Performances des Applications : Fournir des stratégies pour optimiser les performances des applications en utilisant la programmation asynchrone et la personnalisation du code.

Positionnement :

Ce chapitre se situe à un niveau avancé du parcours d'apprentissage, s'appuyant sur les connaissances acquises dans les chapitres précédents sur les bases du langage, la manipulation des types de données, et la gestion des événements. Il vise à élever les compétences des apprenants à un niveau supérieur en leur enseignant comment gérer

des opérations concurrentes et personnaliser le comportement de leurs applications pour répondre à des besoins spécifiques.

Sujets Clés qui Seront Couverts :

1.Principes de la Programmation Asynchrone : Introduction aux concepts de base et aux principes qui sous-tendent la programmation asynchrone.

2.Utilisation des Mots-clés Async et Await : Comment et quand utiliser async et await, et comprendre leur impact sur le flux d'exécution.

3.Gestion des Tâches Asynchrones : Techniques pour gérer et coordonner les tâches asynchrones, y compris la gestion des erreurs et des exceptions.

4.Patterns de Conception Asynchrone : Exploration de divers patterns de conception qui facilitent la programmation asynchrone et améliorent la clarté du code.

5.Personnalisation du Code : Techniques pour personnaliser le comportement des applications, y compris l'utilisation de l'injection de dépendances, des patterns de conception, et d'autres pratiques avancées.

6.Optimisation des Performances : Stratégies et meilleures pratiques pour optimiser les performances des applications en utilisant la programmation asynchrone.

7.Gestion des Ressources : Comment gérer efficacement les ressources dans un environnement asynchrone, y compris la gestion de la mémoire et des threads.

8.Tests et Débogage : Stratégies pour tester et déboguer des applications qui utilisent intensivement la programmation asynchrone.



Commençons par échanger

"Avez-vous déjà rencontré des situations, dans votre utilisation quotidienne des applications, où certaines tâches semblaient s'exécuter en arrière-plan sans interrompre votre travail ? Pouvez-vous donner des exemples ?"

"Pensez à vos applications préférées. Quels sont les aspects que vous avez pu personnaliser pour répondre à vos besoins spécifiques ? Estimez-vous que cette personnalisation améliore votre expérience utilisateur ?"

"Avez-vous déjà abandonné l'utilisation d'une application ou d'un site web parce qu'elle était trop lente ou ne répondait pas ? Comment pensez-vous qu'une meilleure gestion de tâches concurrentes aurait pu améliorer cette expérience ?"

"Savez-vous quelle est la différence entre une opération qui est exécutée de manière concurrente et une opération qui est exécutée en parallèle ? Pourquoi est-ce pertinent pour le développement d'applications modernes ?"

"Selon vous, est-il plus complexe de tester et de déboguer un code qui s'exécute de manière asynchrone par rapport à un code synchrone ? Pourquoi ou pourquoi pas ?"

1. Expérience Asynchrone : "Avez-vous déjà rencontré des situations, dans votre utilisation quotidienne des applications, où certaines tâches semblaient s'exécuter en arrière-plan sans interrompre votre travail ? Pouvez-vous donner des exemples ?"

2. Personnalisation vs. Standardisation : "Pensez à vos applications préférées. Quels sont les aspects que vous avez pu personnaliser pour répondre à vos besoins spécifiques ? Estimez-vous que cette personnalisation améliore votre expérience utilisateur ?"

3. Importance de la Performance : "Avez-vous déjà abandonné l'utilisation d'une application ou d'un site web parce qu'elle était trop lente ou ne répondait pas ? Comment pensez-vous qu'une meilleure gestion de tâches concurrentes aurait pu améliorer cette expérience ?"

4. Concurrence vs. Parallélisme : "Savez-vous quelle est la différence entre une opération qui est exécutée de manière concurrente et une opération qui est exécutée en parallèle ? Pourquoi est-ce pertinent pour le développement d'applications modernes ?"

5. Tests et Débogage : "Selon vous, est-il plus complexe de tester et de déboguer un code qui s'exécute de manière asynchrone par rapport à un code synchrone ? Pourquoi ou pourquoi pas ?"

- Programmation asynchrone
- Création d'une classe de collection personnalisée
- Enregistrements
- Simplification du code

- Bénéfice
- Modèles de programmation asynchrone
- Utilisation de async et await

Bénéfice

- En programmation traditionnel, une tâche longue bloque l'exécution (modèle synchrone)
- En programmation asynchrone, les tâches longues sont exécutées sur un thread parallèle et ne bloquent pas l'exécution
- Evite que l'application apparaisse « plantée » pour l'utilisateur

Modèles de programmation asynchrone

- Task-based Asynchronous Pattern (TAP)
 - Une méthode unique qui retourne une instance de *Task* ou *Task<T>*
 - Utilisation des mots-clés *async* et *await*
- Event-based Asynchronous Pattern (EAP)
 - Une méthode (*xxxAsync*) et un ou plusieurs événements
- Asynchronous Programming Model (APM)
 - Une méthode d'initialisation *Beginxxx*
 - Une méthode de récupération *Endxxx*

Seule la TAP est à présent recommandée

Utilisation de async et await

- Une méthode asynchrone (*awaitable*) est une méthode marquée par le mot-clé **async** et pour laquelle le type de retour est transformé : *void* devient *Task*, *T* devient *Task<T>*
- Une méthode asynchrone a pour but d'appeler une autre méthode asynchrone avec le mot-clé **await**
- L'appel **await** met « en pause » la méthode principale, qui se terminera au retour de la méthode asynchrone

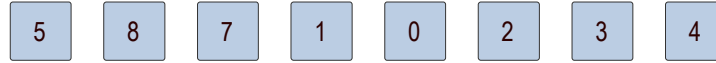
```
private readonly HttpClient _httpClient = new HttpClient();  
[HttpGet, Route("DotNetCount")]  
public async Task<int> GetDotNetCount()  
{  
    var html = await _httpClient.GetStringAsync("https://monsite.org");  
    return Regex.Matches(html, @"\.NET").Count;  
}
```

- Pourquoi créer sa propre collection?
- Interfaces de collection génériques de .NET
- Exemple de collection : File à double entrée
- Enumération

Formation : Les fondamentaux du développement .Net en C# sous Visual Studio - Manuel Stagiaire

Pourquoi créer sa propre collection ?

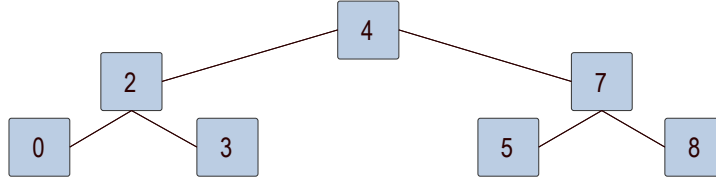
Liste séquentielle -> List<T>



Liste triée -> SortedList<T>



Arbre binaire -> ? -> Il faut créer une nouvelle collection



Formation : Les fondamentaux du développement .Net en C# sous Visual Studio - Manuel Stagiaire

Interfaces de collection génériques de .NET

ICollection<T> :
IEnumerable<T>

Méthodes :

- Add
- Clear
- Contains
- CopyTo
- Remove

Propriétés :

- Count
- IsReadOnly

IList<T> :
ICollection<T>

Méthodes :

- IndexOf
- Insert
- RemoveAt

Propriétés :

- Item

IDictionary<TKey, TValue> :
ICollection<KeyValuePair
<TKey, TValue>>

Méthodes :

- Add
- ContainsKey
- GetEnumerator
- Remove
- TryGetValue

Propriétés :

- Item
- Keys
- Values

Formation : Les fondamentaux du développement .Net en C# sous Visual Studio - Manuel Stagiaire

Exemple de Collection : File à double entrée

```
class DoubleEndedQueue<T> :  
ICollection<T>, IList<T>  
{  
    private List<T> items;  
  
    // Type specific methods.  
    public DoubleEndedQueue() {...}  
  
    public EnqueueItemAtStart() {...}  
  
    public DequeueItemFromStart()  
    {...}  
  
    public EnqueueItemAtEnd() {...}  
  
    public DequeueItemFromEnd() {...}  
  
    // Interface methods.  
    public void Add(T item) {...}  
  
    public void Clear() {...}  
  
    public bool Contains(T item) {...}  
  
    public void CopyTo(T[] array,  
        int arrayIndex) {...}
```

```
    public int Count  
    { get {...} }  
  
    public bool IsReadOnly  
    { get {...} }  
  
    public bool Remove(T item) {...}  
  
    public IEnumerator<T>  
        GetEnumerator() {...}  
  
    IEnumerator  
        IEnumerable.GetEnumerator() {...}  
  
    public int IndexOf(T item) {...}  
  
    public void Insert  
        (int index, T item) {...}  
  
    public void RemoveAt(int index)  
    {...}  
  
    public T this[int index]  
    {  
        get {...}  
        set {...}  
    }  
}
```

Le code complet est fourni en annexe de votre support.

Enumération

- L'utilisation de **foreach** nécessite la méthode **GetEnumerator** (interface **IEnumerable<T>**)
 - Cette méthode retourne une instance d'une classe **IEnumerator<T>**
- Il est souvent plus simple de s'appuyer sur un objet interne qui implémente déjà **IEnumerable**

```
private List<T> data;  
public IEnumerator<T> GetEnumerator()  
{  
    return data.GetEnumerator();  
}
```

Enumération (suite)

Il est également possible de produire ses propres fonctions ou propriétés **IEnumerable** grâce au mot-clé **yield**

```
// data est une collection quelconque
public IEnumerable<T> Backwards
{
    get
    {
        for (int i = data.Count - 1; i >= 0; i--)
        {
            yield return data[i];
        }
    }
}
```

```
foreach (var item in moObjet.Backwards){
    ...
}
```

- Définition
- Syntaxe
- Utilisation

Définition

- Type référence, optimisé pour les tests d'égalité
- Facilite la mise en œuvre de type référence immuable
- Deux enregistrements sont égaux
 - s'ils sont du même type
 - si toutes leurs propriétés sont égales
- Le compilateur génère:
 - Les méthodes de comparaison basées sur les propriétés (Equals, ==, !=)
 - La surcharge de GetHashCode()
 - La surcharge de ToString()

Syntaxe

- Syntaxe abrégée

```
public record Ami(string Nom, string Prenom);
```

- Syntaxe complète

```
public record Ami
{
    public string Nom{ get; init;}
    public string Prenom{ get; init;}

    public Ami(string nom, string prenom) =>
        (Nom, Prenom) = (nom, prenom);
}
```

Rien n'empêche de remplacer l'accesseur *init* par l'accesseur *set*, mais l'enregistrement n'est plus dans ce cas immuable.

Utilisation

Instanciation d'enregistrement

```
Ami a = new Ami("Doe", "John");
Ami b = new("Doe", "Jane");
```

Comparaison d'enregistrement

```
Console.WriteLine( a.Equals(b) );
Console.WriteLine( a==b );
```

Recopie d'enregistrement

```
Ami c = a with { Nom = "Toto" };
Console.WriteLine(c); // Ami { Nom = Toto, Prenom = John }
```

- Instruction de niveau supérieur
- Omission de type
- Abandon de paramètres

Instruction de niveau supérieur

Un seul fichier de l'application peut utiliser cette simplification qui élimine le code inutile

```
using System;
namespace HelloWorld
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello World!");
        }
    }
}
```



```
using System;
Console.WriteLine("Hello World!");
```


Omission de type

- Consiste à omettre le nom du type utilisé dans une expression *new*
- Le nom du type est déterminé par le contexte

```
private List<WeatherObservation> _observations = new();
```

```
public WeatherForecast ForecastFor(DateTime forecastDate,  
    WeatherForecastOptions options){ ... }  
...  
var forecast = station.ForecastFor(DateTime.Now.AddDays(2), new());
```

```
WeatherStation station = new() { Location = "Seattle, WA" };
```

Abandon de paramètres

- Ignorer deux paramètres ou plus passés à une expression Lambda
- Utile pour les gestionnaire d'événements

```
Func<int, int, int> constant = (_, _) => 42;
```

À des fins de compatibilité descendante, si un seul paramètre d'entrée est nommé _ dans l'expression Lambda, alors, _ est traité comme le nom de ce paramètre.

Atelier 01



Activité
20 min

- Exercice 1: Programmation asynchrone
- Exercice 2: Implémentation de l'interface IEnumerable
- Exercice 3: Utilisation de la syntaxe de niveau supérieur



A retenir

1. **Nature Asynchrone** : "La programmation asynchrone permet d'exécuter des tâches de manière non séquentielle, offrant ainsi une meilleure réactivité et performance pour les utilisateurs et les systèmes."
2. **Avantages Asynchrones** : "En utilisant l'asynchronicité, les applications peuvent traiter plusieurs tâches simultanément sans se bloquer, améliorant ainsi l'expérience utilisateur et l'utilisation des ressources."
3. **Collections Personnalisées** : "Créer une classe de collection personnalisée permet d'adapter et d'optimiser le stockage et l'accès aux données selon les besoins spécifiques de votre application."
4. **Un Enregistrement, ce n'est pas qu'une Classe** : "Les enregistrements offrent une manière de représenter des données immuables sans toute la lourdeur associée aux classes. Ils sont parfaits pour représenter des objets de données légers."
5. **Simplicité et Expressivité** : "Simplifier le code ne signifie pas seulement le rendre plus court, mais aussi le rendre plus expressif, lisible et maintenable."
6. **Réduire la Complexité** : "Une des clés de la maintenance du code est de réduire la complexité. Les techniques de simplification du code aident à atteindre cet objectif."
7. **Testabilité** : "Un code simplifié est souvent plus facile à tester, car il a tendance à avoir moins de cas limites et de dépendances."
8. **Structure des Enregistrements** : "Contrairement aux classes, les enregistrements sont conçus pour être immuables et offrent une sémantique de valeur, ce qui les rend idéaux pour représenter des données qui ne changent pas."
9. **Evolution des Collections** : "Même si .NET offre une panoplie de collections, parfois les besoins spécifiques exigent la création d'une collection personnalisée pour optimiser les performances ou le comportement."
10. **L'Asynchrone est Partout** : "À l'ère des applications modernes, la programmation asynchrone n'est plus une option. Que ce soit pour les interfaces utilisateur, les appels réseau ou l'accès aux bases de données, l'asynchronicité est essentielle pour des applications réactives."

Chapitre 14 - Utilisation de LINQ pour interroger des données



Plan de la formation

- 1 - Introduction à C# et .Net
- 2 - Structure de programmation C#
- 3 - Déclaration et appel de méthodes
- 4 - Gestion d'exceptions
- 5 - Lire et écrire dans des fichiers
- 6 - Création de nouveaux types de données
- 7 - Encapsulation de données et de méthodes
- 8 - Héritage de classes et implémentation d'interfaces
- 9 - Gestion de la durée de vie des objets et contrôle des ressources
- 10 - Encapsulation avancée
- 11 - Découplage de méthodes et gestion des ressources
- 12 - Utilisation des collections et des interfaces
- 13 - Programmation asynchrone et parallèle
- 14 - Utilisation de LINQ pour interroger des données**
- 15 - Développement dirigé par les Tests

Objectifs :

- Maîtriser les bases de LINQ
- Savoir construire des requêtes LINQ
- Comprendre l'interaction de LINQ avec d'autres technologies
- Optimiser les requêtes LINQ

Objectifs :

- 1.Maîtriser les bases de LINQ** : Comprendre le rôle et les avantages de LINQ dans le cadre de la manipulation et de la requête de données.
- 2.Savoir construire des requêtes LINQ** : Apprendre à écrire des requêtes LINQ efficaces pour extraire, filtrer, et transformer des données à partir de sources diverses.
- 3.Comprendre l'interaction de LINQ avec d'autres structures** : Étudier comment LINQ interagit avec les collections, les bases de données, et d'autres structures de données.
- 4.Optimiser les requêtes LINQ** : Évaluer les performances et optimiser les requêtes pour garantir une extraction de données rapide et efficace.

Positionnement :

LINQ, ou "Language Integrated Query", représente une avancée majeure dans la manière dont les développeurs .NET peuvent manipuler et interroger des données. Plutôt que de jongler entre différents paradigmes pour différentes sources de données, LINQ offre une approche unifiée, puissante et expressive. Ce chapitre se positionne comme une étape essentielle pour tout développeur souhaitant maîtriser l'art de la manipulation de données en C#.

Sujets clés couverts :

1.Introduction à LINQ :

1. Présentation de LINQ et son rôle dans le framework .NET.
2. Avantages de l'utilisation de LINQ par rapport aux méthodes traditionnelles.

2. Syntaxe des requêtes LINQ :

1. Comprendre la différence entre la syntaxe de requête et la syntaxe de méthode.
2. Exploration des mots-clés tels que from, select, where.

3.Types de sources de données :

1. Utilisation de LINQ avec des collections en mémoire, tels que les listes et les tableaux.
2. Interrogation de bases de données avec LINQ to SQL ou Entity Framework.
3. Requête de données XML avec LINQ to XML.

4.Opérations avancées avec LINQ :

1. Tri, groupement, et agrégation des données.
2. Joins, group joins, et d'autres techniques pour combiner des données.

5.Optimisation et performances :

1. Comprendre le comportement "lazy" de LINQ.
2. Techniques pour améliorer les performances des requêtes LINQ.



Commençons par échanger

"Pouvez-vous partager une expérience où vous avez dû filtrer ou trier des données dans une application ou un projet précédent ? Quels outils ou méthodes avez-vous utilisés ?"

"Quand vous travaillez avec des bases de données, comment procédez-vous généralement pour extraire des informations spécifiques ? Y a-t-il des défis auxquels vous êtes fréquemment confrontés ?"

"Dans quelles situations trouvez-vous qu'il serait plus efficace d'intégrer des requêtes directement dans le code plutôt que de les externaliser ?"

"Lors de la manipulation de listes ou de collections en C#, avez-vous déjà ressenti le besoin d'avoir des outils plus expressifs ou flexibles pour interroger ces données ? Pouvez-vous donner un exemple ?"

"Avez-vous déjà été confronté à des problèmes de performances lors de la récupération ou du traitement de grandes quantités de données ? Comment avez-vous abordé ces défis ?"

1.Question sur la manipulation des données : "Pouvez-vous partager une expérience où vous avez dû filtrer ou trier des données dans une application ou un projet précédent ? Quels outils ou méthodes avez-vous utilisés ?"

2.Question sur les bases de données : "Quand vous travaillez avec des bases de données, comment procédez-vous généralement pour extraire des informations spécifiques ? Y a-t-il des défis auxquels vous êtes fréquemment confrontés ?"

3.Question sur l'intégration du code : "Dans quelles situations trouvez-vous qu'il serait plus efficace d'intégrer des requêtes directement dans le code plutôt que de les externaliser ?"

4.Question sur les collections : "Lors de la manipulation de listes ou de collections en C#, avez-vous déjà ressenti le besoin d'avoir des outils plus expressifs ou flexibles pour interroger ces données ? Pouvez-vous donner un exemple ?"

5.Question sur la performance : "Avez-vous déjà été confronté à des problèmes de performances lors de la récupération ou du traitement de grandes quantités de données ? Comment avez-vous abordé ces défis ?"

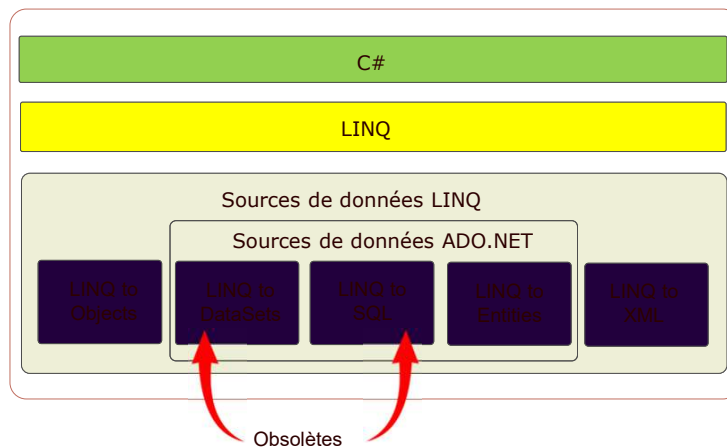
Vue d'ensemble

- Utilisation des méthodes d'extension LINQ et des opérateurs de requête
- Construction de requêtes et d'expressions LINQ dynamiques

Leçon 1: Utilisation des méthodes d'extension LINQ et des opérateurs de requête

- Pourquoi LINQ?
- La face cachée de LINQ
- Interrogation de données
- Filtrage
- Tri
- Regroupement et agrégations
- Jointure
- Opérateurs de requête
- Modèles d'évaluation des requêtes

Pourquoi LINQ?



La face cachée de LINQ

LINQ est avant tout basé sur les méthodes d'extension et les expressions Lambda

LINQ tire partie également du typage implicite...

```
var myString = "Hello World!"; // myString est typé String
var data = 5;                  // data est typé int
```

...et des classes anonymes

```
var obj = new {Name="Doe",FirstName="John",Age=30};
// obj est une instance d'un type anonyme
// qui expose 3 propriétés: Name, FirstName et Age
```

Interrogation de données

Manipuler des données nécessite de pouvoir choisir les données

```
IEnumerable<Customer> customers = new[]
{
    new Customer{ FirstName = "...", LastName="...", Age = 41},
    ...
};
```

```
List<string> customerLastNames = new List<string>();
foreach (Customer customer in customers)
{
    customerLastNames.Add(customer.LastName);
}
```

Approche traditionnelle à l'aide d'un foreach

...

```
IEnumerable<string> customerLastNames =
    customers.Select(cust => cust.LastName);
```

Approche LINQ avec la méthode d'extension Select

La méthode Select est une méthode d'extension des types génériques IQueryable<T> et IEnumerable<T>

Filtrage

La méthode d'extension Where permet de filtrer les résultats

```
var customerLastNames =  
    customers.Where(cust => cust.Age > 25).  
    Select(cust => cust.LastName);
```

Tous les noms de
clients d'âge
supérieur à 25

La méthode Where filtre et la méthode Select projette comme le font les mots clés SELECT et WHERE du langage SQL



Attention:
L'ordre des méthodes d'extension est important

Tri

Le tri des données est assuré par les méthodes `OrderBy`, `OrderByDescending`, `ThenBy` et `ThenByDescending`

- `OrderBy` et `OrderByDescending` assurent les tris simples

```
var sortedCustomers =
    customers.OrderBy(cust => cust.FirstName);
```

Tri par prénom
croissant

- `ThenBy` et `ThenByDescending` assurent les tris complexes

```
var sortedCustomers =
    customers.OrderBy(cust => cust.FirstName).
    ThenBy(cust => cust.Age);
```

Tri par prénom puis
par âge

Regroupement et agrégations

Des calculs d'agrégations sont possibles sur une collection

```
Console.WriteLine(
    "Count:{0}\t\tAverage age:{1}\t\tLowest:{2}\t\tHighest:{3}",
    customers.Count(), customers.Sum(cust => cust.Age),
    customers.Average(cust => cust.Age),
    customers.Min(cust => cust.Age),
    customers.Max(cust => cust.Age));
```

La génération de groupe est possible par la méthode GroupBy

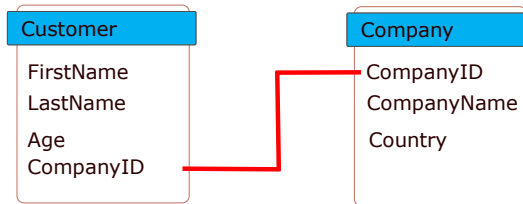
```
var customersGroupedByAgeRange = customers.GroupBy( ... );
```

Il est également possible d'éliminer les doublons

```
Console.WriteLine("{0}",
    customers.Select(cust => cust.Age).Distinct().Count());
```


Jointure

LINQ fournit sa méthode d'extension Join afin de joindre des données de plusieurs sources distinctes en une seule requête



```
var customersAndCompanies = customers.Join(  
    companies,  
    cust => cust.CompanyID,  
    comp => comp.CompanyID,  
    (cust, comp) =>  
        new { cust.FirstName, cust.LastName, comp.Country});
```

Opérateurs de requête

Les opérateurs de requête LINQ fournissent une syntaxe abrégée très proche de celle du langage SQL

```
IEnumerable<string> customerLastNames =  
    customers.Select(cust => cust.LastName);
```

*Approche par méthode
d'extension*

```
IEnumerable<string> customerLastNames =  
    from cust in customers  
    select cust.LastName;
```

*Approche par
opérateur de requête*

Il existe un opérateur de requête pour chaque méthode d'extension

```
var customersOver25 = from cust in customers  
    where cust.Age > 25 select cust;  
...  
var sortedCustomers = from cust in customers  
    orderby cust.FirstName select cust;  
...  
var customersGroupedByAge = from cust in customers  
    group cust by cust.Age;
```

Modèles d'évaluation des requêtes

- La collection définie par votre requête LINQ n'est pas créée au moment où celle-ci est définie

- Cette stratégie est appelée *évaluation différée*

```
var usCompanies = from a in companies
                  where String.Equals(a.Country, "United States")
                  select a.CompanyName;
```

Données non récupérées

```
foreach (string name in usCompanies)
{
    Console.WriteLine(name);
}
```

Données récupérées

- Il est possible de forcer l'évaluation par l'utilisation de méthodes telles que ToList ou ToArray. On parle alors de *matérialisation* des données, car une copie des données est créée dans le programme

Leçon 2: Construction de requêtes et d'expressions LINQ dynamiques

- Requêtes statiques vs Requêtes dynamiques
- Réflexion
- Arbre d'expression
- Expression
- Compilation et exécution

Requêtes statiques vs Requêtes dynamiques

Les requêtes statiques sont puissantes mais nécessitent de connaître la requête au moment de la compilation

Les requêtes statiques:

- Vous oblige à écrire tous les cas possibles
- Nécessite parfois d'écrire du code inutile qui ne sera jamais exécuté
- Masque parfois des erreurs en utilisant des instructions conditionnelles complexes



Les requêtes dynamiques sont encore plus puissantes car elles sont construites au moment de l'exécution

Les requêtes dynamiques permettent:

- D'écrire une seule requête basée sur des paramètres fournis
- De produire une requête en fonction de l'utilisateur
- De n'écrire que le code nécessaire
- De réduire les erreurs potentielles



Réflexion

Ensemble de fonctionnalités pour analyser du code par le code

Nécessaire pour accéder aux membres des objets de manière dynamique

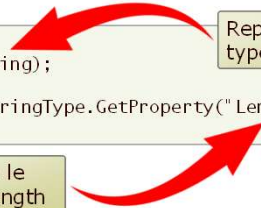
- La classe Type représente le type d'un objet
- La classe MemberInfo représente un membre

```
Type stringType = typeof(string);
```

```
MemberInfo stringLength = stringType.GetProperty("Length");
```

Représente le
type string

Représente le
membre Length

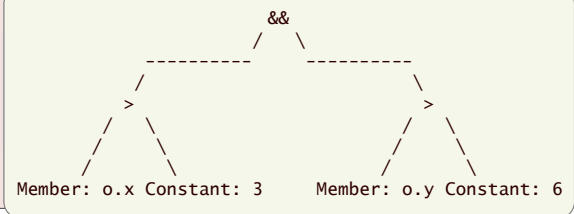


Arbre d'expression

- Les arbres d'expression contiennent plusieurs expressions qui une fois évaluées, formeront la requête dynamique
- Les arbres d'expression fournissent une syntaxe flexible pour développer des expressions Lambda

Pendant la construction de l'arbre, chaque expression est décomposée en éléments individuels

x > 3 && y > 6



Expression

La classe Expression

Fournit des méthodes statiques pour créer les autres objets d'expression

Les classes utiles

BinaryExpression, ConstantExpression, MemberExpression, UnaryExpression...
Expression<TDelegate> qui représente une expression Lambda

`x => x > 2` // expression Lambda à générer

```
Expression<Func<int, bool>> lambda = null;
ParameterExpression param = Expression.Parameter(typeof(int), "x");
ConstantExpression two = Expression.Constant(2, typeof(int));
BinaryExpression body = Expression.GreaterThan(param, two);
lambda = Expression.Lambda<Func<int, bool>>(body, param);
Console.WriteLine(lambda.ToString());
```


Compilation et exécution

La méthode d'instance Compile de la classe Expression<TDelegate> assure la compilation de l'arbre d'expression Lambda en expression Lambda exécutable

```
Expression<Func<int, bool>> expressionTree =
    Expression.Lambda<Func<int, bool>>(expression, parameter);

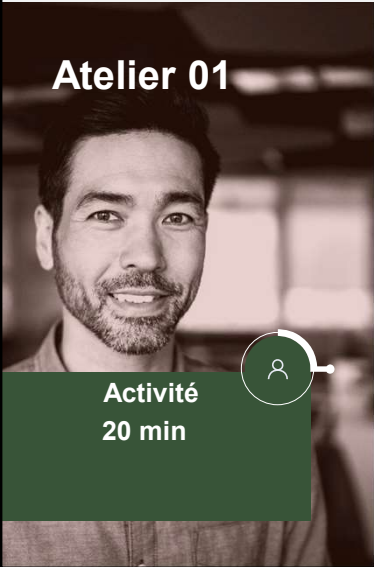
Func<int, bool> myDelegate = null;
myDelegate += expressionTree.Compile();
```

Le résultat de la méthode Compile peut être passé directement en argument à une méthode d'extension LINQ

```
var bankAccounts = accounts.Where(expressionTree.Compile());
```

Il est également possible de tester l'invocation de l'expression Lambda par la méthode DynamicInvoke de l'objet TDelegate retourné par la méthode Compile

```
bool response = expressionTree.Compile().DynamicInvoke(1);
```



Atelier 01

Activité
20 min

- Exercice 1: Utilisation de requêtes LINQ



A retenir

1. **Universalité de LINQ** : LINQ (Language Integrated Query) est une fonctionnalité puissante de C# qui permet d'interroger tout type de données, des collections aux bases de données, en passant par XML.
2. **Méthodes d'extension** : Les méthodes d'extension LINQ comme Where(), Select(), GroupBy(), etc., offrent une manière fluide et intuitive de construire des requêtes sur des collections.
3. **Syntaxe** : Avec LINQ, on peut utiliser une syntaxe proche du SQL directement en C#, ce qui rend les requêtes plus lisibles et expressives.
4. **Types Anonymes** : LINQ permet de créer des types anonymes pour structurer des résultats temporaires sans avoir besoin de définir des classes dédiées.
5. **Requêtes Différées** : Les requêtes LINQ sont évaluées de manière "différée", ce qui signifie qu'elles ne sont exécutées que lorsqu'on accède réellement aux données. Cela optimise les performances en évitant des opérations inutiles.
6. **Requêtes Dynamiques** : LINQ offre la possibilité de construire des requêtes dynamiquement, permettant une flexibilité accrue lors de l'interrogation des données.
7. **Sûreté de type** : Grâce à LINQ, on bénéficie de la vérification de type au moment de la compilation, réduisant les erreurs potentielles lors de l'exécution.
8. **Intégration avec d'autres sources de données** : Au-delà des collections en mémoire, LINQ peut être utilisé avec des bases de données (comme avec Entity Framework) ou des documents XML.
9. **Expressivité** : LINQ simplifie considérablement la construction d'opérations complexes sur les données, comme les jointures, les groupements ou les agrégations.
10. **Optimisation** : Bien que LINQ facilite l'interrogation des données, il est crucial de comprendre les implications en termes de performances et d'optimiser les requêtes lorsque cela est nécessaire.

Chapitre 15 - Utilisation de LINQ pour interroger des données



Plan de la formation

- 1 - Introduction à C# et .Net
- 2 - Structure de programmation C#
- 3 - Déclaration et appel de méthodes
- 4 - Gestion d'exceptions
- 5 - Lire et écrire dans des fichiers
- 6 - Création de nouveaux types de données
- 7 - Encapsulation de données et de méthodes
- 8 - Héritage de classes et implémentation d'interfaces
- 9 - Gestion de la durée de vie des objets et contrôle des ressources
- 10 - Encapsulation avancée
- 11 - Découplage de méthodes et gestion
- 12 - Utilisation des collections et c
- 13 - Programmation asynchrone et
- 14 - Utilisation de LINQ pour interroger des données
- 15 - Développement dirigé par les Tests

Objectifs :

- Maîtriser les bases de
- Savoir construire des requêtes
- Comprendre l'interaction de LINQ avec d'autres
- Optimiser les requêtes LINQ

Objectifs :

- 1.Maîtriser les bases de LINQ :** Comprendre le rôle et les avantages de LINQ dans le cadre de la manipulation et de la requête de données.
- 2.Savoir construire des requêtes LINQ :** Apprendre à écrire des requêtes LINQ efficaces pour extraire, filtrer, et transformer des données à partir de sources diverses.
- 3.Comprendre l'interaction de LINQ avec d'autres structures :** Étudier comment LINQ interagit avec les collections, les bases de données, et d'autres structures de données.
- 4.Optimiser les requêtes LINQ :** Évaluer les performances et optimiser les requêtes pour garantir une extraction de données rapide et efficace.

Positionnement :

LINQ, ou "Language Integrated Query", représente une avancée majeure dans la manière dont les développeurs .NET peuvent manipuler et interroger des données. Plutôt que de jongler entre différents paradigmes pour différentes sources de données, LINQ offre une approche unifiée, puissante et expressive. Ce chapitre se positionne comme une étape essentielle pour tout développeur souhaitant maîtriser l'art de la manipulation de données en C#.

Sujets clés couverts :

1.Introduction à LINQ :

1. Présentation de LINQ et son rôle dans le framework .NET.
2. Avantages de l'utilisation de LINQ par rapport aux méthodes traditionnelles.

2. Syntaxe des requêtes LINQ :

1. Comprendre la différence entre la syntaxe de requête et la syntaxe de méthode.
2. Exploration des mots-clés tels que from, select, where.

3.Types de sources de données :

1. Utilisation de LINQ avec des collections en mémoire, tels que les listes et les tableaux.
2. Interrogation de bases de données avec LINQ to SQL ou Entity Framework.
3. Requête de données XML avec LINQ to XML.

4.Opérations avancées avec LINQ :

1. Tri, groupement, et agrégation des données.
2. Joins, group joins, et d'autres techniques pour combiner des données.

5.Optimisation et performances :

1. Comprendre le comportement "lazy" de LINQ.
2. Techniques pour améliorer les performances des requêtes LINQ.



**Commençons
par échanger**



"Avez-vous déjà vécu une situation où un petit changement dans le code a provoqué des bugs inattendus ailleurs dans votre application ? Comment avez-vous géré cela ?"

"Selon vous, quels sont les critères essentiels pour juger de la qualité d'un code ? Avez-vous des méthodes ou des pratiques que vous adoptez régulièrement pour assurer cette qualité ?"

"Avez-vous déjà écrit des tests pour votre code ? Si oui, comment les avez-vous intégrés dans votre processus de développement ? Les écrivez-vous avant, pendant ou après le développement de la fonctionnalité ?"

"Pouvez-vous citer des avantages de l'automatisation des tests dans un projet ? Comment pensez-vous que cela pourrait impacter la livraison et la maintenance d'une application ?"

"Comment définiriez-vous habituellement le cycle de développement d'une fonctionnalité dans vos projets actuels ? Quelle place pourraient avoir les tests dans ce cycle ?"

Vue d'ensemble

- La place des tests dans le développement
- Modèles de conception d'application : MVC, MVVM
- Tests Unitaires et Visual Studio 2022

Leçon 1: La place des tests dans le développement

- Méthodologies de développement
- Tests en développement

Méthodologies de développement (1/2)

- Approche Traditionnelle
 - En cascade ou en V
 - Processus relativement linéaire
 - Peu évolutif, les besoins doivent être parfaitement connus dès le départ

Méthodologies de développement (2/2)

- Approche Agile
 - Basé sur des cycles courts
 - Processus répétitif et évolutif
 - Les méthodes les plus répandues: Adaptative Software Development, eXtreme Programming, Scrum...
 - Technique souvent couplée à l'approche: Développement Dirigé par les Tests (TDD)
 - Un test est écrit
 - Le test échoue
 - Le code est écrit pour que le test réussisse
 - Le code est nettoyé
 - Un nouveau test est écrit...

Tests en développement

- Test unitaire:
 - tester une fonction du système de manière autonome
- Test d'intégration
 - tester un ensemble de fonctions conjointement
- Test système
 - tester le logiciel sous forme de boîte noire dans un environnement compatible
- Test d'acceptation
 - vérifier que le produit est conforme aux spécifications

Leçon 2: Modèles de conception d'application

- Design Pattern: concepts
- MVVM
- MVC
- Bonnes pratiques du modèle MVC

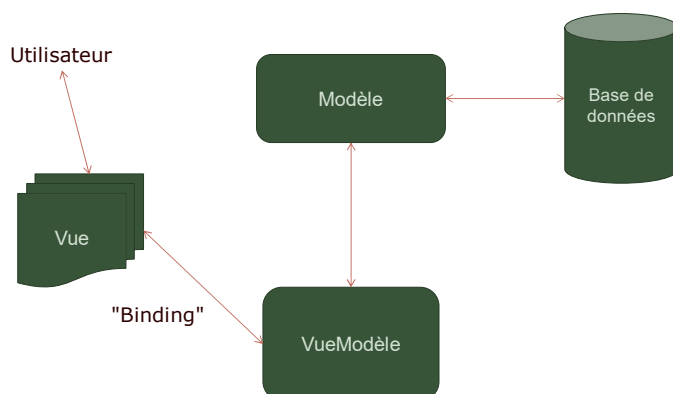
Design Pattern (Patrons de conception)

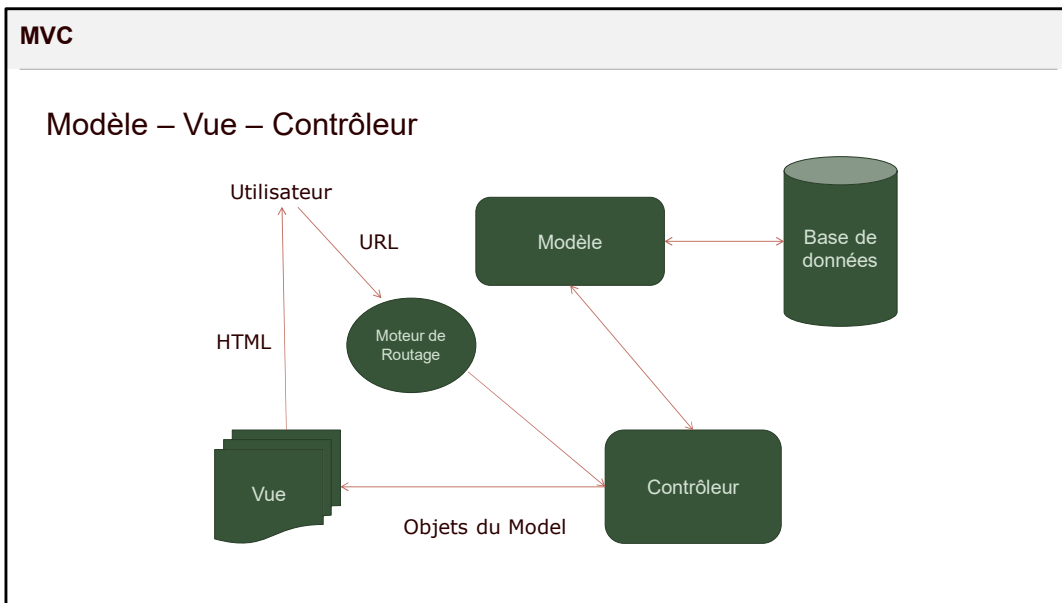
- Ensemble de caractéristiques de module
- Mise en œuvre de bonnes pratiques pour la résolution d'une problématique donnée
- Issus de l'expérience de concepteurs de logiciels
- Exemples:
 - Parmi les 23 patrons du *Gang Of Four**: Factory, Proxy, Singleton, MVC...
 - Repository, Disposable, MVVM ...

* Design Patterns: Elements of Reusable Object-Oriented Software par Gamma E., Helm R., Johnson R. et Vlissides J.

MVVM

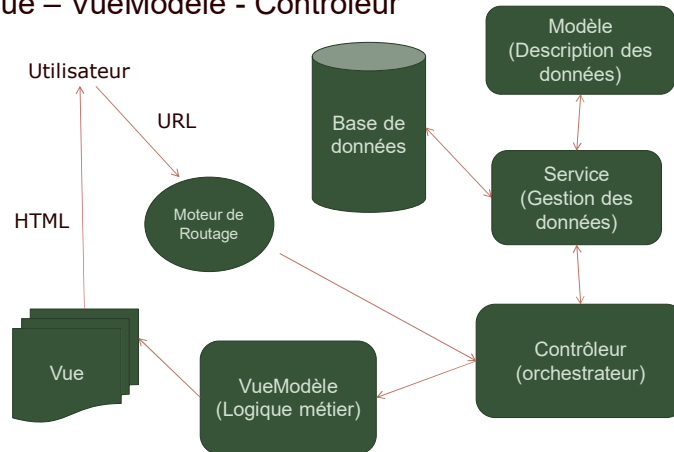
Modèle – Vue - VueModèle





Bonnes pratiques du modèle MVC

Modèle – Vue – VueModèle - Contrôleur



Leçon 3: Tests Unitaires et Visual Studio 2022

- Structure des Tests Unitaires
- Création de Tests dans Visual Studio
- Résultats des Tests dans Visual Studio
- Création de Tests avec .NET CLI
- Résultats des Tests avec .NET CLI
- Démonstration

Structure des Tests Unitaires

- Arrange
 - Préparer le test en initialisant les variables nécessaires
- Act
 - Utiliser les variables pour invoquer la méthode et récupérer le résultat
- Assert
 - Tester le résultat par rapport aux valeurs attendues et signaler la réussite ou l'échec du test

Création de Tests dans Visual Studio

```
int Calculate(int operandOne, int operandTwo)
{
    int result = 0;

    // Un calcul.

    return result;
}
```

Ajout d'un projet
de Test Unitaire

```
[TestMethod()]
public void CalculateTest()
{
    // Arrange
    Program target = new Program();
    int operandOne = 0, operandTwo = 0;
    int expected = 0;
    int actual;
    // Act
    actual = target.Calculate(operandOne, operandTwo);
    // Assert
    Assert.AreEqual(expected, actual);
}
```

Visual Studio 2017
Enterprise Live
Unit Testing:
Validation des tests
au fur et à mesure
de l'écriture du
code

Résultats des Tests dans Visual Studio

Explorateur de tests

Rechercher

Exécuter tout | Exécuter... | Sélection : Tous les tests

✓ Succès tests (4)

- ✓ TestAddition < 1 ms
- ✓ TestOpDroite < 1 ms
- ✓ TestOpGauche 11 ms
- ✓ TestSoustraction < 1 ms

Résumé

Dernière série de tests Succès (Temps total d'exécution 0:00:00,1604251)

✓ 4 tests Succès

Explorateur de tests

Rechercher

Exécuter tout | Exécuter... | Sélection : Tous les tests

✗ Échec tests (1)

- ✗ TestAddition 48 ms

✓ Succès tests (3)

- ✓ TestOpDroite < 1 ms
- ✓ TestOpGauche 13 ms
- ✓ TestSoustraction < 1 ms

TestAddition [Copier tout](#)

Source : UnitTest1.cs ligne 26

✗ Test Échec - TestAddition

Message : Échec de Assert.AreEqual. Attendu : <12>, Réel : <8>.

Temps écoulé : 0:00:00,0480134

StackTrace :

UTClass1.TestAddition()

Création de Tests avec .NET CLI

- Créer le projet de test

```
dotnet new mstest
```

- Référencer le projet à tester

```
dotnet add reference ..\MonProjet\MonProjet.csproj
```

- Ecrire le code nécessaire au test

- Lancer les tests

```
dotnet test
```

Résultats des Tests avec .NET CLI

```
Démarrage de l'exécution de tests, patientez...
Au total, 1 fichiers de test ont correspondu au modèle spécifié.
Réussi! - Failed: 0, Passed: 1, Skipped: 0, Total: 1, Duration: 11 ms- testprj.dll (net5.0)
```

```
Démarrage de l'exécution de tests, patientez...
Au total, 1 fichiers de test ont correspondu au modèle spécifié.
Échoué TestMethod1 [46 ms]
Message d'erreur :
Assert.AreEqual failed. Expected:<150>. Actual:<15>.
Arborescence des appels de procédure :
at testprj.UnitTest1.TestMethod1() in C:\...\UnitTest1.cs:line 15
Échoué! - Failed: 1, Passed: 0, Skipped: 0, Total: 1, Duration: 46 ms- testprj.dll (net5.0)
```

Démonstration

- Création d'un projet de test unitaire



Atelier 01

Activité
20 min

- Exercice 1: Création de Tests Unitaires



A retenir

1. **Primordial dans le Dev** : "Les tests ne sont pas une option ; ils sont essentiels pour garantir la robustesse et la fiabilité d'une application."
2. **Prévention avant la guérison** : "Les tests vous permettent de découvrir les erreurs bien avant que votre application n'atteigne l'utilisateur final, économisant ainsi du temps, de l'argent et de la réputation."
3. **TDD comme philosophie** : "Le TDD (Test Driven Development) n'est pas seulement une technique, c'est une philosophie : écrire des tests avant même le code garantit que le code répond spécifiquement aux besoins définis."
4. **MVC & MVVM** : "MVC et MVVM sont des modèles de conception qui permettent de séparer la logique métier de la présentation. Cela rend non seulement le code plus lisible, mais facilite également les tests."
5. **Autonomie des tests unitaires** : "Un test unitaire doit être autonome et indépendant des autres tests. Il doit tester une seule chose à la fois et le faire de manière cohérente."
6. **VS 2022 & Tests** : "Visual Studio 2022 offre des outils puissants pour écrire, exécuter et déboguer des tests unitaires, simplifiant ainsi l'adoption du TDD."
7. **Rétroaction rapide** : "L'une des plus grandes forces du TDD est la rétroaction rapide qu'il offre. Dès qu'une fonctionnalité est écrite, elle est testée, ce qui permet de corriger immédiatement les erreurs."
8. **Maintenabilité** : "Le code testé est plus facile à maintenir. Si une modification entraîne une régression, les tests le signaleront immédiatement."
9. **Confiance lors des refactorings** : "Avec une suite de tests robuste, refactoring et modification du code deviennent des opérations beaucoup moins risquées, car les tests assurent qu'aucune fonctionnalité n'est brisée."
10. **Document de référence** : "Les tests sont également une forme de documentation. Ils montrent comment le code doit se comporter et peuvent être utilisés pour comprendre les intentions originales du développeur."

Synthèse, échanges



Pour conclure

- L'Organisation : Savoir comment structurer vos équipes et canaux est essentiel pour une communication fluide.
- La Collaboration : Les outils, tels que les tableaux blancs et les sondages, peuvent enrichir vos réunions et discussions.
- La Personnalisation : Adapter Teams à vos besoins, que ce soit à travers les notifications ou les préférences de confidentialité, vous permettra de travailler plus efficacement.
- La Communication : L'utilisation efficace des @mentions et la structuration des conversations garantissent que l'information atteint les bonnes personnes.
- La Résolution de Problèmes : Être proactif et savoir où chercher les solutions est crucial pour minimiser les interruptions.

"Nous avons parcouru un long chemin ensemble aujourd'hui. De la gestion des canaux et des équipes, en passant par des techniques avancées pour rendre les discussions et réunions plus efficaces, jusqu'à la résolution des problèmes courants, notre objectif a été de vous fournir les compétences nécessaires pour devenir un référent solide et efficace pour Microsoft Teams au sein de votre organisation.

N'oubliez pas ces éléments clés :

• **L'Organisation** : Savoir comment structurer vos équipes et canaux est essentiel pour une communication fluide.

• **La Collaboration** : Les outils, tels que les tableaux blancs et les sondages, peuvent enrichir vos réunions et discussions.

• **La Personnalisation** : Adapter Teams à vos besoins, que ce soit à travers les notifications ou les préférences de confidentialité, vous permettra de travailler plus efficacement.

• **La Communication** : L'utilisation efficace des @mentions et la structuration des conversations garantissent que l'information atteint les bonnes personnes.

• **La Résolution de Problèmes** : Être proactif et savoir où chercher les solutions est crucial pour minimiser les interruptions.

Enfin, je vous encourage à continuer à explorer et à apprendre. Teams est un outil évolutif, et Microsoft ajoute régulièrement de nouvelles fonctionnalités. Restez curieux, expérimentez et n'hésitez pas à partager vos découvertes avec vos collègues.



— Retour sur vos attentes de cette formation



Tableau blanc Teams

Revenir sur les postits saisis par les stagiaires dans le Tableau blanc de Teams

